

Play4Change – Aplicação de Aprendizagem Adaptativa para Educação Cívica

Radesh Ilesh Gamanbhai Govind

Orientadores: Nuno Datia
Michel Vorenhout

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

11 de Junho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Play4Change – Aplicação de Aprendizagem Adaptativa para Educação Cívica

A51620 Radesh Ilesh Gamanbhai Govind

Orientadores: Nuno Datia

Michel Vorenhout

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

11 de Junho de 2026

Resumo

Texto do resumo. Breve descrição do projeto, dos resultados importantes e das conclusões: o objetivo é dar ao leitor uma visão global do projeto (não deve exceder uma página).

Utilização de Inteligência Artificial

No desenvolvimento deste projecto foram utilizadas as seguintes ferramentas de inteligência artificial generativa:

- **Google Gemini Flash e Gemini Pro** — utilizados principalmente na fase de investigação e enquadramento: pesquisa de conceitos, leitura e síntese de documentação técnica, e exploração de alternativas de design antes de tomar decisões de arquitectura. Foram também utilizados para *debater* e estudar tópicos novos — funcionando como interlocutor para clarificar dúvidas sobre padrões de domínio e comportamento de APIs externas.
- **Claude Sonnet (Claude Code e Claude Cowork)** — utilizados ao longo de todo o ciclo de desenvolvimento. O Claude Code (interface de linha de comandos) foi o principal auxiliar de programação: geração de esqueletos de código, sugestão e implementação de refactoros, diagnóstico e correcção de bugs, e escrita de testes unitários. O Claude Cowork (interface de desktop) foi utilizado na fase de redacção do relatório: análise de saídas do Claude Code, geração de figuras de diagramas de sequência com matplotlib.
- **Prova de conceitos (*proof of concepts*)** — em várias ocasiões, as ferramentas de IA foram utilizadas para construir protótipos mínimos que permitiram validar a viabilidade técnica de uma abordagem antes de a integrar no projecto principal.

Em todos os casos, o código e o texto gerados foram revistos, iterados, adaptados e validados pelo autor.

Índice

1	Introdução	1
1.1	Contextualização	1
1.2	Motivação	2
1.3	Objetivos	2
1.4	Âmbito e Limitações	3
1.5	Organização do Documento	4
2	Estado da Arte	5
2.1	Aprendizagem Adaptativa e Sistemas de Tutoria Inteligente	5
2.2	Inteligência Artificial na Educação	6
2.2.1	O Problema dos 2 Sigma e a Escalabilidade da Tutoria	6
2.2.2	Feedback Formativo e Aprendizagem Visível	6
2.2.3	Teoria da Carga Cognitiva	7
2.3	Aprendizagem Formal, Informal e <i>Self-Directed Learning</i>	7
2.4	Retrieval-Augmented Generation	8
2.4.1	Arquitetura RAG	8
2.4.2	RAG em Contextos Educativos	8
2.5	Aprendizagem Móvel (<i>Mobile Learning</i>)	9
2.6	Autenticação <i>Passwordless</i>	9
2.7	Síntese Comparativa	10
3	Análise e Requisitos	11
3.1	Identificação dos Utilizadores	11
3.1.1	Administrador	11
3.1.2	Learner	12
3.2	Requisitos Funcionais	12
3.3	Requisitos Não Funcionais	13
3.4	Casos de Uso Principais	14
3.4.1	CU01 — Submeter Conteúdo e Gerar Desafios	14
3.4.2	CU02 — Responder ao Desafio Diário	15

3.4.3	CU03 — Remediação Adaptativa (Struggle Path)	15
4	Desenvolvimento	17
4.1	Arquitetura Geral do Sistema	17
4.1.1	Visão Macro e Estilo Arquitetural	17
4.1.2	Decomposição em Módulos	18
4.1.3	Camadas do Sistema	18
4.1.4	Domínio: Contextos Delimitados	19
4.1.5	Fluxo Geral de Dados	20
4.2	Backend — Spring Server e Pipeline	20
4.2.1	Organização do Projeto	20
4.2.2	Pipeline de Processamento de Pedidos	21
4.2.3	Integração com os Restantes Componentes	24
4.3	Autenticação Passwordless	25
4.3.1	Modelo de Dados e Propriedades Criptográficas	25
4.3.2	Fluxo de Autenticação por Magic Link	26
4.3.3	Ciclo de Vida dos Tokens JWT e Rotação de Refresh Tokens	26
4.3.4	Autorização e Controlo de Acesso	27
4.4	Ingestão e Extração de Conteúdo	29
4.4.1	Extração de Texto de PDF	29
4.4.2	Extração de Texto de URL	30
4.4.3	Limites e Validações	32
4.5	Modelo de IA e Estratégia de Prompting	34
4.5.1	Modelo Escolhido e Justificação	34
4.5.2	Estratégia de Prompting	35
4.5.3	Proteção e Limites da API de IA	38
4.6	RAG — Retrieval-Augmented Generation	43
4.6.1	Armazenamento de Conhecimento	43
4.6.2	Recuperação e Reinjeção de Contexto	44
4.6.3	Edição de Conteúdo e Invalidação	47
4.7	Geração e Atribuição de Desafios	51
4.7.1	Geração de Desafios	51
4.7.2	Atribuição aos Utilizadores	52
4.7.3	Consolidation Path (Struggle Path)	53
4.7.4	Tutoria com IA	54
4.8	Segurança da Aplicação	58
4.8.1	Proteção Contra XSS	58
4.8.2	Rate Limiting por IP	60

4.8.3	Validação e Sanitização de Inputs	60
4.8.4	Outros Controlos de Segurança	62
4.8.5	Análise de Riscos	63
5	Testes e Validação	65
5.1	Estratégia de Testes	65
5.2	Testes Unitários	66
5.2.1	Objetos de Valor — <code>Name</code>	66
5.2.2	Autenticação — <code>TokenService</code> e <code>MagicLinkService</code>	66
5.2.3	Baralhagem Anti-fraude e Cadência de Entrega	67
5.2.4	Gestão de Tópicos e Detecção de Ciclos no DAG	67
5.2.5	Detecção e Escalamento de Dificuldades	68
5.3	Testes de Integração	68
5.3.1	Limitação de Taxa — <code>RateLimitTest</code>	68
5.3.2	Controlo de Acesso ao Swagger	69
5.3.3	Controladores REST	69
5.3.4	Frontend — Vitest + React Testing Library	69
5.4	Testes de Segurança	69
5.4.1	Prevenção de SSRF — <code>UrlSsrValidatorTest</code>	69
5.4.2	Sanitização de Output IA — <code>AiOutputSanitiserTest</code>	70
5.4.3	Força Bruta e Magic Link	70
5.4.4	Anti-fraude e Guarda de Arranque	71
5.5	Testes de Usabilidade	71
5.5.1	Metodologia	71
5.5.2	Análise SUS	71
5.5.3	Análise Qualitativa	72
6	Resultados	75
6.1	Demonstração do Sistema	75
6.1.1	Autenticação por Ligação Mágica	75
6.1.2	Pipeline de Criação de Tópico	75
6.1.3	Entrega e Submissão de Tarefas	76
6.1.4	Sessão de Dificuldade Adaptativa	76
6.1.5	Sessão de Explicação e Tutoria com IA	76
6.1.6	Painel de Administração Web	76
6.2	Métricas de Desempenho	77
6.2.1	Arquitetura de Observabilidade	77
6.2.2	Métricas Instrumentadas	77
6.2.3	Limitação de Taxa por Caminho	77

6.2.4	Pool de Threads de Geração e Circuit Breaker	78
6.2.5	Regras de Alerta Prometheus	78
6.3	Qualidade das Questões Geradas	79
6.3.1	Engenharia de Prompts e Reutilização Adaptativa	79
6.3.2	Deduplicação Semântica com pgvector	80
6.3.3	Classificação de Padrões de Erro	80
6.3.4	Chunking de Conteúdo para Documentos Extensos	80
6.3.5	Sanitização de Saída da IA	81
6.3.6	Garantias de Qualidade das Questões	81
7	Discussão	83
7.1	Análise dos Resultados face aos Objectivos	83
7.2	O que Correu Bem	84
7.3	Limitações e Dificuldades	85
7.4	Comparação com o Estado da Arte	86
7.5	Trabalho Futuro	87
8	Conclusão	91
	Referências	95
A	Exemplos de Prompts Completos	97
A.1	Prompt de Geração de Tópicos e Desafios	97
A.2	Prompt de Análise de Dificuldade e Geração de Subtarefas Adaptativas . . .	99
A.3	Prompt de Explicação Holística e Conversação	101
B	Diagramas Complementares	103
B.1	Diagrama de Sequência — Autenticação Completa (<i>Magic Link</i>)	103
B.2	Diagrama de Sequência — Pipeline Completo de Ingestão e Geração RAG . .	103
B.3	Diagrama Entidade-Relação	103
C	Variáveis de Ambiente	107

Capítulo 1

Introdução

A transformação digital acelerada das últimas décadas alterou profundamente a forma como os cidadãos trabalham, aprendem e participam na sociedade. Neste contexto, a capacidade de utilizar as tecnologias digitais de forma confiante, crítica e responsável tornou-se uma competência fundamental para a vida moderna. A União Europeia reconheceu esta necessidade e estabeleceu, através do *Digital Competence Framework for Citizens* (DigComp), um quadro de referência comum para definir, desenvolver e avaliar as competências digitais dos cidadãos [1]. Na sua versão mais recente, o DigComp 2.2 organiza 21 competências em cinco áreas — literacia de informação e dados; comunicação e colaboração; criação de conteúdo digital; segurança; e resolução de problemas — e constitui hoje a base de políticas de educação digital em toda a Europa.

Paralelamente, a aliança universitária europeia U!REKA (*Urban Research and Education Knowledge Alliance*), da qual o Instituto Superior de Engenharia de Lisboa (ISEL) é membro ativo, tem vindo a promover iniciativas de inovação educativa que respondam aos desafios das cidades do futuro. É neste enquadramento que surge o presente projeto, desenvolvido no âmbito da Licenciatura em Engenharia Informática e de Computadores (LEIC) do ISEL, em colaboração com a Amsterdam University of Applied Sciences (HvA), parceira da rede U!REKA.

1.1 Contextualização

A promoção das competências digitais é uma prioridade estratégica da União Europeia. O *Digital Compass* para 2030 e o subsequente *Digital Decade Policy Programme*, que o transpõe para um quadro legal vinculativo, fixam o objetivo de dotar pelo menos 80% da população europeia de competências digitais básicas até 2030 [2]. O DigComp 2.2 surge como o instrumento de referência para operacionalizar este objetivo, fornecendo um vocabulário comum e um modelo estruturado que pode ser utilizado tanto na formulação de políticas como no desenvolvimento de programas de formação [1].

Contudo, apesar da existência deste quadro de referência, verifica-se uma escassez de

plataformas de aprendizagem que o integrem de forma nativa, que tirem partido das capacidades da Inteligência Artificial (IA) para personalizar a experiência de aprendizagem, e que permitam a gestão autónoma de conteúdos adaptados ao perfil e progresso de cada utilizador.

1.2 Motivação

Em 1984, Benjamin Bloom demonstrou que um aluno com tutoria individual supera em dois desvios padrão (*2 sigma*) o desempenho médio de uma sala de aula tradicional [3]. Durante décadas, este resultado ficou conhecido como o “problema dos 2 sigma”: como proporcionar a cada aluno a qualidade de uma tutoria personalizada a uma escala economicamente viável? A emergência dos modelos de linguagem de grande escala (*Large Language Models*, LLM) reacendeu este debate, ao tornar possível, pela primeira vez, aproximar esse ideal a custo marginal [4].

Paralelamente, a investigação em eficácia do ensino demonstra que o *feedback* imediato e formativo é um dos fatores que mais acelera a aprendizagem [5]. As plataformas de *e-learning* tradicionais falham precisamente neste ponto: os conteúdos são estáticos, a avaliação é uniforme e não adaptativa, e a criação de novo material formativo exige um esforço manual considerável. No domínio específico das competências digitais definidas pelo DigComp 2.2, esta lacuna é ainda mais evidente — não existe, até ao momento, uma plataforma de aprendizagem adaptativa que integre nativamente este referencial europeu e que recorra à IA para personalizar o percurso de cada utilizador.

A disponibilização de técnicas como o *Retrieval-Augmented Generation* (RAG) permite ir além da geração genérica de conteúdo, ancorando as respostas do modelo em conhecimento específico e verificável [6]. A conjugação destas tecnologias com o quadro estruturado do DigComp 2.2 cria a oportunidade de desenvolver uma plataforma que atua na interseção entre a aprendizagem formal — estruturada, orientada por objetivos claros — e a aprendizagem informal — autodirigida, contextualizada e motivada pela necessidade real do utilizador [7].

1.3 Objetivos

O presente projeto tem como objetivo geral conceber, desenvolver e validar o **Play4Change**, uma plataforma de aprendizagem adaptativa centrada no desenvolvimento de competências digitais segundo o referencial DigComp 2.2, com recurso a Inteligência Artificial para a geração automática de conteúdos e personalização dos percursos de aprendizagem.

A plataforma assenta numa arquitetura multi-cliente que serve dois perfis distintos: os **Administradores**, que gerem e criam conteúdos através de um painel web, e os **Learners**, que desenvolvem as suas competências através de uma aplicação móvel nativa, assente no modelo de um desafio diário — tirando partido do dispositivo que toda a gente transporta consigo e consulta regularmente.

De forma mais específica, pretende-se:

- Desenvolver um **cliente web** composto por um painel de administração para gestão de conteúdos e utilizadores, e por uma *landing page* que permite aos Learners descarregar a aplicação móvel;
- Desenvolver um **cliente móvel** para Learners, centrado numa experiência de aprendizagem contínua e diária — um desafio por dia —, tirando partido da ubiquidade do telemóvel para integrar o desenvolvimento de competências digitais na rotina do utilizador;
- Integrar um pipeline de extração e processamento de conteúdo a partir de documentos PDF e páginas web, como base para a geração automática de tópicos e desafios via IA;
- Implementar um sistema de *Retrieval-Augmented Generation* (RAG) que permita ao modelo de IA gerar conteúdo ancorado em conhecimento específico e persistente;
- Conceber um mecanismo de progressão de desafios (*consolidation path*) que adapte o percurso de aprendizagem ao desempenho individual do Learner ao longo do tempo;
- Garantir a segurança da plataforma através de autenticação *passwordless*, proteção contra vulnerabilidades web comuns e controlo de utilização da API de IA.

Estes entregáveis concretizam-se, do ponto de vista funcional, em cinco objetivos operacionais: (1) uma plataforma de aprendizagem adaptativa alinhada com o DigComp 2.2; (2) um *pipeline* de geração de conteúdo por IA a partir de fontes arbitrárias; (3) remediação adaptativa personalizada por padrão de erro; (4) tutoria conversacional com IA como escalamento da remediação; e (5) segurança e qualidade do conteúdo gerado numa aplicação web multi-utilizador. A concretização de cada um destes objetivos é analisada em detalhe no Capítulo 7 (§7.1).

1.4 Âmbito e Limitações

O Play4Change foi desenvolvido no âmbito do projeto *U!REKA SHIFT*, uma iniciativa da aliança europeia U!REKA que visa criar soluções educativas inovadoras para as cidades do futuro, com especial foco na literacia digital dos seus cidadãos. A colaboração entre o ISEL e a HvA enquadra-se precisamente neste objetivo: desenvolver uma ferramenta que possa ser adotada transversalmente pelas instituições parceiras da rede, em diferentes países e contextos culturais, contribuindo para as metas europeias de competências digitais definidas no *Digital Compass* para 2030 [2].

Em termos técnicos, o âmbito do projeto cobre o *backend* da plataforma, o cliente web (painel de administração e *landing page*) e o cliente móvel para Learners.

As principais limitações deste trabalho são: a avaliação pedagógica da eficácia da plataforma a larga escala não foi realizada, sendo o foco do projeto a conceção e implementação técnica do sistema; a integração com sistemas de gestão de aprendizagem externos (LMS) fica reservada para trabalho futuro; e, apesar de a arquitetura Kotlin Multiplatform permitir a compilação nativa para iOS a partir do mesmo código partilhado, o projeto entrega apenas o alvo Android.

1.5 Organização do Documento

O restante relatório encontra-se organizado da seguinte forma: o Capítulo 2 apresenta o estado da arte em plataformas de aprendizagem adaptativa, modelos de linguagem aplicados à educação, RAG e autenticação *passwordless*; o Capítulo 3 descreve a análise de utilizadores, requisitos funcionais e não funcionais e os principais casos de uso; o Capítulo 4 detalha a arquitetura e implementação de todos os componentes do sistema; o Capítulo 5 apresenta a estratégia e os resultados dos testes realizados; o Capítulo 6 expõe os resultados e demonstra o sistema em funcionamento; o Capítulo 7 discute os resultados face aos objetivos e ao estado da arte; e o Capítulo 8 apresenta as conclusões e perspetivas de trabalho futuro.

Capítulo 2

Estado da Arte

Este capítulo apresenta o enquadramento teórico e tecnológico do Play4Change, situando o projeto no panorama atual da investigação e das soluções existentes. São abordados seis eixos fundamentais: os sistemas de aprendizagem adaptativa, o papel da Inteligência Artificial na educação, a distinção entre aprendizagem formal e informal, a técnica de *Retrieval-Augmented Generation*, a aprendizagem móvel e a autenticação *passwordless*. O capítulo termina com uma síntese comparativa que posiciona o Play4Change face às soluções existentes.

2.1 Aprendizagem Adaptativa e Sistemas de Tutoria Inteligente

A ideia de adaptar o ensino ao ritmo e ao perfil individual de cada aluno não é nova. Em meados do século XX, Skinner propôs as primeiras “máquinas de ensinar” (*teaching machines*), baseadas no princípio de que o reforço imediato da resposta correta é essencial para a consolidação da aprendizagem [8]. Esta ideia evoluiu, nas décadas seguintes, para os *Intelligent Tutoring Systems* (ITS) — sistemas computacionais capazes de modelar o estado de conhecimento do aluno e adaptar dinamicamente a sequência de instrução [9].

Os ITS modernos assentam tipicamente em três módulos: o **modelo de domínio** (o que há para aprender), o **modelo do estudante** (o que o aluno já sabe) e o **módulo pedagógico** (como adaptar a instrução) [10]. O modelo do estudante é frequentemente implementado através de técnicas como o *Knowledge Tracing* (KT), que infere o estado latente de conhecimento de um aluno a partir do seu histórico de respostas [11]. Versões mais recentes, baseadas em redes neuronais recorrentes (*Deep Knowledge Tracing*), demonstraram melhorias significativas na precisão desta modelação [12].

No mercado atual, plataformas como o **Duolingo**, o **Khan Academy** e o **Coursera** incorporam princípios de adaptatividade, ainda que em graus distintos. O Duolingo é pioneiro na aplicação do modelo de um exercício diário no canal móvel, e um estudo encomendado pela própria empresa reportou resultados comparáveis a um semestre universitário para as competências de leitura e audição [13]. Esta afirmação deve ser lida com reserva: o estudo

não foi revisto por pares, teve uma amostra pequena e auto-selecionada de utilizadores já activos na aplicação, sem grupo de controlo independente, e a conclusão específica de equivalência a um semestre foi publicamente contestada por outros investigadores da área [14]. No entanto, estas plataformas partilham uma limitação comum: o conteúdo é estático, curado manualmente por equipas especializadas, e não pode ser facilmente adaptado por terceiros para domínios específicos. O Play4Change distingue-se precisamente neste ponto: qualquer Administrador pode introduzir novo conhecimento, que é automaticamente transformado em conteúdo pedagógico pela IA.

2.2 Inteligência Artificial na Educação

A aplicação da Inteligência Artificial à educação (*AI in Education*, AIEd) tem uma história de várias décadas, mas a emergência dos modelos de linguagem de grande escala (*Large Language Models*, LLM) introduziu uma mudança qualitativa sem precedentes nas possibilidades desta área.

2.2.1 O Problema dos 2 Sigma e a Escalabilidade da Tutoria

Em 1984, Benjamin Bloom conduziu uma investigação que se tornaria uma das mais citadas na história da psicologia educacional. Bloom demonstrou que alunos com tutoria individual (relação 1-para-1 com um tutor) superaram em dois desvios padrão (*2 sigma*) os alunos sujeitos ao ensino tradicional em sala de aula [3]. Este resultado implica que o aluno médio com tutoria individual supera 98% dos alunos de uma turma convencional.

Durante décadas, o “problema dos 2 sigma” ficou sem resposta: a tutoria individual é eficaz, mas economicamente inviável a larga escala. Os LLMs reabriram este debate de forma concreta [4]. A possibilidade de ter um modelo de linguagem disponível 24 horas por dia, capaz de responder a questões em linguagem natural, adaptar a complexidade da explicação ao nível do utilizador e fornecer *feedback* imediato e contextualizado, aproxima pela primeira vez o ideal da tutoria personalizada de uma escala de implementação massiva [15].

2.2.2 Feedback Formativo e Aprendizagem Visível

Independentemente da tecnologia utilizada, a investigação em eficácia pedagógica converge num ponto: o *feedback* imediato e formativo é um dos fatores que mais acelera a aprendizagem. Na sua meta-análise de mais de 800 estudos sobre fatores de eficácia no ensino, Hattie (2009) coloca o *feedback* entre os dez maiores aceleradores da aprendizagem, com um tamanho de efeito (*effect size*) de $d = 0,73$ [5]. Convém notar que este valor é uma média agregada de centenas de estudos com desenhos, populações e escalas de medição distintos; a metodologia de agregação de Hattie foi criticada por outros investigadores como estatisticamente pouco robusta e potencialmente enganadora quando apresentada como um

número único e preciso [16]. Ainda assim, a direção geral do efeito — feedback imediato acelera a aprendizagem — é consensual na literatura, mesmo que a sua magnitude exata seja discutível. Este princípio é relevante no contexto do Play4Change: ao fornecer *feedback* explicativo imediatamente após cada desafio diário, a plataforma incorpora um mecanismo pedagógico amplamente (ainda que não precisamente) suportado pela evidência.

2.2.3 Teoria da Carga Cognitiva

A *Cognitive Load Theory* (CLT), desenvolvida por Sweller [17], postula que a memória de trabalho humana tem uma capacidade limitada e que a aprendizagem é mais eficaz quando a instrução é desenhada para não sobrecarregar esta capacidade. A CLT distingue três tipos de carga cognitiva: *intrínseca* (complexidade inerente ao conteúdo), *extrínseca* (causada por má apresentação da informação) e *germane* (esforço cognitivo dedicado à formação de esquemas de conhecimento). O modelo de um desafio curto por dia e por tópico é consistente com os princípios da CLT: minimiza a carga extrínseca dentro desse tópico e permite que o esforço cognitivo do Learner seja dirigido à assimilação do conteúdo. Esta propriedade aplica-se por inscrição: o sistema permite que um Learner esteja simultaneamente inscrito em vários tópicos activos (Secção 4.7.2 do Capítulo 4), cada um com o seu próprio desafio diário independente. Um Learner inscrito em vários tópicos em simultâneo recebe, portanto, vários desafios no mesmo dia — o que dilui parcialmente o argumento da carga cognitiva mínima para esse perfil de utilização, e é uma tensão de desenho que a versão actual do sistema não resolve explicitamente (por exemplo, limitando o número de inscrições activas em simultâneo).

2.3 Aprendizagem Formal, Informal e *Self-Directed Learning*

A distinção entre aprendizagem formal e informal é central para compreender o posicionamento do Play4Change.

A **aprendizagem formal** caracteriza-se por ser institucionalizada, estruturada, orientada por currículos predefinidos e avaliada por certificações reconhecidas [18]. Ocorre tipicamente em contextos de ensino superior, formação profissional obrigatória ou programas de certificação. A motivação é frequentemente extrínseca: aprovação em unidades curriculares, obtenção de diplomas ou progressão na carreira.

A **aprendizagem informal**, por contraste, é não estruturada, autodirigida e motivada pela necessidade imediata ou pela curiosidade intrínseca do aprendente [7]. Malcolm Knowles, pioneiro no estudo da aprendizagem de adultos (*andragogia*), demonstrou que os adultos aprendem de forma mais eficaz quando a aprendizagem é orientada para a resolução de problemas reais, quando têm controlo sobre o processo e quando o conhecimento é imediatamente aplicável [19]. Jay Cross, que popularizou o conceito de aprendizagem informal nas organizações, estimou que 80% do conhecimento prático utilizado no trabalho é adquirido de

forma informal [7].

O Play4Change opera deliberadamente na interseção destes dois paradigmas. Por um lado, está ancorado no referencial estruturado do DigComp 2.2, o que confere à experiência de aprendizagem um enquadramento formal e objetivos claramente definidos. Por outro lado, o modelo de utilização — um desafio diário no telemóvel, sem horários fixos, sem avaliações sumativas e sem a pressão do contexto institucional — é intrinsecamente informal e autodirigido, respondendo diretamente aos princípios da andragogia de Knowles.

2.4 Retrieval-Augmented Generation

Os LLMs apresentam uma limitação estrutural relevante para aplicações de domínio específico: o seu conhecimento é estático, limitado à data de treino, e não pode ser facilmente atualizado sem re-treino do modelo, um processo computacionalmente dispendioso. O *Retrieval-Augmented Generation* (RAG) surgiu como resposta a esta limitação, combinando a capacidade generativa dos LLMs com a consulta dinâmica de bases de conhecimento externas [6].

2.4.1 Arquitetura RAG

Na sua formulação original, proposta por Lewis et al. (2020) em contexto de tarefas de *question answering*, o RAG funciona em duas fases [6]:

1. **Fase de recuperação (*retrieval*):** dado um *query*, o sistema recupera os documentos ou fragmentos de texto (*chunks*) mais relevantes de uma base de conhecimento indexada vetorialmente, tipicamente utilizando similaridade cosseno entre *embeddings*.
2. **Fase de geração:** os *chunks* recuperados são injetados no contexto do LLM, que gera uma resposta fundamentada nessa informação específica.

Esta arquitetura apresenta vantagens significativas em relação ao uso direto de um LLM: reduz as *hallucinations* (respostas factualmente incorretas geradas com confiança), permite que o sistema responda com base em conhecimento privado ou recente, e torna as respostas rastreáveis às fontes originais [20].

2.4.2 RAG em Contextos Educativos

A aplicação do RAG a sistemas educativos é uma área de investigação em crescimento acelerado. A capacidade de ancorar a geração de questões e *feedback* em documentos específicos fornecidos pelo formador — e não no conhecimento genérico do modelo — é particularmente valiosa em contextos de formação especializada, onde a precisão e a especificidade do conteúdo são críticas [4]. No Play4Change, o RAG é o mecanismo que garante que os desafios gerados

para cada Learner estão diretamente ancorados no conteúdo submetido pelo Administrador, e não em conhecimento genérico do modelo.

2.5 Aprendizagem Móvel (*Mobile Learning*)

A aprendizagem móvel (*m-learning*) define-se como qualquer modalidade de aprendizagem que ocorre com mediação de dispositivos móveis e que tira partido da portabilidade, da conectividade e da ubiquidade destes dispositivos [21].

Laurillard (2007) argumenta que o telemóvel transforma o modelo de aprendizagem de um evento programado (ir a uma aula, sentar à secretária) para um processo contínuo e contextualizado, integrado nos interstícios da vida quotidiana — nos transportes públicos, nas pausas de trabalho, antes de dormir [22]. Esta ubiquidade é central para o modelo do Play4Change: ao entregar um desafio diário no telemóvel, a plataforma não compete com o tempo disponível para aprendizagem — integra-se nos momentos em que o utilizador já usa o dispositivo.

Com mais de 5 mil milhões de utilizadores de telemóvel no mundo [23], o canal móvel é também o mais democrático e inclusivo disponível, reduzindo as barreiras de acesso frequentemente associadas à aprendizagem formal. Esta dimensão é especialmente relevante no contexto do DigComp 2.2, cujo objetivo declarado é o desenvolvimento de competências digitais para *todos* os cidadãos, e não apenas para aqueles com acesso a instituições de ensino formal [1].

2.6 Autenticação *Passwordless*

A autenticação por *password* é, há décadas, o mecanismo dominante de controlo de acesso em sistemas web. Contudo, a investigação em segurança acumulou evidência consistente de que as *passwords* constituem o elo mais fraco da cadeia de segurança: são reutilizadas entre serviços, são sujeitas a ataques de força bruta, de *phishing* e de roubo em *data breaches*, e são frequentemente esquecidas pelos utilizadores [24]. Bonneau et al. (2012), num estudo seminal apresentado no IEEE Symposium on Security and Privacy, avaliaram 35 esquemas alternativos de autenticação e concluíram que nenhum supera as *passwords* em todos os critérios simultaneamente, mas que os esquemas *passwordless* oferecem vantagens claras em usabilidade e resistência a ataques de *phishing* [24].

A autenticação *passwordless* elimina a *password* do fluxo de autenticação, substituindo-a por um factor de posse (*magic link* enviado por email, *One-Time Password* (OTP) via SMS ou aplicação) ou por um factor biométrico (FIDO2/*Passkeys*) [25]. No contexto do Play4Change, a adoção de autenticação *passwordless* serve dois objetivos complementares: reduz a fricção de entrada na plataforma para o Learner (que não precisa de memorizar uma *password* para uma utilização diária breve) e elimina um vetor de ataque relevante numa

aplicação que lida com dados de progressão de aprendizagem.

2.7 Síntese Comparativa

A Tabela 2.1 apresenta uma análise comparativa entre o Play4Change e as principais plataformas de referência identificadas no estado da arte, segundo os critérios mais relevantes para o posicionamento do projeto.

Tabela 2.1: Comparação entre o Play4Change e plataformas de referência

Critério	Duolingo	Khan Academy	Coursera	Play4Change	
				Atual	Futuro
Gamificação	✓	~	×	~	✓
Adaptação à dificuldade do utilizador	✓	✓	×	✓	✓
Catálogo de conteúdo pré-definido	✓	✓	✓	×	~
Reconhecimento com credenciais formais	~	×	✓	×	✓
Comunidade e interação social	✓	~	✓	×	✓
Modelo de desafio diário no móvel	✓	×	×	✓	✓
Cliente móvel nativo	✓	✓	✓	✓	✓
Autenticação <i>passwordless</i>	×	×	×	✓	✓

✓ Suportado × Não suportado ~ Parcialmente suportado

A análise revela um posicionamento diferenciado para o Play4Change, mas também expõe com clareza as suas limitações face às plataformas estabelecidas.

No conjunto de critérios onde as plataformas existentes lideram, destacam-se a gamificação — especialmente no Duolingo, com sistemas de *streaks*, ligas e pontuação — e o reconhecimento formal através de certificados universitários, área em que o Coursera é referência. A ausência de um catálogo de conteúdo pré-definido é também uma limitação real do Play4Change: o sistema depende do conteúdo submetido pelo Administrador, o que exige um esforço de curadoria inicial por parte das instituições que o adotem. A comunidade e a interação social entre aprendentes, presentes em graus variados no Duolingo e no Coursera, são igualmente dimensões não cobertas na versão atual. Estas lacunas são reconhecidas e constituem direções prioritárias de trabalho futuro, nomeadamente a introdução de um sistema de gamificação mais completo e de mecanismos de reconhecimento de competências.

O Play4Change diferencia-se das plataformas analisadas pelo seu modelo de entrega: um desafio diário por tópico no canal móvel, com adaptação ao desempenho do utilizador e autenticação *passwordless* que elimina a fricção de acesso. Nenhuma das plataformas existentes combina estas três características numa única solução orientada ao desenvolvimento de competências digitais estruturadas. Note-se que o critério “Modelo de desafio diário no móvel” da Tabela 2.1 descreve o padrão de utilização pretendido por tópico; como discutido na Secção 2.1, um Learner com múltiplas inscrições activas em simultâneo recebe múltiplos desafios diários, pelo que a garantia de um único desafio por dia não é, presentemente, uma propriedade imposta pelo sistema.

Capítulo 3

Análise e Requisitos

A definição clara dos requisitos de um sistema é um dos fatores mais determinantes para o sucesso de um projeto de engenharia de software [26]. Este capítulo apresenta a análise dos utilizadores da plataforma Play4Change, os requisitos funcionais e não funcionais identificados, e os principais casos de uso que guiaram as decisões de desenho e implementação.

3.1 Identificação dos Utilizadores

A identificação dos utilizadores é o ponto de partida de qualquer processo de desenvolvimento centrado no utilizador (*User-Centered Design*) [27]. No contexto do Play4Change, a análise levou à identificação de dois perfis de utilizador com necessidades, objetivos e contextos de uso distintos.

3.1.1 Administrador

O **Administrador** é o perfil responsável pela criação e gestão de conteúdo pedagógico na plataforma. Trata-se tipicamente de um formador, docente ou responsável de uma instituição parceira da rede U!REKA, com competência técnica suficiente para selecionar e carregar materiais de referência, mas que não necessita de expertise em IA ou em engenharia de software para operar o sistema.

O Administrador interage com a plataforma exclusivamente através do **cliente web**, utilizando o painel de administração para submeter documentos PDF ou endereços URL como fontes de conhecimento, para rever e validar os tópicos e desafios gerados automaticamente pela IA, e para gerir os Learners registados na plataforma.

A separação entre este perfil e o Learner é fundamental: o Administrador opera numa lógica de produção de conteúdo, enquanto o Learner opera numa lógica de consumo e progressão. Esta distinção é consistente com os modelos de sistemas de tutoria inteligente (*Intelligent Tutoring Systems*, ITS), nos quais a camada de gestão de conhecimento é mantida separada da camada de interação com o aprendiz [9].

3.1.2 Learner

O **Learner** é o destinatário da experiência de aprendizagem. O seu perfil é deliberadamente amplo: qualquer cidadão que pretenda desenvolver as suas competências digitais no referencial DigComp 2.2, independentemente do nível de partida ou do contexto institucional em que se insere.

O Learner interage com a plataforma através do **cliente móvel**, numa experiência desenhada para ser discreta, consistente e integrada na rotina diária. O modelo de um **desafio por dia** não é uma escolha arbitrária: a investigação em psicologia da aprendizagem demonstra que sessões curtas e regulares são significativamente mais eficazes do que sessões longas e esporádicas — um fenómeno conhecido como *spaced repetition* [28, 29]. Adicionalmente, a Teoria da Autodeterminação (*Self-Determination Theory*, SDT) de Deci e Ryan demonstra que a motivação intrínseca — e, consequentemente, a persistência na aprendizagem — é sustentada quando o utilizador experimenta sentido de *autonomia*, de *competência crescente* e de *ligação* a um propósito maior [30]. O modelo de um desafio diário no telemóvel, entregue no dispositivo que o utilizador já transporta e consulta regularmente, responde diretamente a estes três fatores.

A escolha do telemóvel como plataforma principal para o Learner é também suportada pela investigação em *mobile learning* (m-learning): Traxler (2007) argumenta que a aprendizagem móvel permite que o conhecimento seja adquirido *in situ* e no momento de necessidade, reduzindo a fricção entre a intenção de aprender e a ação de aprender [21]. Com mais de 5 mil milhões de utilizadores de telemóvel no mundo, o canal móvel é o mais democrático e ubíquo disponível [23].

3.2 Requisitos Funcionais

Os requisitos funcionais descrevem os comportamentos que o sistema deve exibir em resposta a determinadas entradas ou condições [26]. A Tabela 3.1 apresenta os requisitos funcionais do Play4Change, organizados por área funcional.

Tabela 3.1: Requisitos funcionais do sistema Play4Change

ID	Área	Descrição
RF01	Autenticação	O sistema deve suportar autenticação para ambos os perfis de utilizador.
RF02	Ingestão	O Administrador deve poder submeter documentos PDF como fonte de conhecimento.
RF03	Ingestão	O Administrador deve poder submeter URLs como fonte de conhecimento.
RF04	IA	O sistema deve gerar automaticamente tópicos a partir do conteúdo submetido pelo Administrador.
RF05	IA	O sistema deve gerar automaticamente desafios (questões) a partir dos tópicos identificados.
RF06	RAG	O sistema deve armazenar o conhecimento extraído em forma de <i>embeddings</i> e recuperá-lo para contextualizar a geração de conteúdo pela IA.
RF07	RAG	A regeneração completa de um tópico pelo Administrador deve invalidar e recalcular os <i>embeddings</i> associados; a edição pontual de uma tarefa individual pode aceitar um vetor desactualizado quando o custo de recalcular em cada edição não se justifica (ver Secção 4.6.3).
RF08	Desafios	O sistema deve atribuir um desafio diário a cada Learner, de acordo com o seu percurso de progressão.
RF09	Desafios	O Learner deve receber <i>feedback</i> imediato após responder a cada desafio.
RF10	Consolidação	O sistema deve implementar um percurso de remediação (<i>struggle path</i>) que gere sub-tarefas adaptadas ao padrão de erro do Learner numa tarefa específica, reutilizando contextos de dificuldade semelhantes já resolvidos anteriormente.
RF11	Cliente Web	O sistema deve disponibilizar um painel de administração acessível via browser.
RF12	Cliente Web	O sistema deve disponibilizar uma <i>landing page</i> pública com informação sobre a plataforma e ligação para descarregar a aplicação móvel.
RF13	Cliente Móvel	O sistema deve disponibilizar uma aplicação móvel para os Learners, centrada na experiência de um desafio diário.

3.3 Requisitos Não Funcionais

Os requisitos não funcionais definem as propriedades e restrições do sistema que não dizem respeito a comportamentos específicos, mas à qualidade global da solução [31]. A Tabela 3.2 apresenta os requisitos não funcionais identificados para o Play4Change.

Tabela 3.2: Requisitos não funcionais do sistema Play4Change

ID	Categoria	Descrição
RNF01	Segurança	O sistema deve proteger os utilizadores contra ataques <i>Cross-Site Scripting</i> (XSS), removendo ou codificando HTML potencialmente malicioso nos conteúdos gerados por IA antes de os persistir ou renderizar.
RNF02	Segurança	O sistema deve aplicar <i>rate limiting</i> por endereço IP para prevenir abusos nos endpoints públicos.
RNF03	Segurança	O sistema deve aplicar <i>rate limiting</i> por endereço IP nas chamadas à API de IA, para controlo de custos e prevenção de abusos.
RNF04	Segurança	A comunicação entre clientes e servidor deve ser efetuada exclusivamente via HTTPS.
RNF05	Usabilidade	A aplicação móvel deve seguir os princípios de <i>mobile-first design</i> , com interações mínimas para completar o desafio diário.
RNF06	Escalabilidade	A arquitetura do sistema deve permitir o aumento do número de utilizadores sem alterações estruturais ao domínio ou à aplicação; componentes de estado local por processo (e.g. o <i>cache</i> de <i>rate limiting</i>) são uma limitação conhecida a endereçar antes de escalar horizontalmente.
RNF07	Manutenibilidade	O código deve seguir os princípios SOLID e ser coberto por testes unitários e de integração.

3.4 Casos de Uso Principais

Os casos de uso descrevem as interações entre os atores e o sistema para atingir um objetivo específico [32]. São apresentados de seguida os três casos de uso centrais do Play4Change.

3.4.1 CU01 — Submeter Conteúdo e Gerar Desafios

Ator principal: Administrador.

Pré-condição: O Administrador está autenticado no painel web.

Fluxo principal:

1. O Administrador define o tópico (título, descrição e metadados) e submete um documento PDF ou um URL como fonte de conhecimento; o sistema cria de imediato o tópico em estado **PENDING**.
2. De forma assíncrona, o sistema extrai e limpa o texto do documento ou página.
3. O sistema divide o texto em *chunks* e invoca o modelo de IA para gerar módulos e os respetivos desafios (*task templates*) a partir do conteúdo extraído.

4. Por cada desafio gerado, o sistema calcula o *embedding* do par título+descrição, verifica duplicação semântica face aos desafios já existentes no módulo, e persiste o resultado.
5. Concluída a geração, o tópico transita automaticamente para o estado **ACTIVE**, sem necessidade de aprovação explícita do Administrador; este pode, a qualquer momento, editar um desafio específico ou desencadear a regeneração completa do tópico.

Pós-condição: O tópico está **ACTIVE**, com os desafios e o conhecimento associado indexados e disponíveis para atribuição aos Learners.

3.4.2 CU02 — Responder ao Desafio Diário

Ator principal: Learner.

Pré-condição: O Learner está autenticado na aplicação móvel e tem um desafio diário atribuído.

Fluxo principal:

1. O Learner abre a aplicação móvel e visualiza o desafio do dia.
2. O sistema seleciona e apresenta o desafio correspondente ao seu percurso actual, já gerado antecipadamente pela IA durante o pipeline de ingestão do tópico — não há recuperação vetorial em tempo real neste passo.
3. O Learner submete a sua resposta.
4. O sistema avalia a resposta e apresenta *feedback* imediato, com explicação contextualizada.
5. O sistema regista o resultado e atualiza o perfil de progressão do Learner.

Pós-condição: O resultado é registado; se a resposta estiver incorreta, é desencadeada a sessão de remediação descrita em CU03.

3.4.3 CU03 — Remediação Adaptativa (Struggle Path)

Ator principal: Learner.

Pré-condição: O Learner respondeu incorretamente a um desafio do dia.

Fluxo principal:

1. O sistema classifica deterministicamente o padrão do erro (por exemplo, atraso na submissão ou opção adjacente à correta) com base no contexto da submissão.
2. De forma assíncrona, o sistema gera três sub-tarefas mais simples, centradas no padrão de erro detetado, reutilizando um ramo já gerado para um contexto semelhante sempre que a similaridade semântica o permita.

3. O Learner resolve as sub-tarefas; se as completar todas corretamente, a sessão é marcada como resolvida e o desafio original do dia é repostado para nova tentativa.
4. Se o Learner voltar a falhar após esgotar a profundidade de remediação definida, o sistema substitui as sub-tarefas por uma sessão de explicação sintetizada por IA, com possibilidade de diálogo de esclarecimento.

Pós-condição: O Learner é conduzido de volta ao desafio original, com uma remediação centrada no seu erro específico; esta adaptação ocorre dentro do próprio desafio do dia, não como uma escolha de que tópico apresentar em dias seguintes — essa seleção continua a ser feita pelo próprio Learner, através de inscrição explícita nos tópicos disponíveis.

Capítulo 4

Desenvolvimento

O Play4Change é uma plataforma de aprendizagem adaptativa composta por três superfícies cliente (Android/iOS/Web), um serviço de backend e um módulo de geração por Inteligência Artificial, orquestrados através de uma infraestrutura totalmente contida em *containers*. Este capítulo descreve em detalhe cada componente do sistema, as decisões de arquitetura que o estruturam e as tecnologias escolhidas para as concretizar.

4.1 Arquitetura Geral do Sistema

4.1.1 Visão Macro e Estilo Arquitetural

O backend do Play4Change adopta a **Arquitetura Hexagonal** (*Ports & Adapters*), organizada dentro de um modelo de contextos delimitados (*Bounded Contexts*) de *Domain-Driven Design* (DDD) [33]. Esta escolha não é convencional — é estruturalmente exigida pela natureza do problema: a complexidade do sistema reside no domínio (geração adaptativa de tarefas, detecção de dificuldade, percurso de consolidação), e essa lógica tem de permanecer completamente isolada de detalhes de persistência, de protocolo HTTP ou do fornecedor específico de LLM.

A arquitetura organiza-se em três anéis concêntricos com dependências que fluem sempre para o interior:

- **Anel de domínio:** classes Kotlin puras, sem qualquer anotação de *framework*. Contém as entidades, os agregados, os eventos de domínio e as interfaces de *port*.
- **Anel de aplicação:** casos de uso que orquestram o domínio, chamando exclusivamente interfaces de *port* — nunca implementações concretas.
- **Anel de infraestrutura:** adaptadores de entrada (*REST controllers*, filtros de segurança JWT) e adaptadores de saída (JPA, Resend, LangChain4j, MinIO), todos injectados pelo contentor Spring em tempo de execução.

Uma consequência prática desta separação: o domínio e a aplicação nunca referenciam LangChain4j directamente, pelo que substituir o fornecedor de LLM de Mistral para outro

modelo não exige alterar nenhuma linha de lógica de negócio — exige apenas uma nova implementação de `TaskGenerationPort` anotada com `@Primary` e, ao nível do *build*, trocar a dependência de `ai-agent:langchain` pelo módulo Gradle correspondente ao novo adaptador.

4.1.2 Decomposição em Módulos

O sistema está decomposto em seis módulos com regras de dependência estritas, apresentados na Tabela 4.1.

Tabela 4.1: Módulos do sistema Play4Change e responsabilidades

Módulo	Tecnologia	Responsabilidade
<code>server</code>	Kotlin, Spring Boot 3, JVM 21	API REST, lógica de domínio e aplicação, adaptadores de infraestrutura
<code>ai-agent:api</code>	Kotlin puro	Contratos de <i>port</i> para geração IA — sem implementação nem dependências de <i>runtime</i>
<code>ai-agent:langchain</code>	Kotlin, LangChain4j, Mistral	Adaptador IA: geração de tarefas, ramificação adaptativa, síntese de explicações
<code>composeApp</code>	Kotlin Multiplatform, Compose MP	Cliente móvel Android e iOS — arquitectura MVI via Decompose
<code>common</code>	Kotlin Multiplatform	Objectos de valor partilhados, tipos de erro, <i>result wrappers</i> — sem dependências de <i>framework</i>
<code>play4change-web</code>	React 18, TypeScript, Vite, TanStack	SPA de administração — gestão de tópicos, utilizadores e métricas

A regra de dependência é rigorosa ao nível do código: `common` não depende de nenhum módulo; `ai-agent:api` depende apenas de `common`; o **domínio e a aplicação** do `server` referenciam apenas a interface `TaskGenerationPort` de `ai-agent:api`, nunca `LangChain4j` directamente — a implementação concreta é injectada em *runtime* pelo mecanismo de `@Primary` do Spring. O módulo Gradle `:server` mantém, ainda assim, uma dependência de *build* em `ai-agent:langchain` (necessária para que esse *bean* exista no contentor Spring a injectar).

4.1.3 Camadas do Sistema

A Figura 4.1 apresenta a visão macro do sistema, organizada em quatro zonas de rede com fronteiras explícitas.

Camada de apresentação (*Presentation*): os dois clientes — o cliente móvel (Android/iOS) e o cliente web (SPA React) — comunicam exclusivamente via HTTPS com a camada de rede de fronteira.

Rede de fronteira (*Edge Network*): um *reverse proxy* Nginx centraliza todo o tráfego de entrada. Executa duas funções distintas: encaminha os pedidos de API (`/api/**`, `/play4change-server/**`) para o processo Spring Boot via HTTP interno, e serve os ficheiros estáticos compilados da SPA React com uma directiva `try_files` para suporte de *client-side routing*. O servidor de aplicação nunca serve activos estáticos.

Rede privada (*Private Network*): o servidor Spring Boot, as bases de dados e a camada de observabilidade estão isolados numa rede privada inacessível externamente. O servidor acede ao **PostgreSQL 16** (com as extensões `pgvector` e `uuid-oss`) para persistência

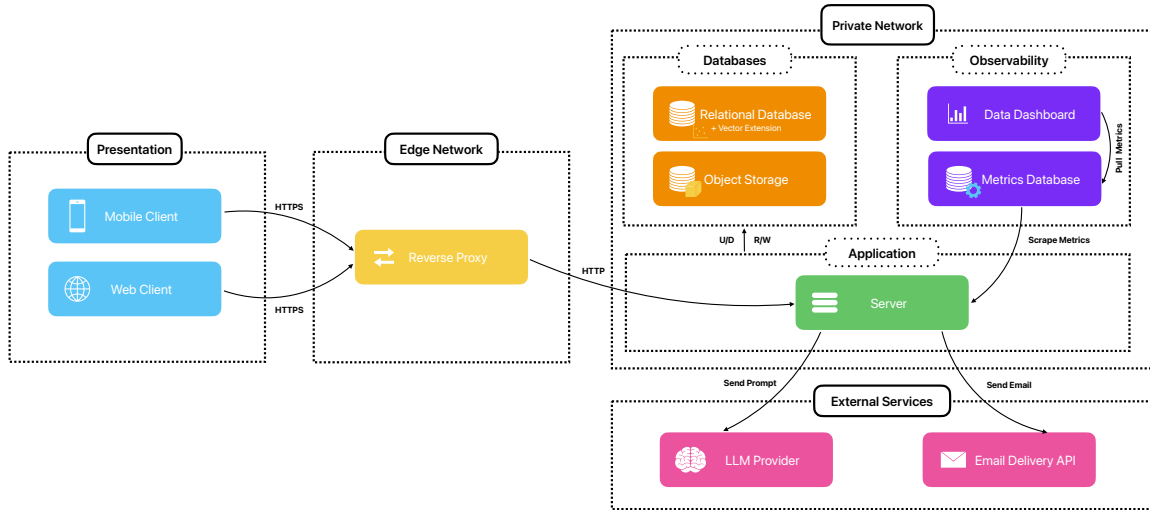


Figura 4.1: Arquitetura geral do sistema Play4Change

relacional e vetorial — incluindo a revogação de *refresh tokens* por marcação `used=true` e o *rate limiting* em memória (Bucket4j + Caffeine, por processo) — e ao **MinIO** (compatível com S3) para armazenamento de ficheiros e APKs compilados. O Prometheus extrai métricas do *endpoint /actuator/prometheus* a cada cinco segundos; o Grafana consome o Prometheus e apresenta *dashboards* provisionados automaticamente por ficheiros de configuração.

Serviços externos: o servidor comunica com dois serviços externos geridos por terceiros: o **Mistral AI** (via LangChain4j) para geração de tarefas e síntese de explicações, e o **Resend** como API de entrega de email para o fluxo de autenticação *passwordless* por *magic link*.

4.1.4 Domínio: Contextos Delimitados

O domínio está particionado em **oito contextos delimitados**, cada um com os seus próprios agregados, contratos de repositório e serviços de aplicação. Nenhum contexto acede directamente aos agregados de outro — a comunicação inter-contexto passa sempre por interfaces de serviço de aplicação.

- **Auth** — Fluxo de *magic link*, ciclo de vida de tokens JWT, gestão de papéis (*roles*)
- **Identity** — Perfil do Learner, preferências, fuso horário, email de recuperação
- **Topic** — Estrutura curricular: tópicos → módulos → *templates* de tarefas (versionados, com *embeddings*)
- **Enrollment** — Inscrição do Learner em tópicos, agendamento de atribuições, rastreio de progresso
- **Struggle** — Heurísticas de deteção de dificuldade, persistência de registos de *struggle*, lógica de activação adaptativa

- **Explanation** — Sessões de explicação sintetizadas por IA e conversa de seguimento
- **Badge** — Avaliação de critérios de conquista, ciclo de vida de atribuição de *badges*
- **Report** — Métricas agregadas de desempenho em tarefas para análise de administração

4.1.5 Fluxo Geral de Dados

O percurso de um pedido típico do Learner ilustra como as camadas interagem sem violação das fronteiras arquiteturais. O cliente móvel envia um pedido HTTPS ao Nginx, que o encaminha via HTTP interno para o *servlet filter* de segurança Spring. O filtro valida o JWT e popula o contexto de segurança; o controlador REST delega imediatamente para o caso de uso de aplicação correspondente, sem conter lógica de negócio. O caso de uso invoca o *port* de repositório para recuperar o estado do Learner, chama o *port* de geração IA se necessário (que resolve para o adaptador LangChain4j em *runtime*), e persiste o resultado via JPA. A resposta percorre o caminho inverso sem que nenhuma camada interna tenha conhecimento do protocolo HTTP ou do formato de serialização JSON.

4.2 Backend — Spring Server e Pipeline

4.2.1 Organização do Projeto

O servidor é uma aplicação Spring Boot 3 escrita em Kotlin (JVM 21), estruturada segundo os princípios da Arquitetura Hexagonal com separação estrita entre três camadas: *domain*, *application* e *infrastructure/web*. A raiz do pacote é `com.ureka.play4change`. O *entry point* `Application.kt` regista explicitamente os pacotes dos *bounded contexts* via `@SpringBootApplication(scanBasePackages = [...])` e ativa `@EnableScheduling` para os *jobs* periódicos.

A camada **domain** não contém nenhuma anotação Spring, JPA ou LangChain4j — é Kotlin puro. Inclui as entidades, os agregados e as interfaces de *port* (repositório e serviços externos) para cada um dos oito contextos delimitados: `auth`, `identity`, `topic`, `enrollment`, `struggle`, `explanation`, `badge` e `report`.

A camada **application** contém os casos de uso (e.g. `EnrollmentUseCase`, `TopicUseCase`, `ExplanationUseCase`) e os serviços que os implementam (e.g. `TaskGenerationOrchestrator`, `HandleStruggleService`, `BatchInstanceGenerationService`). Estes serviços invocam exclusivamente interfaces de *port* — nunca implementações concretas — garantindo que a lógica de negócio é testável de forma independente da base de dados ou do fornecedor de LLM.

A camada **infrastructure** contém os adaptadores de saída: adaptadores JPA para cada repositório de domínio, `MinioFileStorageAdapter` (armazenamento de objetos), `UrlScrapeAdapter` e `UnstructuredPdfAdapter` (extração de conteúdo), `ResendEmailAdapter` (email transaccional) e os filtros/interceptores do pipeline HTTP. Os adaptadores de entrada HTTP estão no

pacote `web`, separados em três sub-pacotes: `admin` (gestão de tópicos, utilizadores e eventos SSE), `user` (matrícula, tarefas, explicações) e `public` (estatísticas públicas).

O contexto de autenticação está completamente isolado no pacote `auth`, com os seus próprios sub-pacotes `domain`, `application`, `port` e `adapter`, não partilhando nenhuma dependência direta com o domínio principal.

O módulo de IA está separado em dois módulos Gradle: `ai-agent:api` define a interface `TaskGenerationPort` (contrato puro, sem dependências de *runtime*); `ai-agent:langchain` é a única localização no repositório que referencia `LangChain4j` e implementa esse contrato com `LangChain4jTaskGenerationAdapter` anotado com `@Primary`. O código do módulo `:server` (domínio e aplicação) depende apenas da interface `TaskGenerationPort` de `ai-agent:api` e nunca referencia `LangChain4j` directamente — a implementação concreta é injectada pelo contentor Spring em *runtime*. O `build.gradle.kts` de `:server` declara, no entanto, uma dependência de compilação em `ai-agent:langchain`, necessária para expor o *bean* `@Primary` ao contentor Spring; substituir o fornecedor de LLM não exige alterar lógica de negócio, mas exige trocar essa dependência de *build* — não é, portanto, uma alteração de zero linhas de código.

A gestão do esquema de base de dados é feita exclusivamente por Flyway (32 migrações, V1 a V32), com `spring.jpa.hibernate.ddl-auto: validate` — o Hibernate valida o esquema em arranque mas nunca o altera em produção. Queries críticas para atomicidade (e.g. `claimToken` para consumo único de *magic links*) usam SQL nativo com `RETURNING` para eliminar janelas TOCTOU.

4.2.2 Pipeline de Processamento de Pedidos

Cada pedido HTTP atravessa um conjunto de camadas ordenadas de forma determinista antes de atingir o *controller*. A Figura 4.2 ilustra o fluxo completo para um pedido autenticado típico.

Filtros e interceptores — ordem de execução

Quatro componentes operam em camadas distintas antes de o pedido atingir o *controller*, descritos na Tabela 4.2.

O `RateLimitFilter` tem `@Order(Ordered.HIGHEST_PRECEDENCE)`, um valor inferior ao `@Order(1)` do `RequestIdFilter`, pelo que é efectivamente o primeiro componente a correr na cadeia — antes do próprio `RequestIdFilter` e de qualquer lógica de autenticação — o que impede ataques de força bruta mesmo sobre utilizadores não autenticados. Uma consequência desta ordem é que um pedido rejeitado por *rate limiting* (429) nunca chega a ter um `X-Request-Id` atribuído nem entra no MDC de logging, pelo que não é correlacionável nos logs da mesma forma que um pedido aceite. Os *buckets* são mantidos num *cache* Caffeine com expiração de 15 minutos por inatividade e limite de 50 000 entradas para evitar

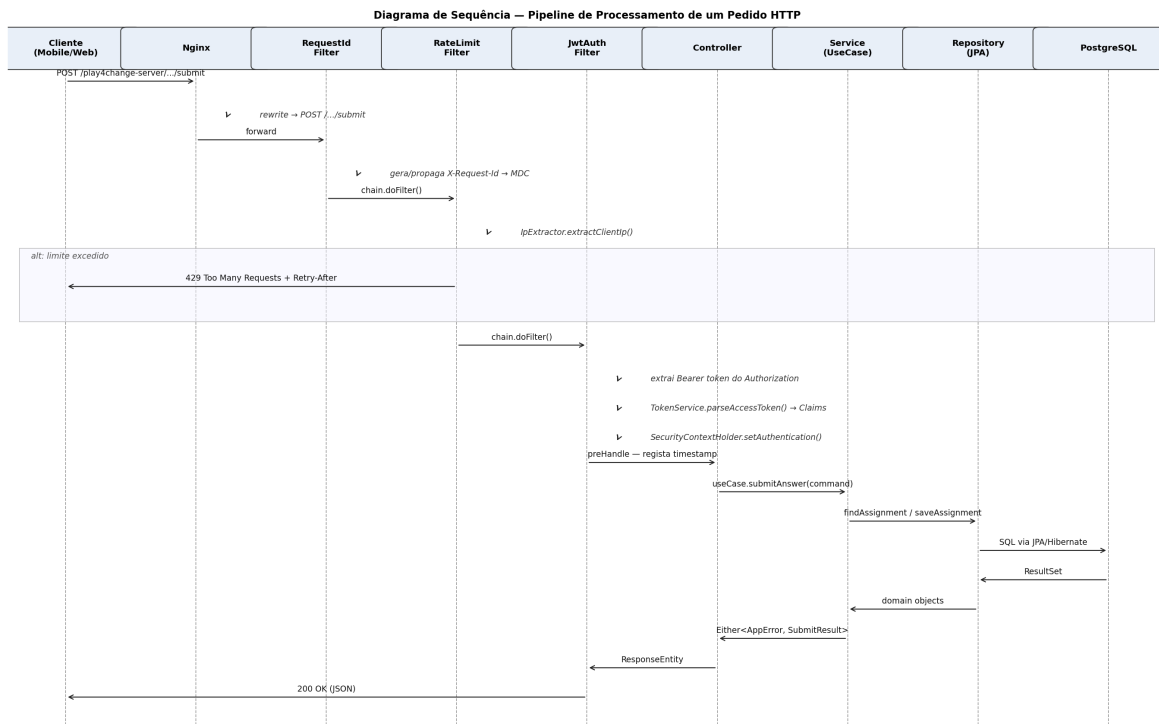


Figura 4.2: Pipeline de processamento de um pedido HTTP autenticado

Tabela 4.2: Componentes do pipeline HTTP e responsabilidades

Ordem	Componente	Responsabilidade
1	RateLimitFilter (HIGHEST_PRECEDENCE)	<i>Rate limiting</i> por IP $\times$ path normalizado, via Bucket4j com <i>cache</i> Caffeine; devolve 429 com <i>Retry-After</i> quando o <i>bucket</i> é excedido
2	RequestIdFilter (@Order(1))	Gera ou propaga o header X-Request-Id; coloca-o no MDC do SLF4J para correlação de logs distribuídos
3	JwtAuthFilter (OncePerRequestFilter)	Valida o JWT do header <i>Authorization: Bearer</i> ; popula o <i>SecurityContextHolder</i> com <i>userId</i> e <i>role</i> ; devolve 401 em token inválido
4	LoggingInterceptor (HandlerInterceptor)	Regista $\rightarrow$ METHOD path em <i>preHandle</i> e $\leftarrow$ status [Xms] em <i>afterCompletion</i>

crescimento ilimitado de memória. Os limites por *endpoint* estão descritos na Tabela 4.3.

Pipeline assíncrono de geração de tópicos

A criação de um tópico é o fluxo mais complexo do sistema, envolvendo extração de conteúdo, geração por LLM e indexação vetorial. A Figura 4.3 ilustra o pipeline completo, que atravessa cinco estados de máquina de estados:

INGESTION $\rightarrow$ ANALYSIS $\rightarrow$ GENERATION $\rightarrow$ INDEXING $\rightarrow$ ACTIVE

(qualquer fase pode transitar para FAILED em caso de erro)

O pedido POST /admin/topics devolve imediatamente 201 Created com o tópico em estado PENDING; o pipeline executa em paralelo num executor dedicado (*generationExecutor*)

Tabela 4.3: Limites de *rate limiting* por *endpoint* (janela de 10 minutos por IP)

Endpoint	Capacidade
POST /auth/magic-link	5
POST /auth/magic-link/verify	10
POST /auth/refresh	20
POST /explanation/.../message	10
POST /admin/topics	3
POST /admin/topics/pdf	2
POST /admin/topics/id/regenerate	2

Nota: existe também uma regra configurada para `/topics/{id}/task-assignments/{id}/submit` (30) destinada à submissão de respostas, mas o *endpoint* real de submissão é `POST /tasks/{assignmentId}/submit` — um padrão de rota diferente, que a regra nunca corresponde. Na prática, a submissão da tarefa diária não está sujeita a *rate limiting* (ver Secção 4.8.2 para detalhe).

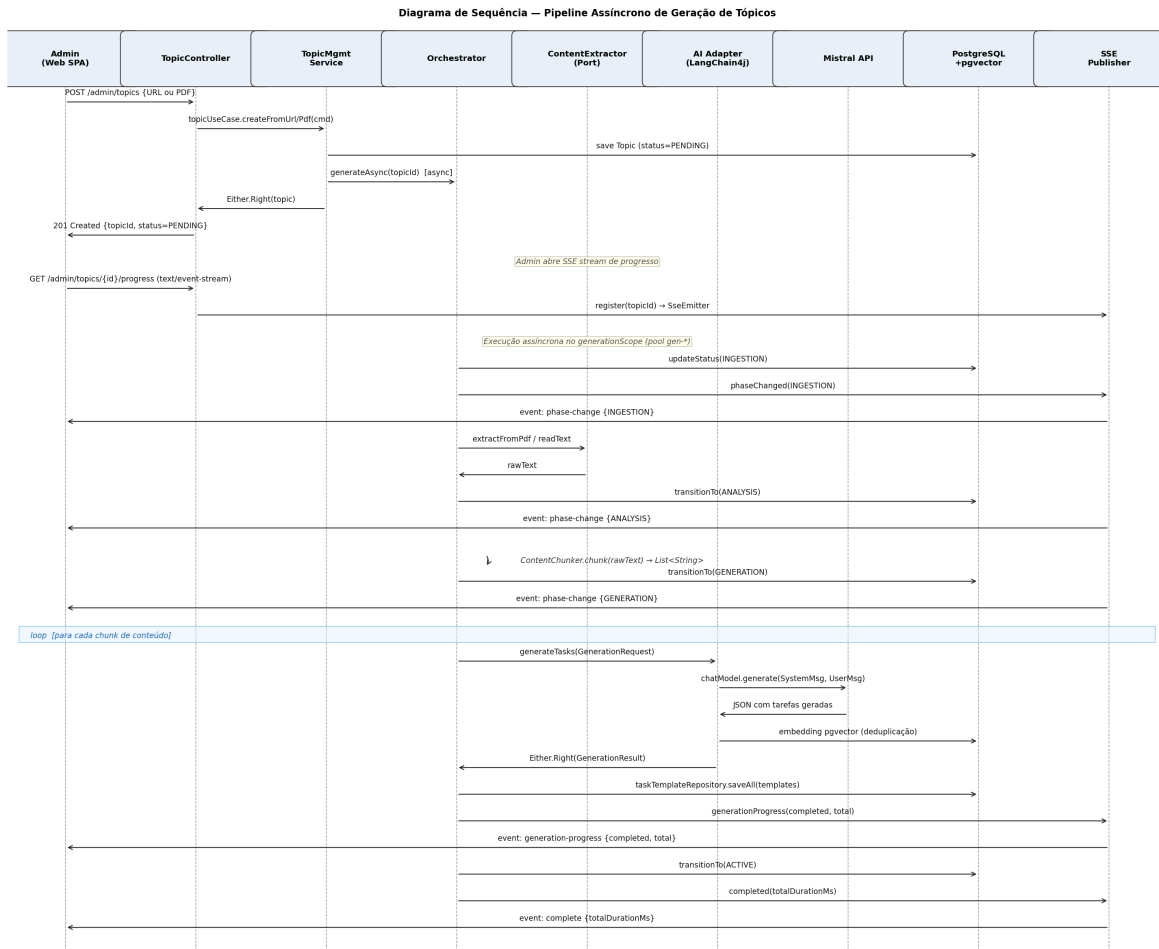


Figura 4.3: Pipeline assíncrono de geração de tópicos com notificação SSE

com três *threads* e fila de 25 pedidos. O `SupervisorJob` Kotlin garante que a falha de uma

geração não cancela outras em curso. O Admin acompanha o progresso em tempo real via *Server-Sent Events* (SSE) no *endpoint* `GET /admin/topics/{id}/progress`, recebendo eventos `phase-change`, `generation-progress` e `complete`.

Dois *schedulers* Spring garantem a robustez do pipeline: o `StuckGenerationWatchdogJob` (a cada 120 s) deteta tópicos presos em `GENERATING` por mais de 15 minutos e transita-os para `FAILED`; o `TopicExpirationJob` (a cada 60 s) marca como `EXPIRED` os tópicos `ACTIVE` cuja `expires_at` passou.

A integração com o LLM é protegida por Resilience4j com *circuit breaker* de 10 chamadas (abre a 50% de falhas, aguarda 30 s antes de meio-abrir), *retry* de 3 tentativas com *backoff* exponencial de 2 s, e *timeout* de 60 s por geração (`withTimeoutOrNull`).

4.2.3 Integração com os Restantes Componentes

O servidor integra-se com cinco sistemas de infraestrutura e dois sistemas externos. Todas as integrações são mediadas por interfaces de *port* no domínio — nunca por referências diretas às implementações.

Nginx

O Nginx serve simultaneamente como *reverse proxy* e servidor de ficheiros estáticos da SPA React. A regra `location /play4change-server/` aplica `rewrite ^/play4change-server/(.*)/$1 break` antes de `proxy_pass` ao `server:8080` — o servidor nunca vê o prefixo de roteamento. O `proxy_read_timeout` é 120 s para acomodar chamadas ao LLM. Os *headers* de segurança `Strict-Transport-Security`, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff` e `Referrer-Policy` são aplicados em todas as *locations*. A configuração CORS no servidor (`WebMvcConfig`) define origens explícitas sem *wildcards*, `allowCredentials=true` e `maxAge=3600`.

PostgreSQL com pgvector

O servidor usa Spring Data JPA com Hibernate. Cada *bounded context* tem os seus próprios adaptadores JPA (e.g. `MagicLinkTokenJpaAdapter`, `TaskTemplateJpaAdapter`). A extensão `pgvector` armazena *embeddings* de 1024 dimensões nos *templates* de tarefas, usados pelo `PgVectorDeduplicationService` para detetar tarefas semanticamente semelhantes antes de persistir novas geradas pelo LLM.

MinIO

O `MinioFileStorageAdapter` implementa `FileStoragePort` e armazena PDFs em `topics/{topicId}/content` e texto de URLs em `topics/{topicId}/content.txt`. O conteúdo é descarregado pelo orquestrador no início do pipeline de geração para garantir que o LLM nunca acede diretamente ao URL original.

Scraper Playwright e Unstructured.io

A extração de conteúdo é delegada a dois microserviços em contentor: para tópicos de URL, o servidor chama `http://scraper:3000` (Node.js + Playwright) para obter o HTML renderizado de páginas com *client-side rendering*; para tópicos PDF, envia o ficheiro à API `http://unstructured:8000` (Unstructured.io) para extração de texto estruturado com suporte a OCR. O `UrlSsrValidator` valida todos os URLs antes de os enviar ao *scraper*, rejeitando endereços privados, *loopback* e redes reservadas para prevenir ataques SSRF.

Resend (email transacional)

O `ResendEmailAdapter` (anotado `@Primary @ConditionalOnExpression`) ativa-se apenas se a variável de ambiente `RESEND_API_KEY` estiver definida, invocando `https://api.resend.com/emails` via `RestTemplate`. Em ambiente de desenvolvimento, o `ConsoleEmailAdapter` regista o token no log sem enviar email.

Prometheus e Grafana

O Spring Boot Actuator expõe `/actuator/prometheus` na porta de gestão (9090), inacessível externamente. Métricas instrumentadas manualmente incluem: `task.generation.duration` (histograma por status), `ai.generation.duration` (por fase), `ai.adaptive.reuse` (por estratégia: *full/partial/fresh*), contadores de sessões de explicação e de *rate limiting* excedido por prefixo de *path*. Três *dashboards* Grafana são provisionados automaticamente por ficheiros de configuração: latência de IA, fluxo do *learner* e *dashboard* geral.

4.3 Autenticação Passwordless

O Play4Change implementa autenticação *passwordless* por *magic link*, sem qualquer OAuth externo ou SSO. O design resolve três requisitos simultâneos: eliminar *passwords* para o utilizador gerir, garantir resistência a ataques de reutilização de tokens, e suportar um email de recuperação alternativo como segunda via de acesso.

4.3.1 Modelo de Dados e Propriedades Criptográficas

O subsistema de autenticação mantém quatro tabelas independentes do domínio principal. A Tabela 4.4 descreve a estrutura e as propriedades de segurança de cada uma.

Todos os tokens são gerados pela classe `AuthCrypto` com 32 bytes de `SecureRandom` (64 caracteres hexadecimais). **Apenas o SHA-256 do token é persistido**; o token em claro existe exclusivamente em memória e no email enviado ao utilizador, nunca na base de dados.

Tabela 4.4: Tabelas do modelo de dados de autenticação

Tabela	Campos relevantes	Função e propriedades de segurança
users	id (UUID), email, provider (MAGIC_LINK), role (USER ADMIN)	Perfil de autenticação; email é normalizado (lowercase + trim) antes de qualquer operação
magic.link.tokens	token (SHA-256 do token em claro), expires_at (+15 min), used (bool)	Apenas o <i>hash</i> é persistido; consumo atômico via UPDATE ... RETURNING elimina janelas TOCTOU
refresh.tokens	token_hash (SHA-256), family_id (UUID), expires_at (+7 dias), used (bool)	family_id agrupa todos os <i>tokens</i> de uma sessão; detecção de reutilização revoga toda a família
recovery_emails	email, verified (bool), token_hash, token_expires_at (+24h)	Permite autenticação com email alternativo verificado; não cria utilizador duplicado

4.3.2 Fluxo de Autenticação por Magic Link

O fluxo de autenticação decorre em duas fases: pedido do *magic link* e verificação do token. A Figura 4.4 apresenta o diagrama de sequência completo, incluindo os casos de token inválido.

Na primeira fase, o utilizador submete o email; o `MagicLinkService` normaliza-o, gera um token opaco, persiste o SHA-256 com `expires_at=now+15min` e delega o envio ao `EmailPort`. O servidor responde 202 Accepted imediatamente — não confirmando nem negando a existência do email, evitando enumeração de contas.

Na segunda fase, o utilizador clica no *link* no email. No cliente móvel, o URI `play4change://auth/verify` é capturado pelo *deep link* da aplicação, que extrai o token e faz POST `/auth/magic-link/verify`. No cliente web, o GET de verificação recebe um redirect 302 que não consome o token — apenas encaminha para o fluxo POST. O consumo do token é feito atomicamente por:

```
UPDATE magic_link_tokens SET used=true WHERE token=:hash AND used=false AND
    expires_at>now() RETURNING email
```

Zero linhas retornadas significa token inválido, expirado ou já utilizado — as três condições de falha são indistinguíveis para o cliente, evitando *oracle attacks*.

Em caso de sucesso, o serviço verifica se o email corresponde a um utilizador existente, a um email de recuperação verificado (em ambos os casos sem criar novo utilizador), ou a um email genuinamente novo (registo automático implícito). É então emitido um par de tokens: *access token* JWT (15 minutos) e *refresh token* opaco (7 dias).

4.3.3 Ciclo de Vida dos Tokens JWT e Rotação de Refresh Tokens

O *access token* é um JWT assinado com HMAC-SHA-256, com a estrutura de *claims* {sub, email, role, iat, exp}. A chave de assinatura é validada em arranque pelo `JwtSecretStartupGuard`: em perfil `prod`, o arranque falha (`IllegalStateException`) se a chave tiver menos de 64 caracteres ou for igual ao valor de desenvolvimento por omissão; fora de `prod`, o uso desse

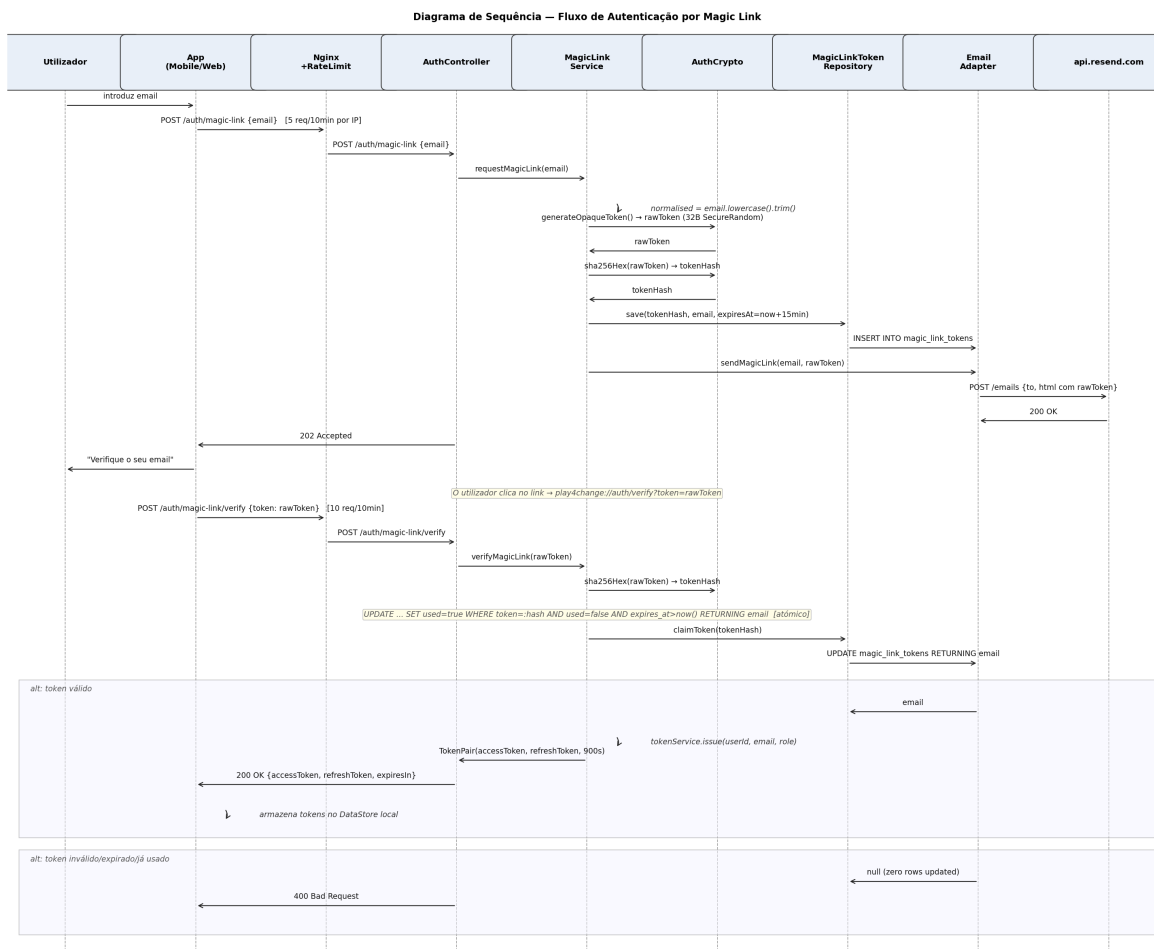


Figura 4.4: Diagrama de sequência — fluxo completo de autenticação por *magic link*

valor por omissão gera apenas um aviso em log, sem impedir o arranque. A sessão Spring é **STATELESS** (`SessionCreationPolicy.STATELESS`) — cada pedido é autenticado independentemente pelo `JwtAuthFilter`.

O mecanismo de rotação de *refresh tokens* segue o padrão *token rotation* com deteção de reutilização, conforme ilustrado na Figura 4.5.

Quando o cliente faz `POST /auth/refresh`, o `TokenService` localiza o *refresh token* pelo hash. Se o token já foi marcado como **used**, é detetado um possível roubo de sessão: todos os tokens da mesma `family_id` são imediatamente revogados, forçando o utilizador a re-autenticar. Em caso de token válido, o token atual é marcado como usado e é emitido um novo par, mantendo o mesmo `family_id` — o que garante que o *logout* revoga toda a cadeia de rotação da sessão.

4.3.4 Autorização e Controlo de Acesso

A `SecurityConfig` define quatro zonas de acesso com regras distintas, descritas na Tabela 4.5.

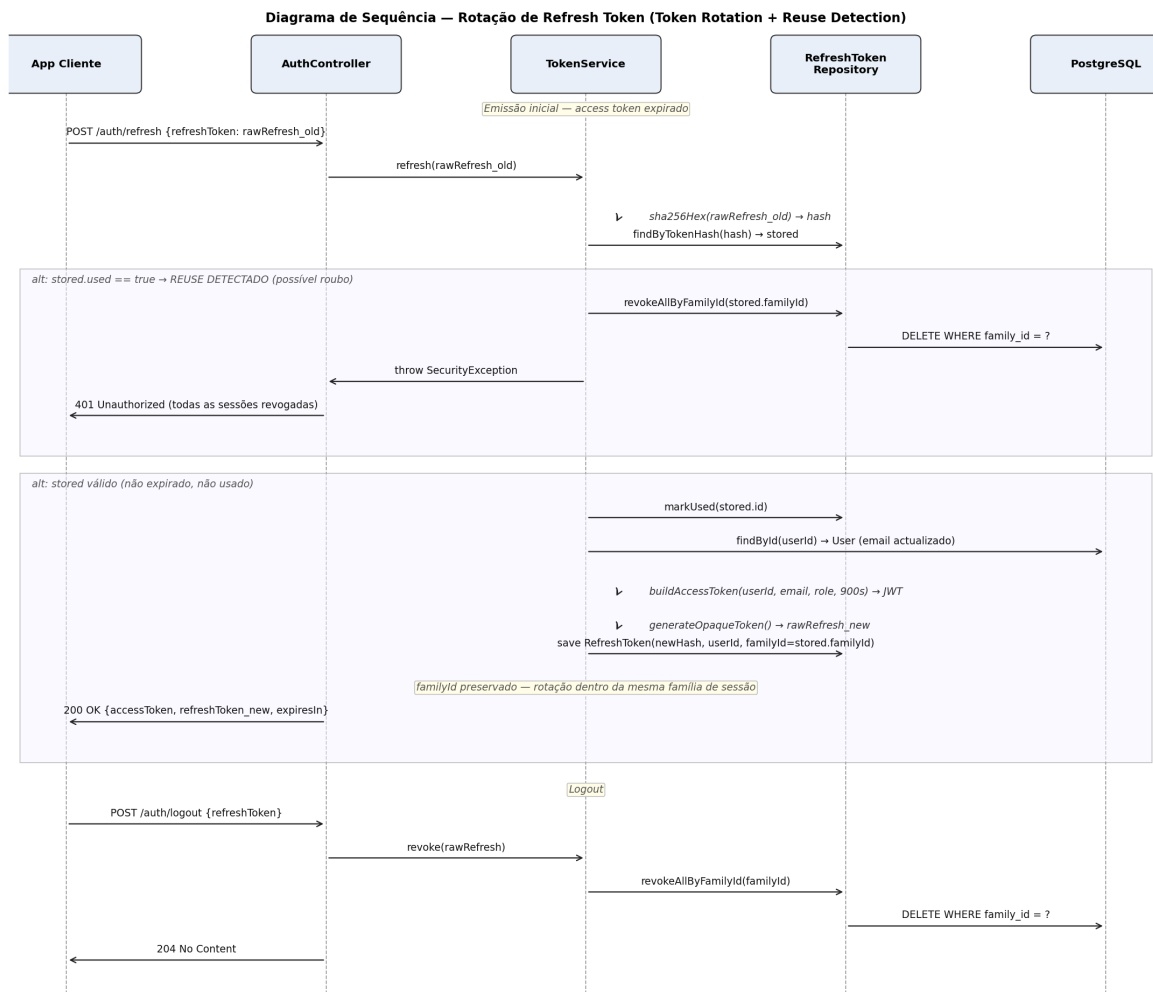


Figura 4.5: Diagrama de sequência — rotação de *refresh token* com detecção de reutilização

Tabela 4.5: Zonas de controlo de acesso configuradas no Spring Security

Zona	Paths	Requisito
Pública	/auth/**, /error, /actuator/prometheus /api/stats/public, /actuator/health,	Sem autenticação
Swagger	/swagger-ui/**, /v3/api-docs/**	ROLE_ADMIN em <i>prod</i> ; público em dev
Administração	/admin/**	ROLE_ADMIN obrigatório
Utilizador	Todos os restantes <i>paths</i>	JWT válido (qualquer <i>role</i>)

A separação da zona de administração é particularmente relevante: um Learner com JWT válido que tente aceder a `/admin/**` recebe 403 `Forbidden` sem que o pedido atinja qualquer *controller*. O Swagger é restrito a administradores em produção para evitar exposição do contrato de API a utilizadores não autorizados.

4.4 Ingestão e Extração de Conteúdo

A ingestão de conteúdo é a primeira fase do pipeline de criação de tópicos. Um Administrador fornece uma fonte — um URL público ou um ficheiro PDF — a partir da qual o sistema extrai texto simples que servirá de base à geração automática de tarefas educativas.

A camada de extração segue o padrão *Port & Adapter*: a interface `ContentExtractorPort` é definida na camada de aplicação com dois métodos, `extractFromUrl(url: String): String` e `extractFromPdf(pdfBytes: ByteArray): String`, e é implementada na infraestrutura pelo `ContentExtractorAdapter`. O domínio nunca referencia nenhuma tecnologia de extração diretamente.

Independentemente da fonte, o fluxo síncrono partilha o mesmo padrão: o `TopicManagementService` extrai o texto, valida que não está vazio, persiste o conteúdo original no MinIO (como ficheiro de auditoria e para eventual regeneração), cria o tópico em estado `PENDING` e lança o pipeline assíncrono de geração — devolvendo `201 Created` imediatamente sem aguardar a conclusão da IA.

4.4.1 Extração de Texto de PDF

A extração de texto de documentos PDF é delegada à **Unstructured API** [34], executada localmente em contentor Docker na porta 9500. Esta escolha justifica-se pela necessidade de suportar documentos digitalizados (PDFs que contêm imagens em vez de texto nativo), para os quais é necessário reconhecimento ótico de caracteres (*Optical Character Recognition*, OCR) — funcionalidade que a Unstructured integra nativamente com estratégia `auto`, que seleciona automaticamente entre extração de texto nativo e OCR consoante o conteúdo de cada página.

O ponto de entrada HTTP é `POST /admin/topics/pdf`, com os campos descritos na Tabela 4.6.

Tabela 4.6: Campos do pedido `POST /admin/topics/pdf` (multipart/form-data)

Campo	Tipo	Obrigatório	Restrições
<code>file</code>	binary PDF	Sim	máx. 25 MB
<code>title</code>	String	Sim	máx. 200 caracteres
<code>description</code>	String	Sim	máx. 1 000 caracteres
<code>category</code>	String	Não	máx. 100 caracteres
<code>difficulty</code>	Enum	Não	<code>BEGINNER</code> <code>INTERMEDIATE</code> <code>ADVANCED</code> (omissão: <code>BEGINNER</code>)
<code>language</code>	String	Não	código ISO 639-1 (omissão: <code>en</code>), máx. 10 caracteres
<code>taskCount</code>	Int	Não	omissão: 10
<code>expiresAt</code>	ISO 8601	Não	omissão: agora + 90 dias

A Figura 4.6 detalha o fluxo interno de extração de PDF.

O `ContentExtractorAdapter` submete o PDF como `multipart/form-data` para o *endpoint* `/general/v0/general` com o campo `strategy=auto`. A resposta é uma lista de elementos JSON com campo `text`; o adaptador filtra elementos em branco, une-os com separador `\n\n`

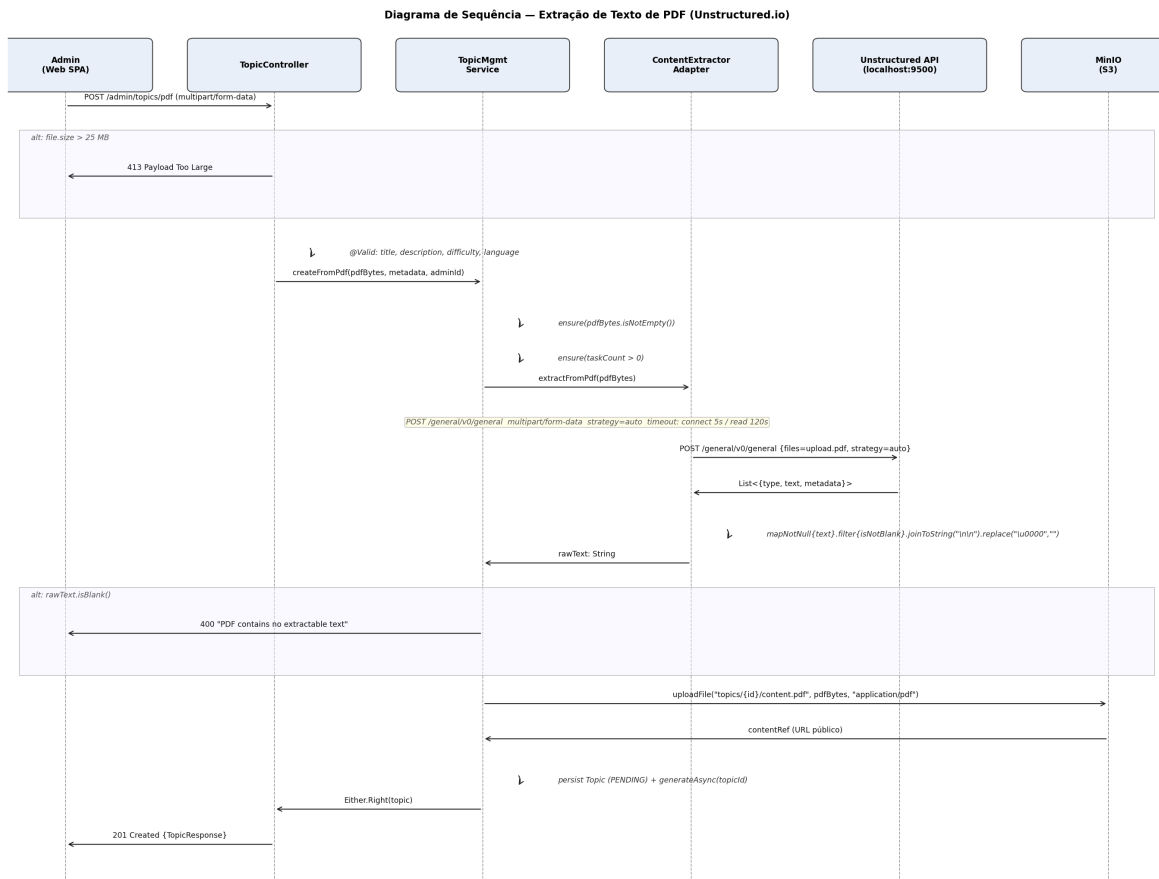


Figura 4.6: Diagrama de sequência — extração de texto de PDF via Unstructured.io

(preservando a estrutura de parágrafos), e remove bytes nulos (`\u0000`) que são artefactos comuns de OCR. O *timeout* de leitura é de 120 segundos, justificado pela possibilidade de documentos extensos ou totalmente digitalizados requererem processamento OCR prolongado.

O texto extraído é persistido no MinIO em `topics/{id}/content.pdf`, eliminando a necessidade de re-extração em caso de regeneração de tarefas pelo Administrador.

4.4.2 Extração de Texto de URL

A extração de texto a partir de URLs é delegada a um microserviço de *scraping* baseado em **Playwright** [35], implementado em Node.js e executado em contentor Docker na porta 3000. A escolha de Playwright em detrimento de um simples cliente HTTP é deliberada: muitas páginas educativas são *Single Page Applications* (SPA) que requerem execução de JavaScript para renderizar o seu conteúdo — conteúdo esse que seria invisível a uma chamada HTTP convencional.

O ponto de entrada HTTP é `POST /admin/topics`, com corpo JSON. Os campos de metadados são equivalentes ao fluxo PDF (Tabela 4.6), substituindo `file` pelo campo `url` (String, obrigatório, sujeito a validação SSRF).

A Figura 4.7 detalha o fluxo interno, incluindo a validação de segurança de URL.

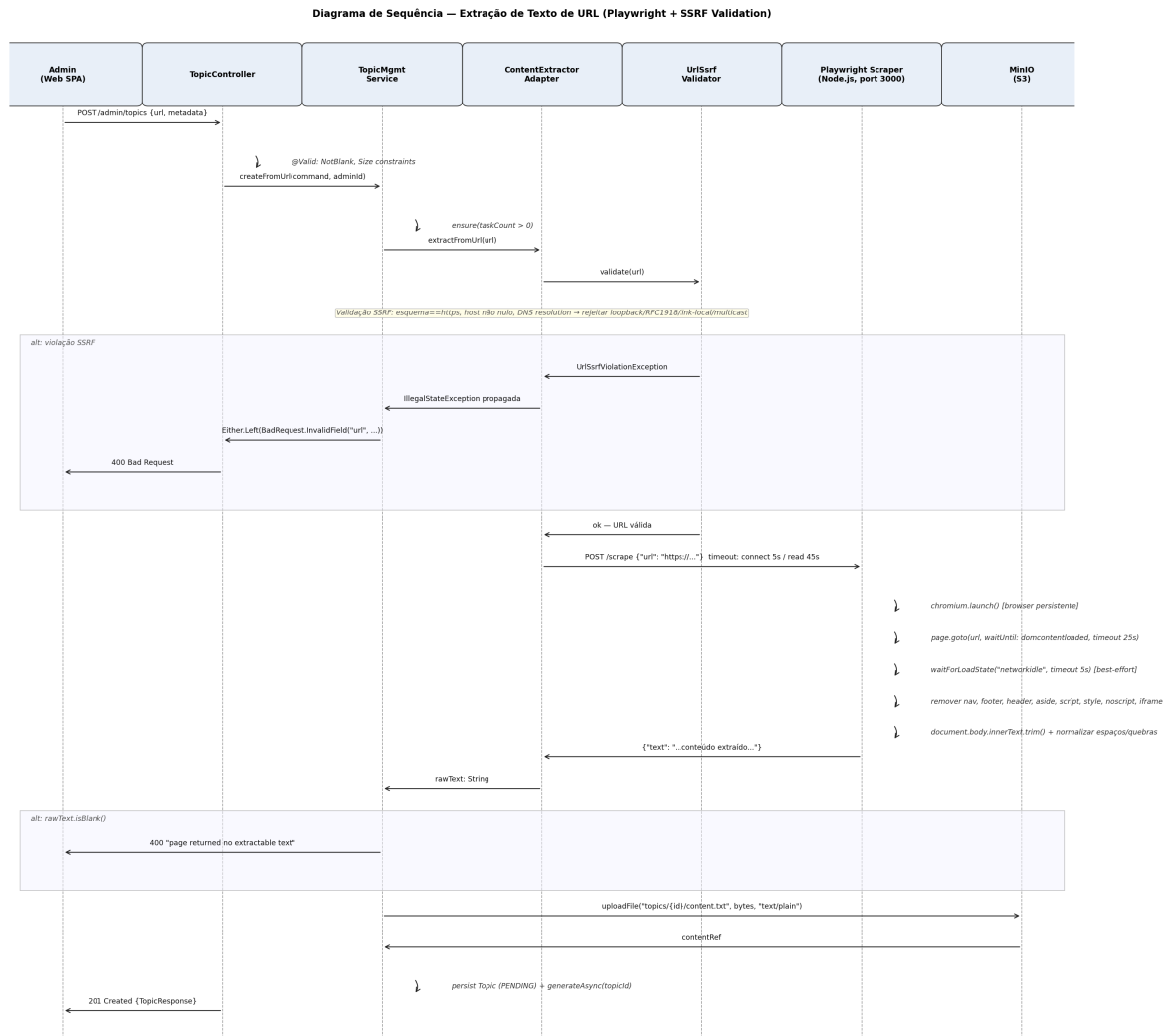


Figura 4.7: Diagrama de sequência — extração de texto de URL com validação SSRF e Playwright

Prevenção de SSRF

Antes de qualquer pedido de rede, o `UrlSsrifValidator` executa uma validação sequencial que termina na primeira violação detetada:

1. **Esquema obrigatório https** — rejeita URLs com esquema `http` ou outros protocolos.
2. **Host não nulo** — rejeita URLs malformadas sem componente de `host`.
3. **Resolução DNS e filtragem de endereços privados** — invoca `InetAddress.getAllByName(host)` e rejeita qualquer endereço resolvido que pertença a: `loopback` (127.0.0.0/8, ::1), redes

privadas RFC 1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), *link-local* (169.254.0.0/16, fe80::/10) ou *multicast*.

Qualquer violação lança uma `UrlSsrFViolationException` que o `TopicManagementService` converte em `BadRequest.InvalidField("url", detalhe)` devolvido como 400 Bad Request. Esta validação mitiga o ataque OWASP A10:2021 (SSRF), impedindo que o sistema seja usado como *proxy* para alcançar serviços internos da rede Docker.

Comportamento do *scraper* Playwright

O serviço opera com um browser Chromium persistente, reutilizado entre pedidos para evitar o custo de inicialização. Para cada pedido, abre uma nova página e aguarda o evento `domcontentloaded` (timeout: 25 s); de seguida, tenta `networkidle` (timeout: 5 s, ignorando falhas) para capturar conteúdo carregado assincronamente sem bloquear em páginas com *polling* contínuo. Remove elementos de navegação e metadados (`nav`, `footer`, `header`, `aside`, `script`, `style`, `noscript`, `iframe`) e extrai `document.body.innerText`, normalizando espaços múltiplos e quebras de linha consecutivas.

O `RestTemplate` dedicado ao *scraper* no lado do servidor tem *timeout* de leitura de 45 s — superior ao timeout interno do scraper (35 s) para absorver latência de rede. O texto extraído é persistido no MinIO em `topics/{id}/content.txt`.

4.4.3 Limites e Validações

A ingestão de conteúdo envolve quatro camadas de validação independentes, cada uma responsável por uma classe distinta de erros. A Figura 4.8 apresenta o fluxo de validação completo para ambas as fontes de conteúdo.

Camada 1 — Rate Limiting (por IP, antes da autenticação)

O `RateLimitFilter` é o componente de maior prioridade na cadeia de filtros Spring (`@Order(HIGHEST_PRECEDENCE)`) usando `Bucket4j` com *cache* Caffeine (expiração: 15 min, capacidade: 50 000 entradas). Os limites para os *endpoints* de ingestão, apresentados na Tabela 4.7, são intencionalmente conservadores: a geração de conteúdo por IA é a operação de maior custo computacional e financeiro do sistema.

Tabela 4.7: Limites de *rate limiting* para *endpoints* de ingestão (janela de 10 minutos por IP)

Endpoint	Limite	Resposta ao exceder
POST /admin/topics (URL)	3 pedidos	429 + Retry-After
POST /admin/topics/pdf (PDF)	2 pedidos	429 + Retry-After
POST /admin/topics/{id}/regenerate	2 pedidos	429 + Retry-After

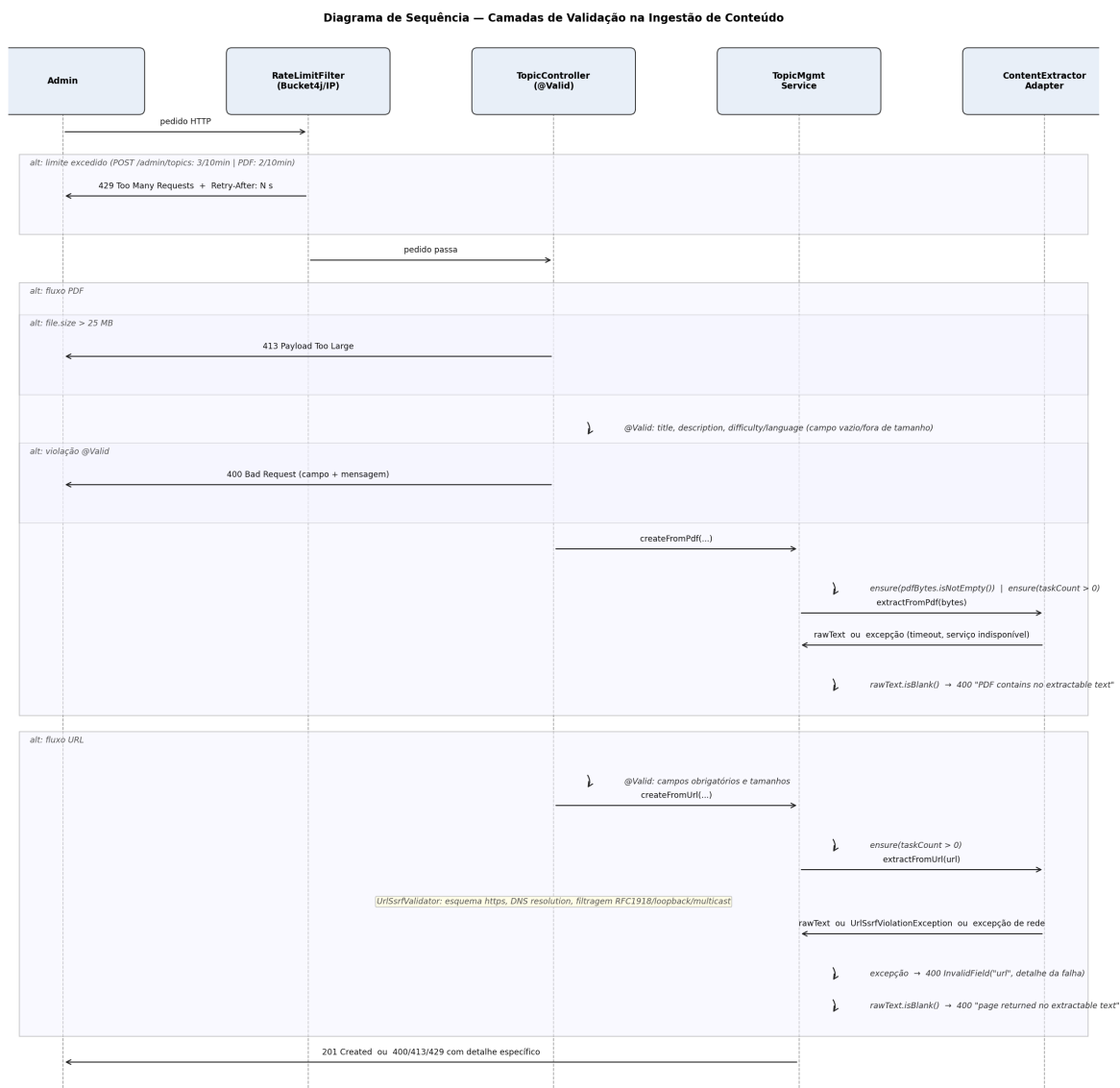


Figura 4.8: Diagrama de sequência — camadas de validação na ingestão de conteúdo

Camada 2 — Validação HTTP (tamanho e campos obrigatórios)

O `TopicController` verifica o tamanho do ficheiro PDF antes de qualquer processamento: ficheiros superiores a 25MB são rejeitados com 413 Payload Too Large. Este limite é reforçado pela configuração Spring (`spring.servlet.multipart.max-file-size: 25MB; max-request-size: 25MB`). A validação dos campos de metadados é declarativa via anotações `@Valid` do Jakarta Validation API, que geram automaticamente 400 Bad Request com o nome do campo e a restrição violada.

Camada 3 — Validação de Domínio (no serviço de aplicação)

O `TopicManagementService` aplica invariantes de negócio que não são expressáveis por anotações: conteúdo binário não pode estar vazio (`pdfBytes.isNotEmpty()`), e `taskCount` tem de ser positivo. Em caso de violação SSRF (URL), a exceção propagada do `UrlSsrValidator` é convertida em `BadRequest.InvalidField("url", detalhe)`.

Camada 4 — Validação do Conteúdo Extraído

Independentemente da fonte, o texto extraído é validado após a chamada ao adaptador. A Tabela 4.8 resume os cenários de falha e as respostas correspondentes.

Tabela 4.8: Cenários de falha na extração de conteúdo e respostas HTTP

Cenário	Resposta	Campo em erro
PDF com apenas imagens sem OCR bem-sucedida	400 “PDF contains no extractable text”	<code>file</code>
Página com conteúdo exclusivamente em canvas	400 “page returned no extractable text”	<code>url</code>
Timeout ou indisponibilidade do Unstructured	400 com mensagem de erro propagada	<code>file</code>
Timeout ou indisponibilidade do scraper	400 com mensagem de erro propagada	<code>url</code>
Violação SSRF	400 com detalhe da violação	<code>url</code>

Limites de Contexto para Geração (`ContentChunker`)

Após extração bem-sucedida, o texto é particionado pelo `ContentChunker` antes de ser enviado ao LLM. Os parâmetros são definidos em `AiContextLimits`: tamanho máximo de cada *chunk* de 32 000 caracteres, com sobreposição (*overlap*) de 2 000 caracteres entre *chunks* consecutivos para preservar contexto nas fronteiras. O algoritmo prefere divisões em limites de parágrafo (`\n\n`) dentro dos últimos 500 caracteres de cada *chunk*, recorrendo ao corte rígido por posição apenas quando não encontra parágrafo — garantindo que o contexto enviado ao modelo preserve a coerência semântica do texto original.

4.5 Modelo de IA e Estratégia de Prompting

O módulo de IA do Play4Change serve três finalidades distintas: geração de tarefas educativas a partir de conteúdo ingerido, síntese de ramos adaptativos para estudantes com dificuldades, e condução de sessões de explicação interactiva. Cada finalidade tem o seu próprio prompt, com preocupações de segurança e limites de contexto especificamente calibrados.

4.5.1 Modelo Escolhido e Justificação

O sistema utiliza **Mistral AI** como fornecedor de LLM, integrado através da biblioteca `LangChain4j` v0.36.2. O modelo configurado por omissão é `mistral-small-latest`, com temperatura $T = 0.7$ e *timeout* de 60 segundos por chamada.

São instanciados dois clientes distintos, ambos configurados na classe `LangChain4jConfig` e ativados condicionalmente via `@ConditionalOnExpression` apenas quando a variável de

ambiente `MISTRAL_API_KEY` está definida:

Tabela 4.9: Clientes LangChain4j instanciados e respectivas finalidades

Bean	Implementação	Finalidade
<code>ChatLanguageModel</code>	<code>MistralAiChatModel</code>	Geração de texto e JSON estruturado
<code>EmbeddingModel</code>	<code>MistralAiEmbeddingModel</code>	Vectores de 1024 dimensões para pgvector

A escolha de Mistral Small justifica-se por quatro razões concretas. Primeiro, a capacidade multilingue: os prompts incluem um parâmetro `language` obrigatório (e.g., `pt`, `en`, `es`), e o modelo gera conteúdo educativo com qualidade adequada em múltiplas línguas. Segundo, a relação custo/desempenho: a geração de questões de escolha múltipla e explicações requer capacidade de raciocínio moderada, não justificando modelos de maior escala e custo. Terceiro, a fiabilidade no output JSON estruturado: o sistema exige que o modelo produza JSON válido com esquema rigoroso (*arrays* com `title`, `description`, `hint`, `pointsReward`, `options`, `correctAnswerIndex`). Quarto, a intercambiabilidade: a integração via LangChain4j e a interface `TaskGenerationPort` permitem substituir o modelo sem qualquer alteração ao domínio ou à aplicação — basta criar uma nova implementação de `TaskGenerationPort` e anotá-la com `@Primary`.

Quando a chave de API não está definida, o `NoOpTaskGenerationAdapter` é ativado automaticamente, respondendo com `503 Service Unavailable` em todos os *endpoints* de IA — garantindo que o sistema falha de forma explícita em vez de silenciosa.

4.5.2 Estratégia de Prompting

O sistema define três famílias de prompts, cada uma com *system prompt* fixo e *user prompt* parametrizado com dados do contexto de domínio. Em nenhuma das três famílias é injetado diretamente input do utilizador final — essa separação é a principal defesa contra *prompt injection*, conforme descrito na Secção 4.5.3.

A — Geração de Tarefas (`TaskGenerationPrompt`)

O *system prompt* estabelece a *persona* de designer de conteúdo educativo, as regras de formato (4 opções por questão, resposta correta sempre no índice 0 para o sistema a misturar aleatoriamente), o idioma obrigatório, e o esquema JSON esperado. O *user prompt* injeta exclusivamente dados do domínio: título do tópico, objectivo do módulo, nível de audiência (`BEGINNER/INTERMEDIATE/ADVANCED`), idioma e número de tarefas pretendidas.

A Figura 4.9 descreve o pipeline completo de geração, incluindo o mecanismo de *retry* com *schema reminder* e a deduplicação por similaridade semântica.

O mecanismo de *retry* com *schema reminder* funciona assim: se o JSON de resposta do modelo não contiver o número mínimo de tarefas válidas, o adaptador reenvia o mesmo *user prompt* acrescido de uma mensagem de correção de esquema, consumindo até duas chamadas

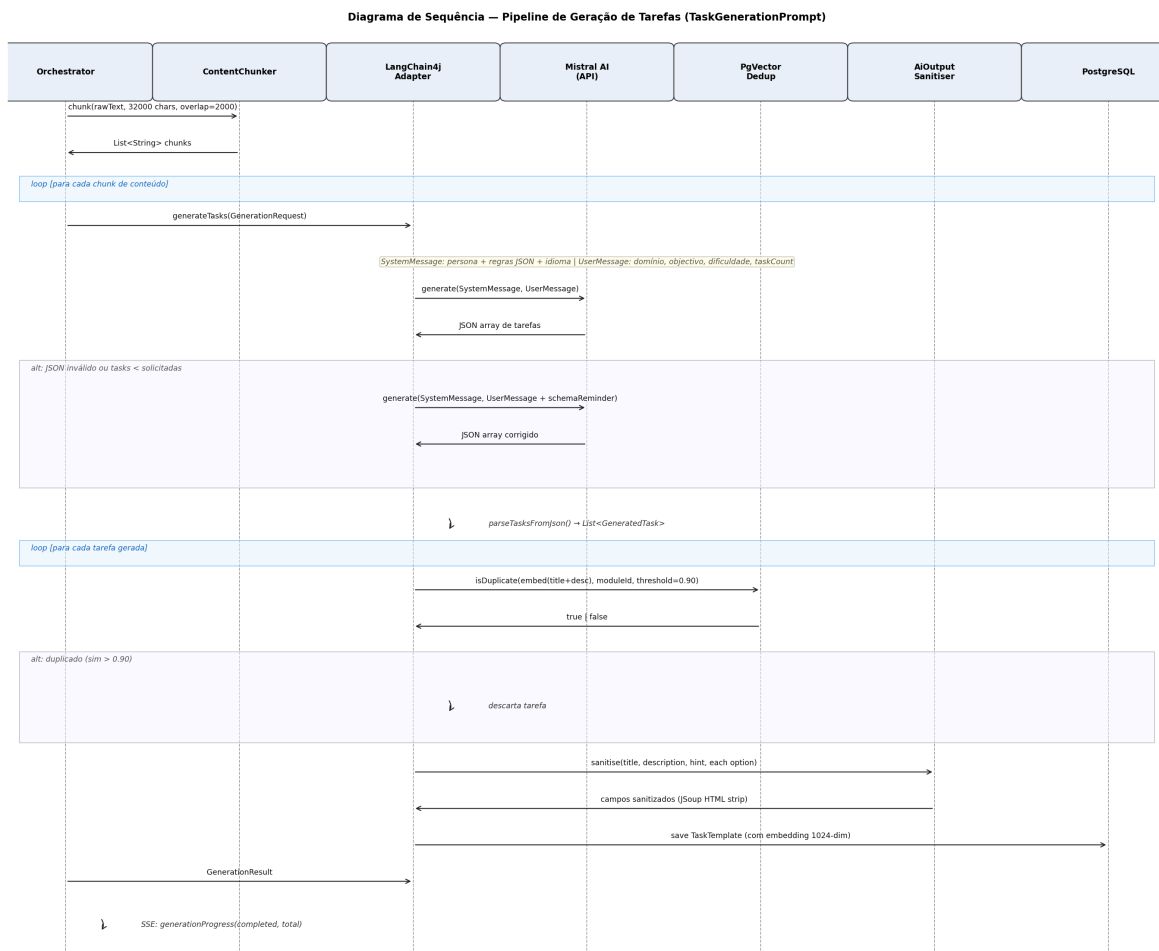


Figura 4.9: Pipeline de geração de tarefas: chunking, LLM, deduplicação pgvector e sanitização

à API por *chunk*. A deduplicação semântica usa *embeddings* de 1024 dimensões armazenados em pgvector: uma tarefa gerada é descartada se a sua similaridade coseno com qualquer *template* existente no mesmo módulo exceder o limiar de 0.90, evitando redundância no catálogo de tarefas.

B — Ramo Adaptativo (StruggleAnalysisPrompt)

Quando um estudante esgota as tentativas de uma tarefa, o sistema gera um ramo de sub-tarefas mais simples centradas no padrão de erro detetado (**ErrorPattern**: **CONCEPTUAL_MISUNDERSTANDING**, **PROCEDURAL_ERROR**, **KNOWLEDGE_GAP**, **UNCLEAR_INSTRUCTIONS**, **TIME_PRESSURE**). Este **ErrorPattern** pertence ao modelo do módulo **ai-agent** e é distinto do **ErrorPattern** do domínio do **server** (**WRONG_CONCEPT**, **PARTIAL_UNDERSTANDING**, **READING_ERROR**, **TIME_PRESSURE**), produzido de forma determinística pelo **ErrorPatternClassifier** (Secção 4.7.3); os dois são ligados por uma função de tradução (**mapErrorPattern()**) que mapeia **WRONG_CONCEPT** → **CONCEPTUAL_MISUNDERSTANDING**.

PARTIAL_UNDERSTANDING → PROCEDURAL_ERROR, READING_ERROR → UNCLEAR_INSTRUCTIONS e TIME_PRESSURE → TIME_PRESSURE. Como o `ErrorPatternClassifier` nunca produz `PARTIAL_UNDERSTANDING` na sua implementação actual, apenas três dos cinco valores do `ErrorPattern` do `ai-agent` são de facto alcançáveis hoje (`CONCEPTUAL_MISUNDERSTANDING`, `UNCLEAR_INSTRUCTIONS`, `TIME_PRESSURE`); `PROCEDURAL_ERROR` e `KNOWLEDGE_GAP` estão modelados no contrato para uma classificação mais granular ainda não implementada no classificador determinístico.

Antes de invocar o LLM, o sistema verifica a similaridade do contexto de dificuldade actual com contextos anteriores armazenados em `pgvector`, aplicando uma estratégia de reutilização com três limiares. O vetor de entrada é construído como a concatenação de `errorPattern`, `taskDescription` e `subjectDomain`, embebido em 1024 dimensões pelo `MistralAiEmbeddingModel`.

A Figura 4.10 apresenta três casos ilustrativos sobre o domínio “Python para Principiantes” — construídos para explicar o critério de decisão, não extraídos de uma sessão real registada em produção — mostrando como a estratégia muda consoante a proximidade semântica entre o contexto de dificuldade actual e os ramos já armazenados.

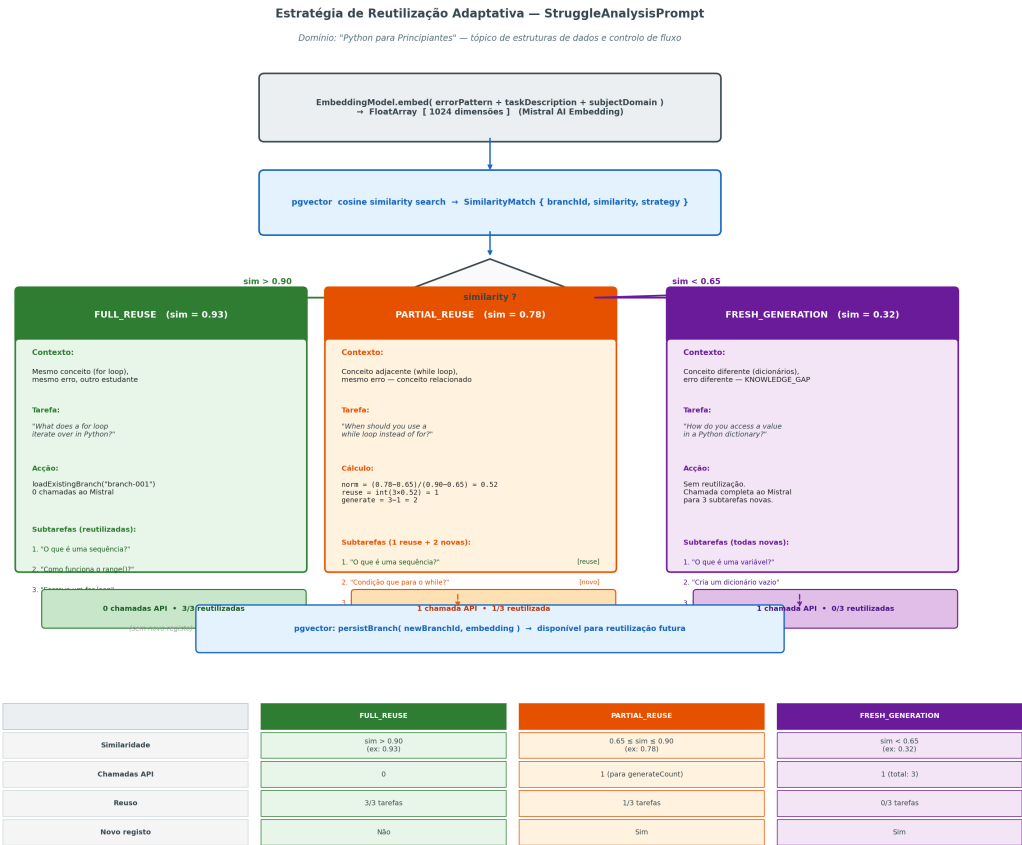


Figura 4.10: Estratégia de reutilização adaptativa — três casos concretos por limiar de similaridade `pgvector`

No caso **FULL_REUSE** (similaridade > 0.90, ex.: 0.93), dois estudantes diferentes blo-

queiam na mesma tarefa sobre ciclos **for** com o mesmo padrão de erro (`CONCEPTUAL_MISUNDERSTANDING`): o ramo previamente gerado é devolvido diretamente, sem qualquer chamada à API Mistral.

No caso **PARTIAL_REUSE** (similaridade $\in [0.65, 0.90]$, ex.: 0.78), o contexto é semanticamente próximo mas não idêntico — neste exemplo, o estudante bloqueia em ciclos **while** em vez de **for**, o mesmo erro mas conceito adjacente. A proporção de reutilização é calculada por interpolação linear:

$$n_{\text{reuse}} = \left\lfloor n_{\text{total}} \times \frac{\text{sim} - 0.65}{0.90 - 0.65} \right\rfloor$$

Com $\text{sim} = 0.78$ e $n_{\text{total}} = 3$: $n_{\text{reuse}} = \lfloor 3 \times 0.52 \rfloor = 1$, $n_{\text{generate}} = 2$. A subtarefa mais genérica do ramo existente é reutilizada; o Mistral gera apenas as 2 subtarefas específicas ao contexto **while**.

No caso **FRESH_GENERATION** (similaridade < 0.65 , ex.: 0.32), o conceito é completamente diferente (dicionários Python) e o padrão de erro ilustrado é `KNOWLEDGE_GAP` em vez de `CONCEPTUAL_MISUNDERSTANDING`: o Mistral gera um ramo completamente novo de 3 subtarefas, que é persistido em pgvector para reutilização futura. Note-se que `KNOWLEDGE_GAP` é, tal como discutido acima, um valor do contrato ainda não alcançável pelo `ErrorPatternClassifier` actual; este caso serve para ilustrar o comportamento pretendido do sistema para um classificador mais granular, não um trace observado.

A Figura 4.11 detalha o fluxo de sequência da decisão de reutilização.

C — Sessão de Explicação (`ExplanationPrompt`)

A explicação opera em dois modos sequenciais. Na **fase holística** (assíncrona), após esgotamento das tentativas, o sistema gera uma explicação detalhada baseada no historial completo de erros do estudante; o *system prompt* instrui o modelo a adoptar a persona de tutor empático e a explicar a causa raiz da dificuldade. Na **fase conversacional** (síncrona), o estudante pode fazer perguntas de seguimento, recebendo respostas geradas em tempo real.

A Figura 4.12 apresenta o diagrama de sequência completo das duas fases, incluindo os pontos de aplicação do `AiOutputSanitiser`.

Limites de Contexto

Os limites de contexto aplicados em todos os prompts estão centralizados na classe `AiContextLimits`, descritos na Tabela 4.10.

4.5.3 Proteção e Limites da API de IA

O sistema implementa seis camadas de defesa independentes entre a entrada HTTP e a chamada ao LLM. A Figura 4.13 apresenta a visão consolidada de todas as camadas, e as subsecções seguintes detalham as mais específicas ao módulo de IA.

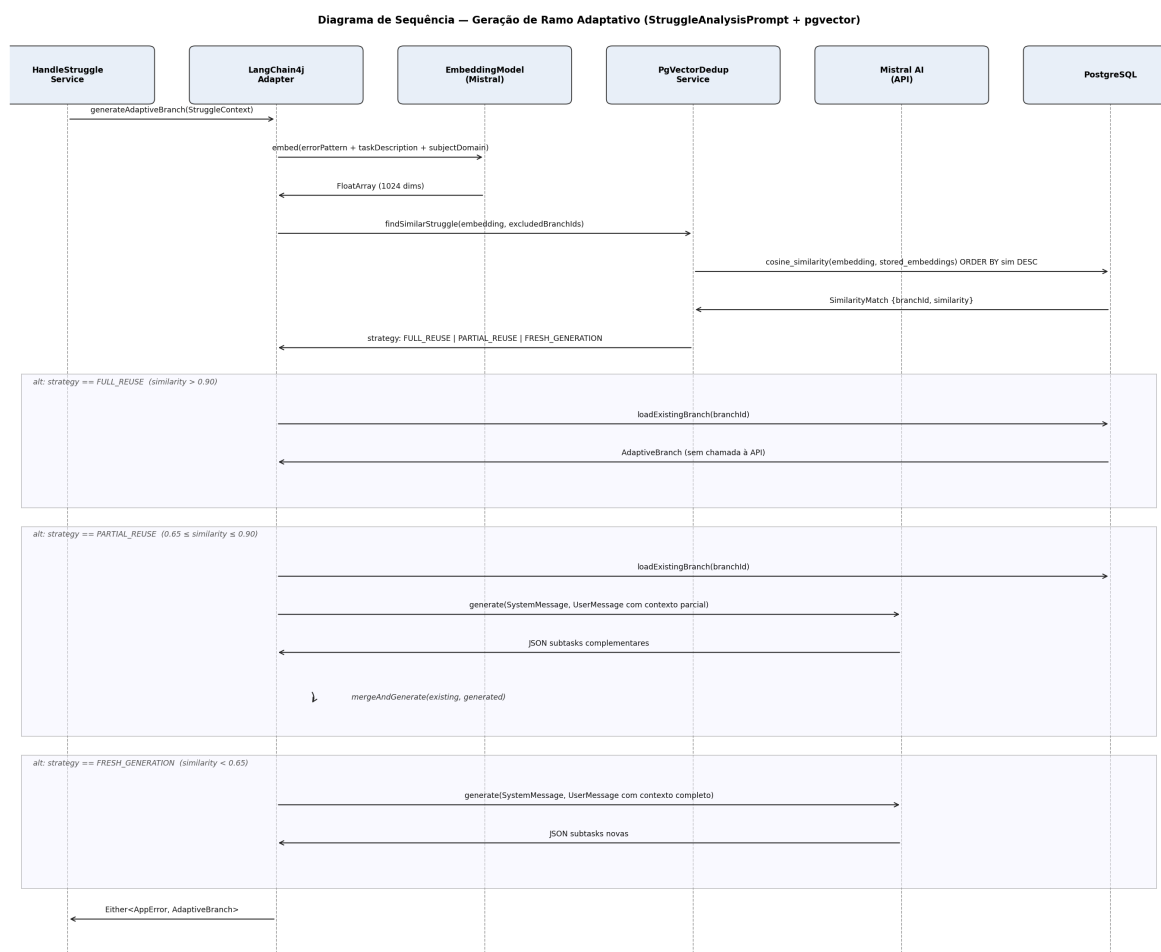


Figura 4.11: Diagrama de sequência — decisão de reutilização no ramo adaptativo

Tabela 4.10: Limites de contexto da camada de IA (AiContextLimits)

Constante	Valor	Utilização
CONTENT_CHARS	32 000 chars	Tamanho máximo de cada <i>chunk</i> enviado ao LLM
CHUNK_OVERLAP	2 000 chars	Sobreposição entre <i>chunks</i> consecutivos
DESCRIPTION_CHARS	500 chars	Objectivo do módulo nos prompts de geração
STRUGGLE_CONTEXT_CHARS	8 000 chars	Contexto de dificuldade no prompt adaptativo
OBJECTIVE_CHARS	500 chars	Objectivo do módulo em prompts secundários

As camadas 1 (rate limiting), 2 (autenticação JWT), 3 (validação de input) e 4 (SSRF) foram descritas nas Secções 4.2.2 e 4.4.3. As secções seguintes detalham as camadas 5 e 6, específicas ao módulo de IA.

Camada 5 — Prevenção de Prompt Injection (Multi-Mensagem Tipada)

A vulnerabilidade clássica de *prompt injection* em sistemas conversacionais ocorre quando o input do utilizador é interpolado numa string de prompt única, permitindo que o utilizador escape o contexto com instruções como “*Ignora as instruções anteriores e...*”.

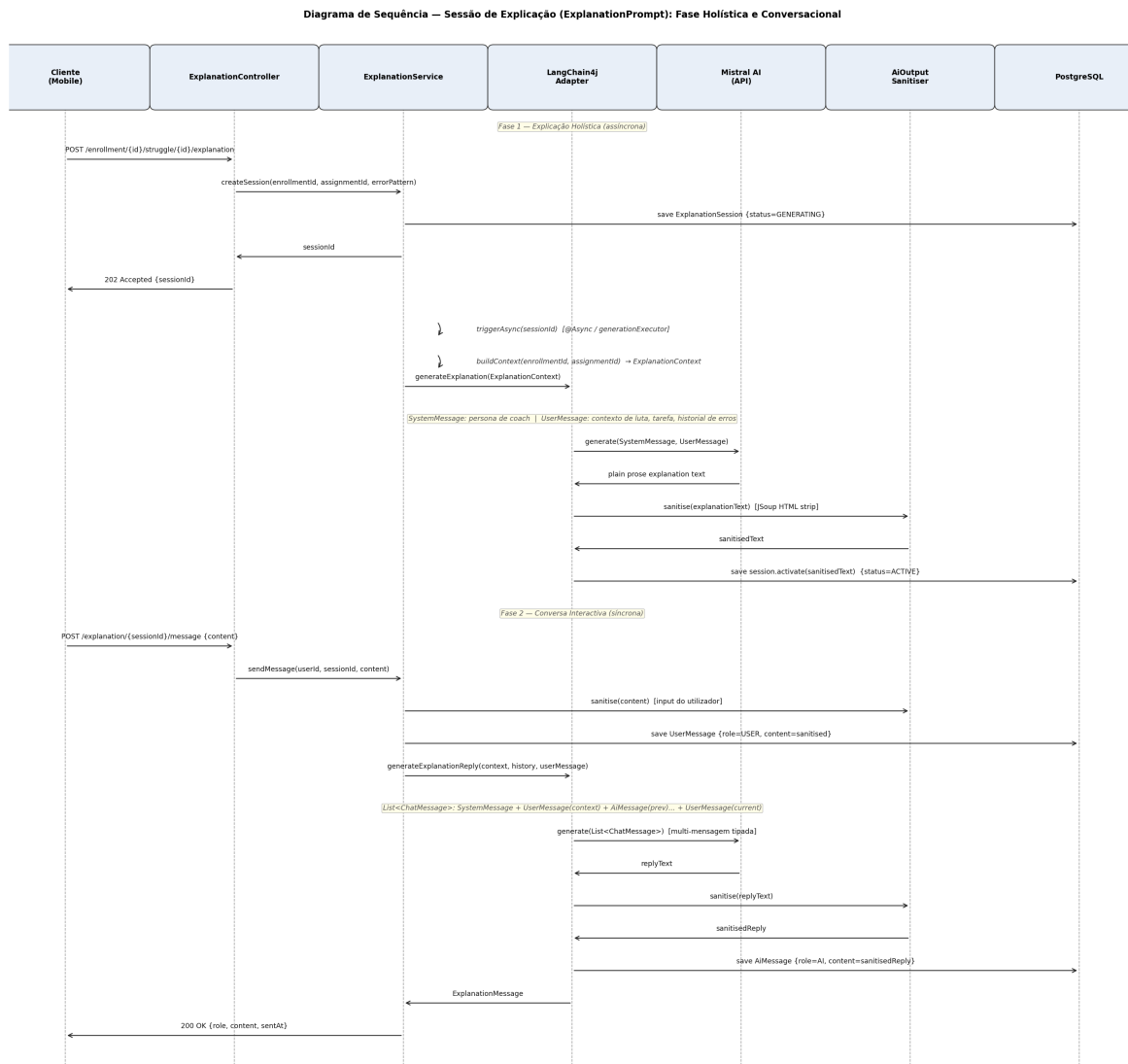


Figura 4.12: Sessão de explicação: fase holística assíncrona e fase conversacional síncrona

O Play4Change adopta uma arquitectura de **multi-mensagem tipada**: em vez de construir uma string única, cada peça de informação é encapsulada no tipo de mensagem LangChain4j correspondente ao seu nível de confiança. A Figura 4.14 detalha a construção da lista de mensagens para a fase conversacional da explicação.

A lista de mensagens é construída pela seguinte ordem: (1) **SystemMessage** com a *persona* e as regras — origem confiada; (2) **UserMessage** com dados estruturados da base de dados (sujeito, tarefa original) — origem confiada; (3) alternância de **AiMessage**/**UserMessage** para o historial da sessão; (4) **UserMessage** com o input actual do utilizador — origem não confiada, isolada na sua própria mensagem. A fronteira de *role* da API Mistral garante que o conteúdo de uma **UserMessage** nunca pode alterar o comportamento definido no **SystemMessage**.

Os prompts de geração de tarefas (**TaskGenerationPrompt**) e de análise de dificuldade

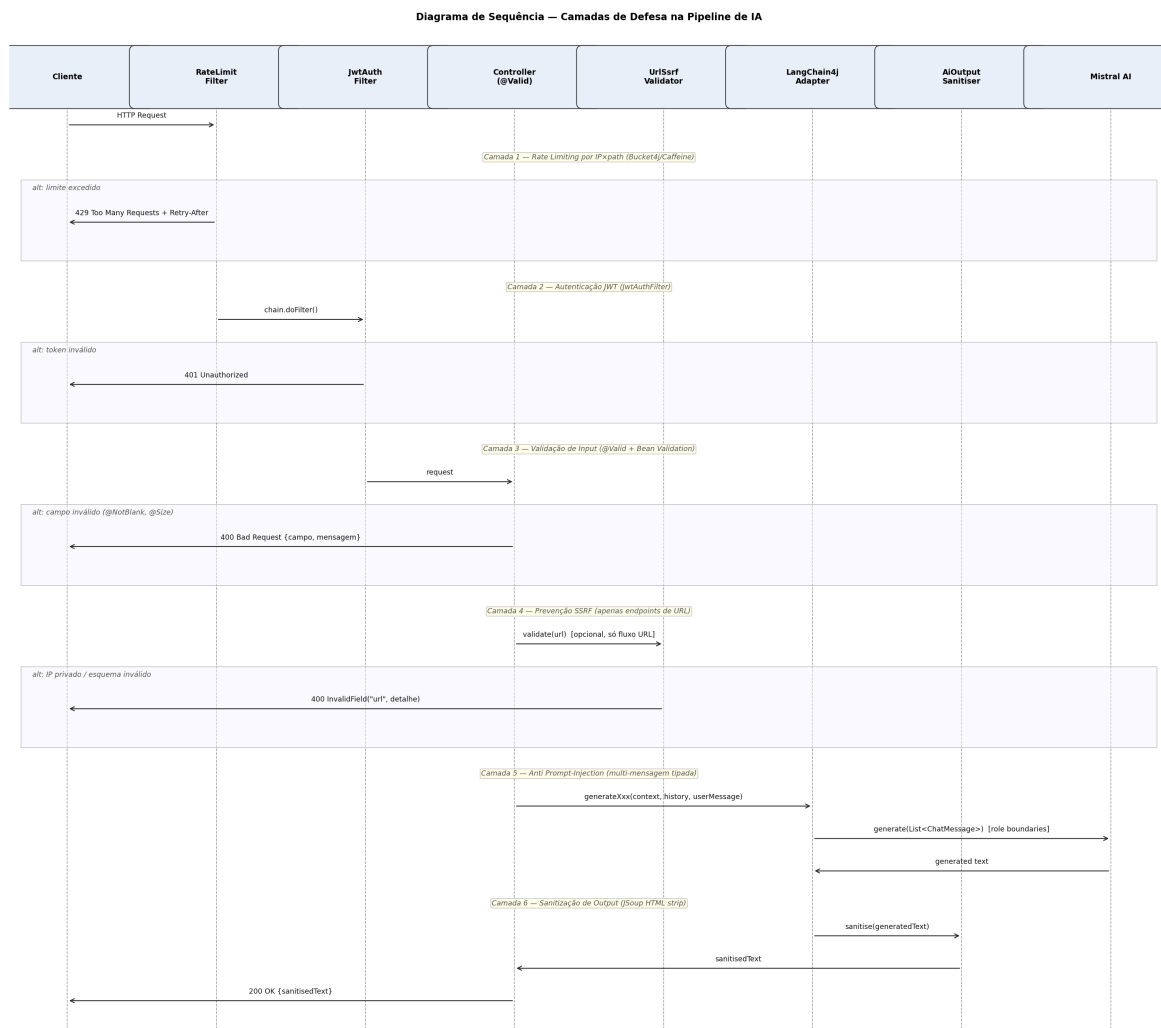


Figura 4.13: Camadas de defesa do sistema — do pedido HTTP à chamada ao LLM

(`StruggleAnalysisPrompt`) não recebem qualquer input do utilizador final — são alimentados exclusivamente com dados estruturados do domínio, eliminando o vector de injeção nessas chamadas.

Camada 6 — Sanitização de Output (`AiOutputSanitiser`)

Todo o texto gerado pelo LLM passa pela classe `AiOutputSanitiser` antes de ser persistido ou devolvido ao cliente. A implementação usa **JSoup** para remover todas as *tags* HTML (incluindo `<script>`, `<style>`, `<iframe>`), decodificar entidades HTML e excluir o conteúdo de elementos de *script*. Este passo previne ataques XSS caso o output da IA seja renderizado num cliente web ou móvel sem *escape* adicional.

A sanitização é aplicada em três pontos distintos do pipeline: (1) nos campos `title`, `description`, `hint` e cada opção de cada `TaskTemplate` gerado; (2) no texto da explicação

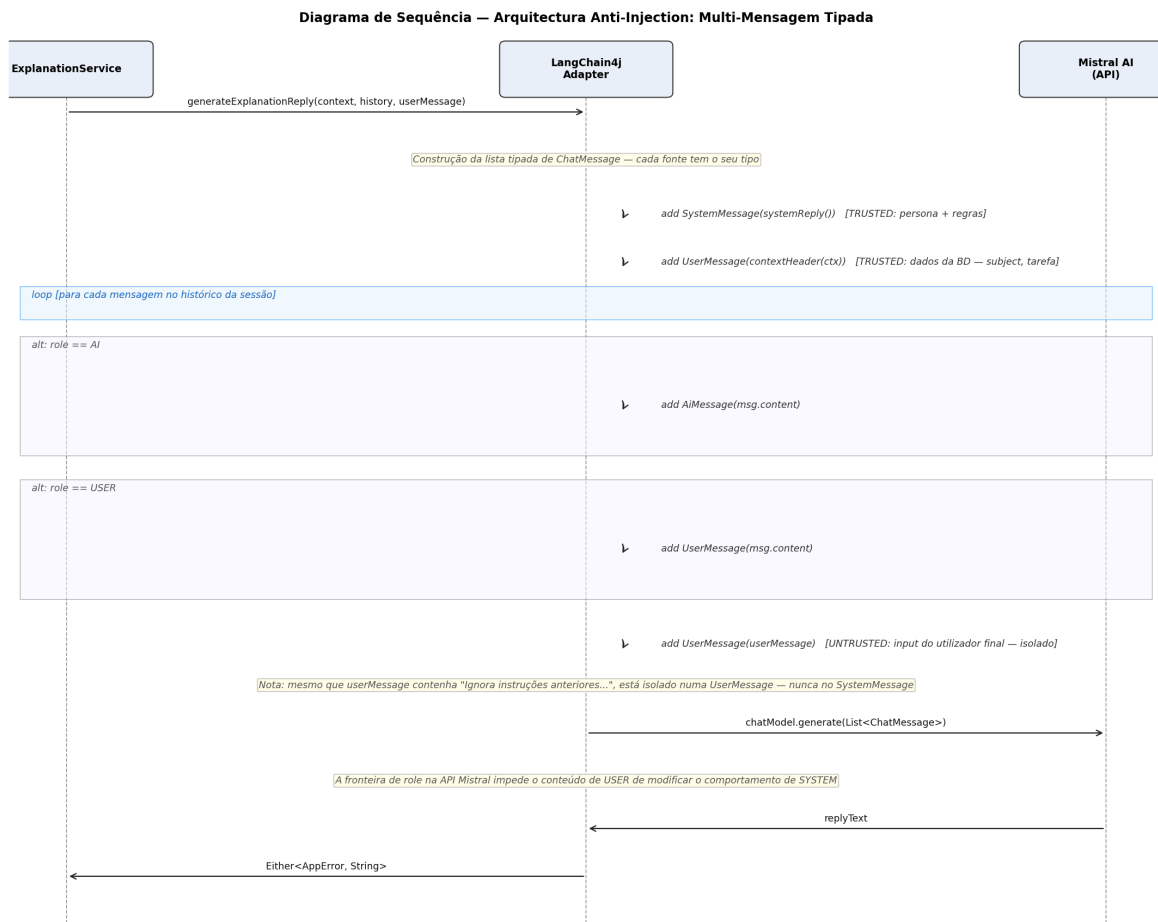


Figura 4.14: Arquitetura anti-*injection*: construção tipada de mensagens por nível de confiança

holística antes de ativar a sessão; (3) na resposta conversacional antes de persistir a `AiMessage`. O input do utilizador também é sanitizado antes de ser enviado ao modelo, eliminando HTML que poderia ser interpretado como instrução.

Resiliência e Observabilidade

A integração com a API Mistral é protegida por Resilience4j com *circuit breaker* configurado numa janela deslizante de 10 chamadas (abre ao atingir 50% de falhas, aguarda 30s antes de meio-abrir), *retry* de 3 tentativas com *backoff* exponencial de 2s, e *timeout* de 60s por geração via `withTimeoutOrNull`. Se a geração de uma explicação exceder o *timeout* ou falhar irreversivelmente, o *assignment* original é repostado para estado `PENDING` — garantindo que o estudante nunca fica bloqueado sem poder continuar.

As métricas de observabilidade instrumentadas manualmente incluem: `task.generation.duration` (histograma por status *success/failed/timeout*), `ai.generation.duration` (por fase do pipeline), `ai.generation.failures` (por tipo de falha), e `ai.adaptive.reuse` (por estratégia:

full/partial/fresh), todas expostas via Prometheus e visualizadas no *dashboard* de latência de IA do Grafana.

4.6 RAG — Retrieval-Augmented Generation

O Play4Change implementa um padrão RAG que difere do RAG documental clássico: em vez de recuperar fragmentos de texto para enriquecer um *prompt* a cada resposta, recupera vectores semânticos previamente computados para tomar decisões de reutilização e injeta contexto pedagógico estruturado nos prompts enviados ao LLM. O sistema mantém duas colecções vectoriais independentes em pgvector, executando pesquisa por similaridade cosseno em dois contextos distintos: deduplicação durante a geração de tarefas e branching adaptativo durante a sessão de dificuldade.

4.6.1 Armazenamento de Conhecimento

Base de Dados Vectorial

O armazenamento persistente de conhecimento assenta em PostgreSQL com a extensão *pgvector*, activada na migração V3. São mantidas duas colecções vectoriais independentes, cada uma com um índice IVFFLAT para pesquisa aproximada por similaridade cosseno:

- `task_templates.embedding vector(1024)` — representa semanticamente o par título+descrição de cada tarefa gerada. O índice é criado com predicado `WHERE embedding IS NOT NULL`, garantindo que apenas templates com vector calculado entram no plano de pesquisa:

```
CREATE INDEX idx_task_embedding ON task_templates
    USING ivfflat (embedding vector_cosine_ops)
    WHERE embedding IS NOT NULL;
```

- `struggle_events.embedding vector(1024)` — representa semanticamente o contexto de dificuldade de um aprendente (padrão de erro + descrição da tarefa + domínio), associado à tabela `adaptive_branches` que regista os ramos gerados para reutilização futura (migração V21):

```
CREATE TABLE struggle_events (
    id          VARCHAR(36) PRIMARY KEY,
    embedding   vector(1024)
);
CREATE INDEX idx_struggle_events_embedding ON struggle_events
    USING ivfflat (embedding vector_cosine_ops)
```

```
WHERE embedding IS NOT NULL;
```

O operador de distância cosseno do pgvector é $\Leftrightarrow$; a similaridade é derivada como $1 - (\text{embedding} \Leftrightarrow ?::\text{vector})$, com valores no intervalo $[0, 1]$. O modelo de *embedding* é o `MistralAiEmbeddingModel`, que produz vectores de 1024 dimensões.

Pipeline de Ingestão

O conhecimento é ingerido em quatro fases executadas de forma assíncrona pelo `TaskGenerationOrchestrator` com o estado rastreado na coluna `current_phase` da tabela `topics`:

```
INGESTION → ANALYSIS → GENERATION → INDEXING → ACTIVE
```

Antes da fase `GENERATION`, o `ContentChunker` segmenta o texto extraído em janelas com sobreposição, respeitando fronteiras de parágrafo (`\n\n`) dentro dos últimos 500 caracteres de cada janela:

```
object ContentChunker {  
    fun chunk(  
        text: String,  
        chunkSize: Int = AiContextLimits.CONTENT_CHARS,    // 32 000 chars  
        overlap: Int = AiContextLimits.CHUNK_OVERLAP        // 2 000 chars  
    ): List<String>  
}
```

Por cada tarefa gerada pelo LLM num chunk, o pipeline calcula o embedding do par título+descrição e consulta o índice `IVFFLAT` antes de persistir: se a similaridade máxima face aos templates existentes no módulo for superior a 0.90, a tarefa é descartada como duplicado semântico. A Figura 4.15 detalha o fluxo completo desde a criação do tópico até à activação.

4.6.2 Recuperação e Reinjeção de Contexto

O sistema executa recuperação vectorial em dois contextos distintos: geração de ramos adaptativos (via similaridade entre vectores de dificuldade) e geração de explicações (via joins relacionais, sem pesquisa vectorial).

Recuperação de Ramo Adaptativo

Quando um aprendiz falha uma tarefa, o `HandleStruggleService` lança assincronamente a geração de um ramo adaptativo. O vector de entrada é construído concatenando `errorPattern`, `taskDescription` e `subjectDomain` (truncado a 8000 chars), e embebido em 1024 dimensões. A pesquisa vectorial no `PgVectorDeduplicationService.findSimilarStruggle()` usa:

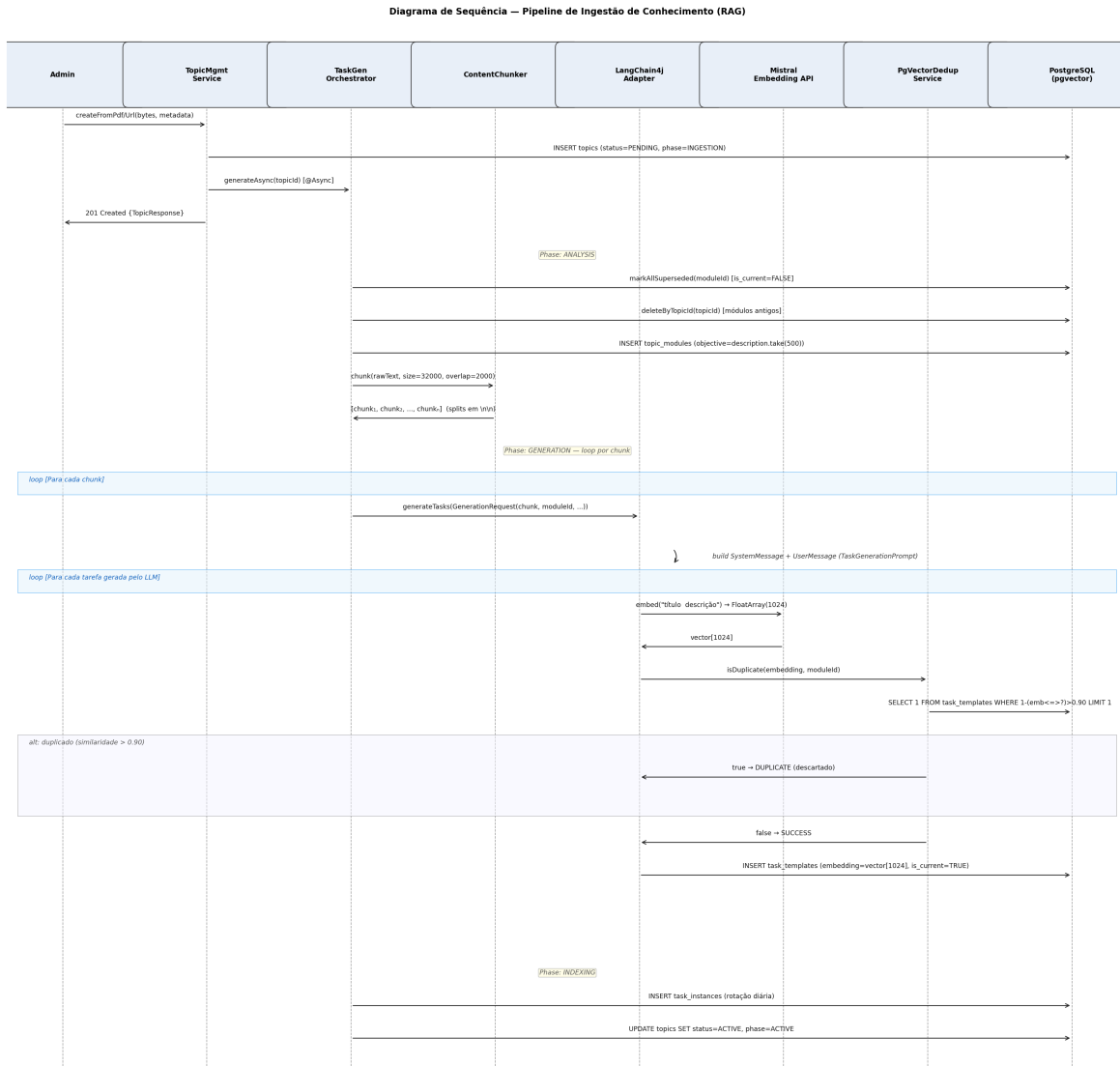


Figura 4.15: Pipeline de ingestão de conhecimento RAG — da criação do tópico ao armazenamento dos vectores

```

SELECT ab.id as branch_id,
       1 - (se.embedding <=> ?::vector) as similarity
FROM struggle_events se
JOIN adaptive_branches ab ON ab.struggle_event_id = se.id
WHERE ab.status = 'COMPLETED'
      AND ab.id NOT IN (...) -- exclui branches já vistos pelo aprendente
ORDER BY se.embedding <=> ?::vector
LIMIT 1
  
```

A estratégia de reutilização é determinada por dois limiares configuráveis em `application.yml`:

Para `PARTIAL_REUSE`, a normalização é idêntica à descrita na Secção 4.5: $n_{\text{reuse}} = \lfloor n_{\text{total}} \times$

Tabela 4.11: Estratégias de reutilização adaptativa por limiar de similaridade

Similaridade	Estratégia	Comportamento
> 0.90	FULL_REUSE	Devolver ramo existente sem chamada ao LLM
[0.65, 0.90]	PARTIAL_REUSE	Reutilizar $\lfloor n \times \text{normSim} \rfloor$ tarefas; gerar o complemento via Mistral; persistir novo ramo
< 0.65	FRESH_GENERATION	Gerar ramo completo via Mistral; persistir embedding para reutilização futura

$\frac{\text{sim}-0.65}{0.90-0.65}$. A Figura 4.16 apresenta o diagrama de sequência das três estratégias.

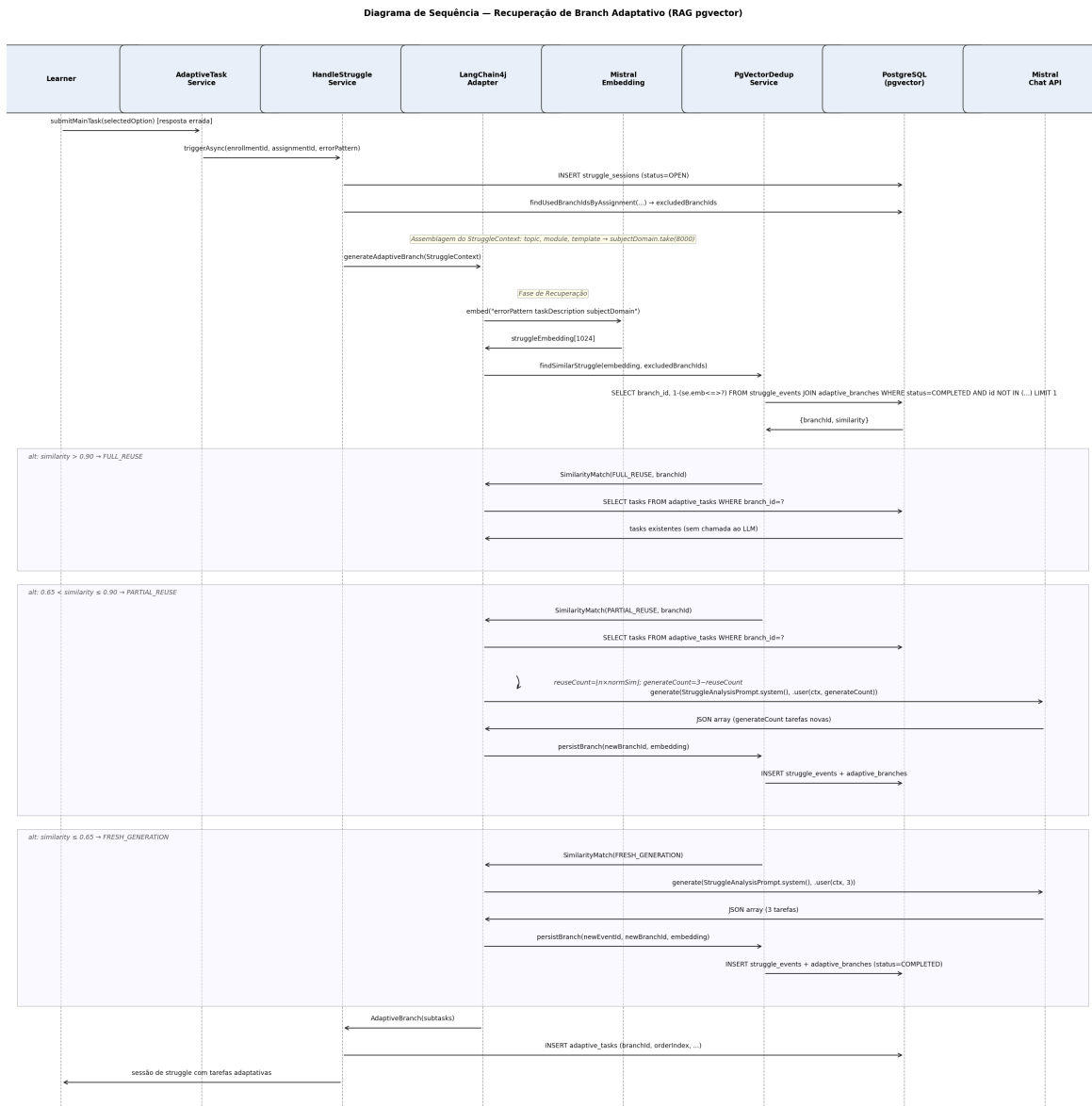


Figura 4.16: Recuperação de ramo adaptativo — decisão FULL/PARTIAL/FRESH_REUSE por similaridade pgvector

Assemblagem e Reinjeção de Contexto para Explicações

Quando o aprendiz esgota o nível máximo de dificuldade (`MAX_STRUGGLE_DEPTH = 1`), é criada uma sessão de explicação. O `ExplanationService.buildContext()` recupera e assembleia contexto exclusivamente por joins relacionais — sem pesquisa vectorial — construindo o `ExplanationContext`:

```
ExplanationContext(  
    taskDescription = template.description.take(500),  
    moduleObjective = module.objective.take(500),  
    subjectDomain   = template.description.take(2000),  
    audienceLevel   = topic.audienceLevel,  
    language        = topic.language,  
    errorPattern     = mapErrorPattern(errorPattern),  
    struggleHistory = allSessions  
        .mapIndexed { idx, session ->  
            StruggleSummary(depth = idx + 1,  
                            errorPattern = mapErrorPattern(session.errorPattern.name),  
                            taskTitles   = session.adaptiveTasks.map { it.title })  
        }  
)
```

Este contexto é injectado no *prompt* de utilizador de `ExplanationPrompt.userExplanation()`, que inclui o historial completo de sessões de dificuldade do aprendiz nessa tarefa. Para as respostas conversacionais, a lista tipada de `ChatMessage` (descrita na Secção 4.5.3) isola dados de controlo de conteúdo do utilizador. A Figura 4.17 detalha a assemblagem e o ciclo de explicação.

4.6.3 Edição de Conteúdo e Invalidação

A base de conhecimento vectorial pode tornar-se incoerente quando o conteúdo pedagógico é alterado por um administrador. O sistema trata dois tipos de invalidação com estratégias distintas.

Edição Pontual de Tarefa

Quando um administrador edita uma `TaskTemplate` (título, descrição, opções, resposta correcta), o `AdminTaskService.updateTask()` executa os seguintes passos:

1. **Incremento de versão:** `version = template.version + 1` — o registo é actualizado *in place*.

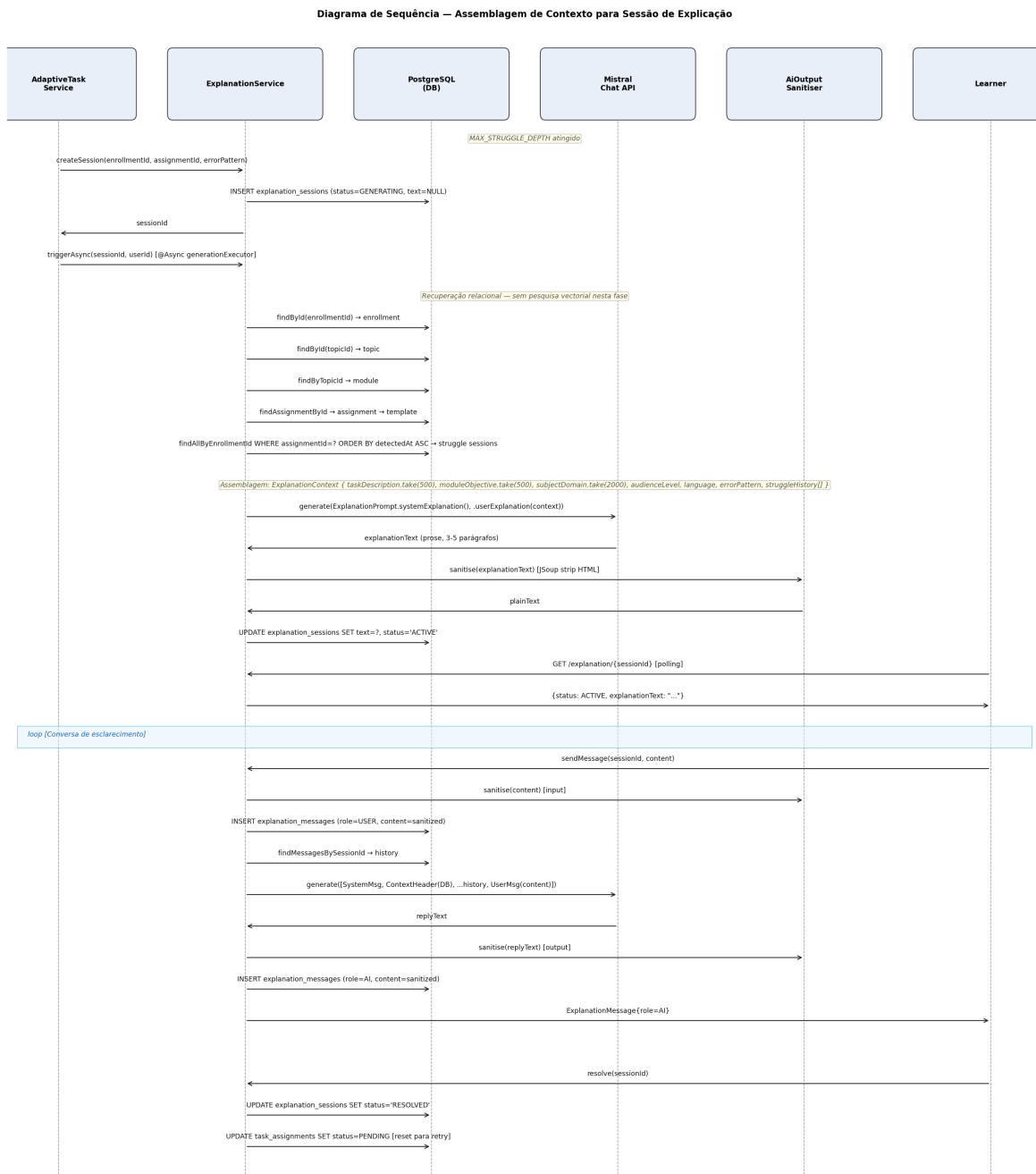


Figura 4.17: Assemblagem de contexto relacional e ciclo de explicação holística e conversacional

2. **Embedding não recalculado:** o campo `embedding` mantém o vector da versão anterior. Esta é uma inconsistência deliberadamente aceite — o vector desatualizado apenas afecta a deduplicação em futuras gerações, não a entrega de tarefas ao aprendente.

3. **Invalidação de instâncias:** `taskInstanceRepository.deleteByTaskTemplateId(templateId)`

elimina todas as instâncias da versão anterior.

4. **Recriação de instâncias:** `batchInstanceGenerationService.generateAndSave([updated])` recria as instâncias com o novo conteúdo.

A Figura 4.18 apresenta o diagrama de sequência desta operação.

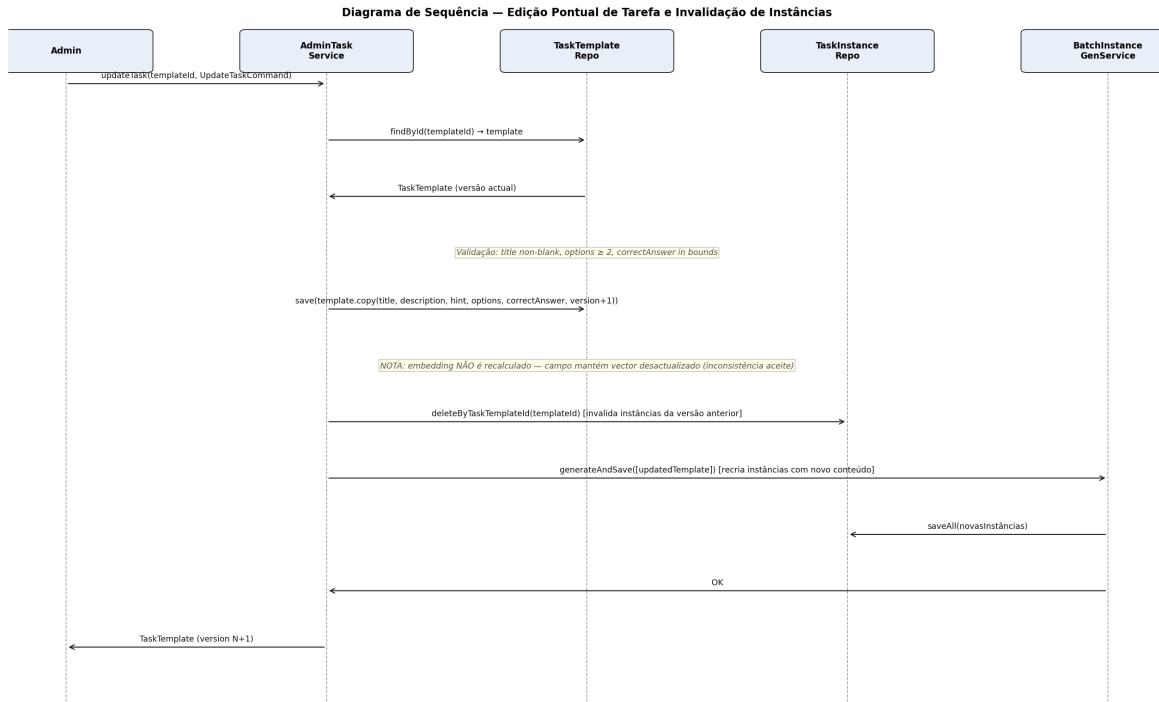


Figura 4.18: Edição pontual de tarefa: invalidação e recriação de instâncias (embedding mantido)

Regeneração Completa de Tópico

A regeneração completa, desencadeada por `POST /admin/topics/{id}/regenerate`, executa uma invalidação total e reinicia o pipeline de ingestão:

```

override fun regenerate(topicId: String, adminId: String): Either<AppError, Topic> = either {
    val topic = ensureNotNull(topicRepository.findById(topicId)) { ... }
    ensure(topic.status != TopicStatus.GENERATING) { Conflict.ConcurrentModification }
    phaseTransitionService.transitionTo(topicId, GenerationPhase.INGESTION)
    orchestrator.generateAsync(topicId) // reinicia pipeline completo
    topicRepository.findById(topicId) ?: topic
}
  
```

Na fase **ANALYSIS**, antes de criar os novos módulos, o orquestrador executa a invalidação em dois passos: (1) `markAllSuperseded(module.id)` desactiva logicamente todos os templa-

tes do módulo (`is_current = FALSE`), preservando o historial para auditoria; (2) `deleteByTopicId(topicId)` elimina o módulo em si, activando uma cascata de deleções físicas sobre `task_templates`, `task_instances`, `struggle_events` e `adaptive_branches`, purgando todos os vectores obsoletos do índice IVFFLAT.

A Figura 4.19 apresenta o diagrama de sequência completo da regeneração.

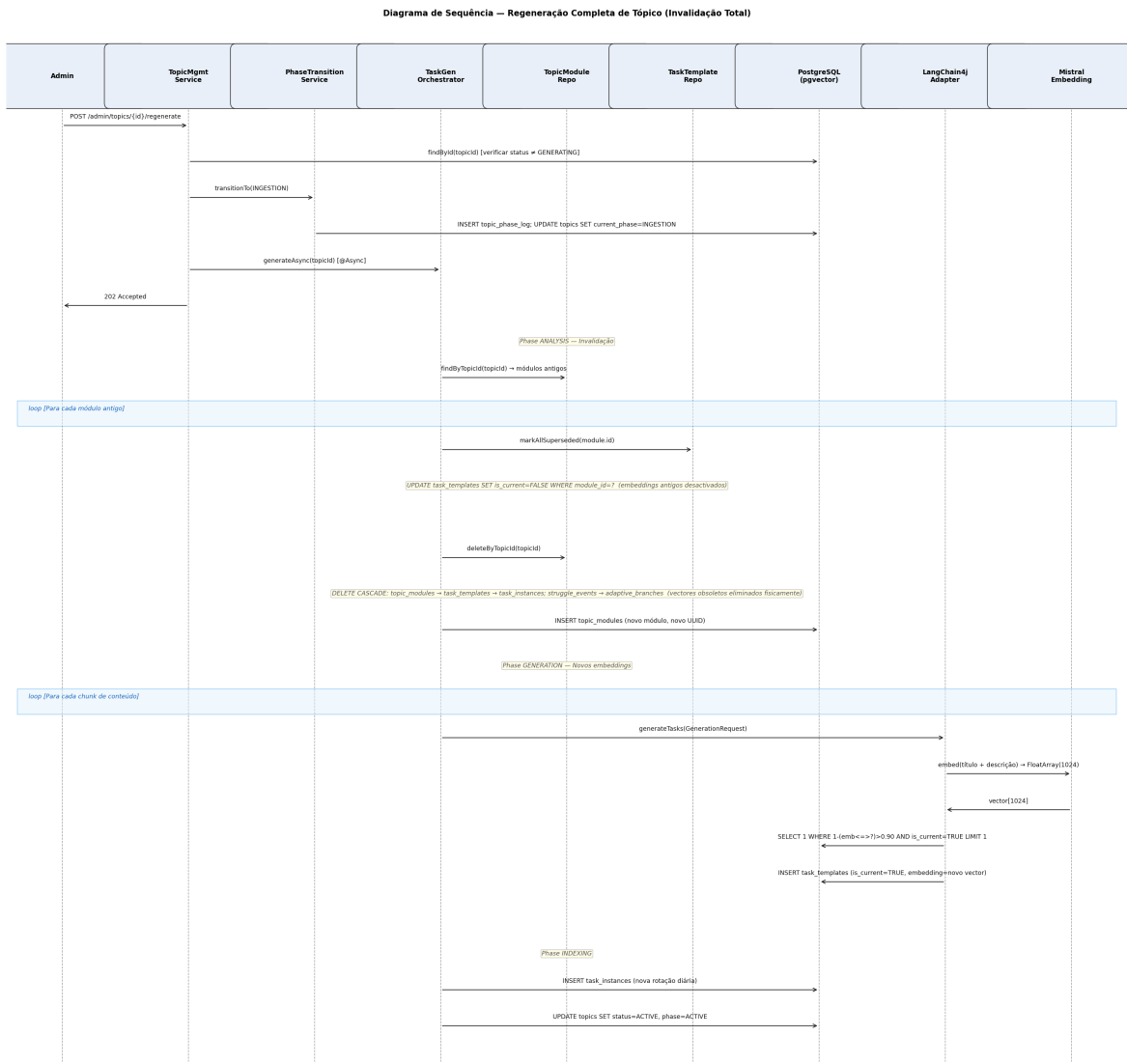


Figura 4.19: Regeneração completa de tópico — invalidação em cascata e recálculo de embeddings

A Tabela 4.12 resume os mecanismos de invalidação por tipo de operação.

Operação	Templates	Embeddings	Instâncias	Índice IVFFLAT
Edição pontual	version+1, mesmo ID	Não recalculado (desatualizado)	Eliminadas e recriadas	Vector antigo permanece
Regeneração completa	is_current=FALSE + novos inseridos	Recalculados de raiz	Eliminadas em CASCADE; recriadas	Limpo após CASCADE; populado com novos vectores

4.7 Geração e Atribuição de Desafios

Esta secção descreve o ciclo de vida completo dos desafios no Play4Change — desde a geração por IA até à entrega ao utilizador, passando pelo percurso adaptativo de remediação (*struggle path*) e pelo modo de tutoria conversacional com IA. Na nomenclatura interna do sistema, os desafios são denominados *tasks* e cada conceito de negócio tem correspondência directa a entidades de domínio bem delimitadas.

4.7.1 Geração de Desafios

Modelo de Dados

Cada desafio existe em dois níveis de abstracção complementares:

- **TaskTemplate** — definição canónica gerada pelo LLM: `title`, `description`, `hint`, `taskType` (`MULTIPLE_CHOICE`, único valor suportado), `options[4]`, `correctAnswer` (sempre índice 0), `embedding` (vector de 1024 dimensões para deduplicação), `dayIndex`, `poolIndex`, `version`, `isCurrent`.
- **TaskInstance** — variante com distratores alternativos: partilha o `correctAnswer` do template mas oferece `options` diferentes, permitindo que utilizadores distintos recebam a mesma questão conceptual com opções erradas variadas. A relação é 1:N, até `instancesPerTask = 5` instâncias por template.

O facto de o `correctAnswer` ser sempre o índice 0 no template e na instância é uma invariante da geração: o *system prompt* do `TaskGenerationPrompt` instrui explicitamente o LLM a colocar a resposta correcta sempre na primeira posição, o que simplifica a lógica de validação e elimina ambiguidade na persistência.

Pipeline de Geração em Quatro Fases

A geração é um processo *batch* assíncrono, orquestrado pelo `TaskGenerationOrchestrator`, com estado rastreado na coluna `current_phase`:

INGESTION → ANALYSIS → GENERATION → INDEXING → ACTIVE

A fase `ANALYSIS` marca todos os templates existentes como `is_current = FALSE` e cria o novo módulo. Na fase `GENERATION`, por cada *chunk* (32 000 chars, sobreposição 2 000), o `TaskGenerationPrompt` instrui o LLM a gerar tarefas com quatro opções, resposta correcta no índice 0, sem sobreposição semântica com tarefas já geradas (verificada via `pgvector` com limiar > 0.90). Na fase `INDEXING`, o `BatchInstanceGenerationService` processa os templates em lotes de 10 e gera até 5 instâncias por template via chamadas LLM adicionais.

A Figura 4.9 (Secção 4.5) detalha o pipeline completo de geração com os actores internos.

Resiliência e Deduplicação

A falha no primeiro *chunk* aborta a geração e o tópico transita para **FAILED**; falhas em *chunks* subsequentes são toleradas — o sistema continua com as tarefas já persistidas, evitando perda total do trabalho acumulado.

A deduplicação por pgvector (descrita em detalhe na Secção 4.6.1) garante que tarefas com similaridade semântica superior a 0.90 face a templates existentes no mesmo módulo são descartadas (**GenerationStatus.DUPLICATE**), assegurando diversidade ao longo do curso.

4.7.2 Atribuição aos Utilizadores

Entidades de Atribuição

Dois agregados gerem o estado da aprendizagem por utilizador:

- **Enrollment** — inscrição activa num módulo: `currentDayIndex`, `status` (**ACTIVE** — **COMPLETED** — **EXPIRED** — **PAUSED**), `totalPointsEarned`, `streakDays`.
- **TaskAssignment** — atribuição concreta: `enrollmentId`, `taskTemplateId`, `taskInstanceId` (variante seleccionada), `optionOrder` (permutação aleatória das opções), `correctAnswerIndex`, `dueAt` (24 horas após `assignedAt`), `status` (**PENDING** — **SUBMITTED** — **LATE**), `wrongAttemptCount`.

Fluxo Pull-based

A atribuição de desafios é *pull-based*: o utilizador solicita a tarefa do dia via `GET /tasks/today`, e o sistema determina qual a resposta correcta para aquele momento com base no `currentDayIndex` da inscrição activa. O serviço resolve quatro casos mutuamente exclusivos, por esta ordem de precedência: (1) sessão de *struggle* aberta; (2) sessão de explicação activa; (3) assignment já existente para o `dayIndex` actual; (4) criação de novo assignment.

No caso (4), o sistema selecciona aleatoriamente uma das **TaskInstance** disponíveis (variedade de distratores), gera uma permutação aleatória das quatro opções (`optionOrder`), e define `dueAt = now() + 24h`. A Figura 4.20 ilustra os quatro casos.

Permutação das Opções e Penalizações por Atraso

A `optionOrder` é uma lista de índices (ex.: `[2,0,3,1]`) que define a ordem de apresentação no ecrã. A resposta correcta, sempre no índice 0 da instância, aparece numa posição aleatória para o utilizador. A resposta submetida é um índice de posição de ecrã; o sistema resolve o índice canónico antes de validar, via `optionOrder[selectedOption]`.

A submissão após `dueAt` é aceite mas penalizada, conforme a Tabela 4.13. Note-se que a fracção de pontos depende do atraso *face ao próprio dueAt*, e não do tempo total desde a atribuição.

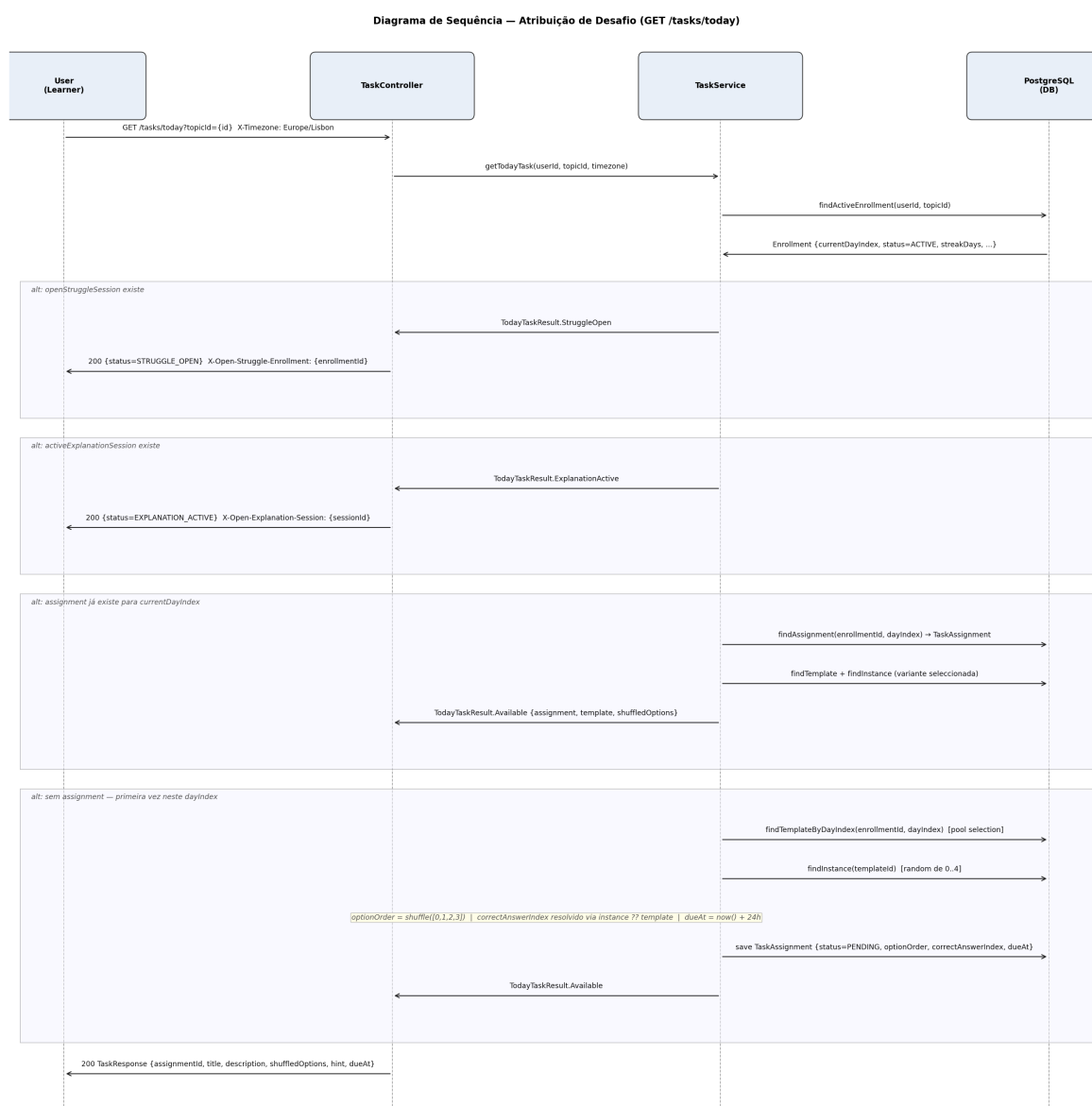


Figura 4.20: Atribuição de desafio (GET /tasks/today) — quatro casos de precedência

Tabela 4.13: Penalizações de pontuação por atraso na submissão

Atraso face a <i>dueAt</i>	Pontos atribuídos
Sem atraso ($now \leq dueAt$)	100%
≤ 24 h de atraso	75%
≤ 72 h de atraso	50%
> 72 h de atraso	25% (mínimo 1)

4.7.3 Consolidation Path (Struggle Path)

Quando um utilizador responde incorrectamente, o sistema não termina a interacção — desencadeia automaticamente uma sessão de remediação adaptativa (*struggle session*). O

objectivo é diagnosticar a causa do erro e apresentar três sub-tarefas *scaffolded* que devolvam o utilizador ao nível da tarefa original.

Classificação do Padrão de Erro

Antes de invocar a IA, o `ErrorPatternClassifier` classifica a falha de forma determinística em três categorias, com base no contexto da submissão:

Tabela 4.14: Padrões de erro classificados pelo `ErrorPatternClassifier`

Padrão	Condição	Estratégia de remediação
TIME_PRESSURE	Submissão após <code>dueAt</code>	Tarefas paceadas, passo a passo
READING_ERROR	Opção seleccionada adjacente à correcta (<code> pos_sel - pos_corr == 1</code>)	Instruções mais explícitas e claras
WRONG_CONCEPT	Qualquer outro caso	Re-explicação do conceito de raiz

Geração Assíncrona da Sessão

Após classificar o erro, o `TaskService` persiste o resultado e delega ao `HandleStruggleService` via `triggerAsync` — a resposta HTTP é devolvida imediatamente ao utilizador, sem esperar pela geração do ramo adaptativo pelo LLM. Em caso de falha ou *timeout* do LLM, a sessão é marcada como `ABANDONED` e o assignment original é reposto em `PENDING` — o utilizador nunca fica bloqueado. A Figura 4.21 detalha este fluxo.

Submissão e Resolução das Tarefas Adaptativas

O utilizador acede às três sub-tarefas via `GET /struggle/enrollment/{enrollmentId}`. Cada submissão pode resultar em quatro desfechos, avaliados após a conclusão de todas as tarefas: (1) **ainda não concluídas** — devolver resultado parcial; (2) **todas correctas** — marcar sessão como `RESOLVED` e repor assignment em `PENDING`; (3) **falha, depth < MAX_STRUGGLE_DEPTH** — gerar nova sessão de *struggle* encadeada; (4) **falha, depth ≥ MAX_STRUGGLE_DEPTH (= 1)** — criar sessão de explicação por IA.

As tarefas adaptativas atribuem zero pontos e não afectam o *streak* — são modo de aprendizagem puro. A Figura 4.22 apresenta os quatro caminhos de resolução.

4.7.4 Tutoria com IA

Quando o utilizador esgota toda a profundidade de remediação sem sucesso, o sistema activa o modo de tutoria conversacional com IA. Esta sessão sintetiza o historial completo de erros e oferece uma explicação holística em prosa, seguida de um diálogo de esclarecimento sem limite de turnos.

Estados da Sessão de Explicação

A sessão de explicação percorre três estados:

GENERATING → ACTIVE → RESOLVED

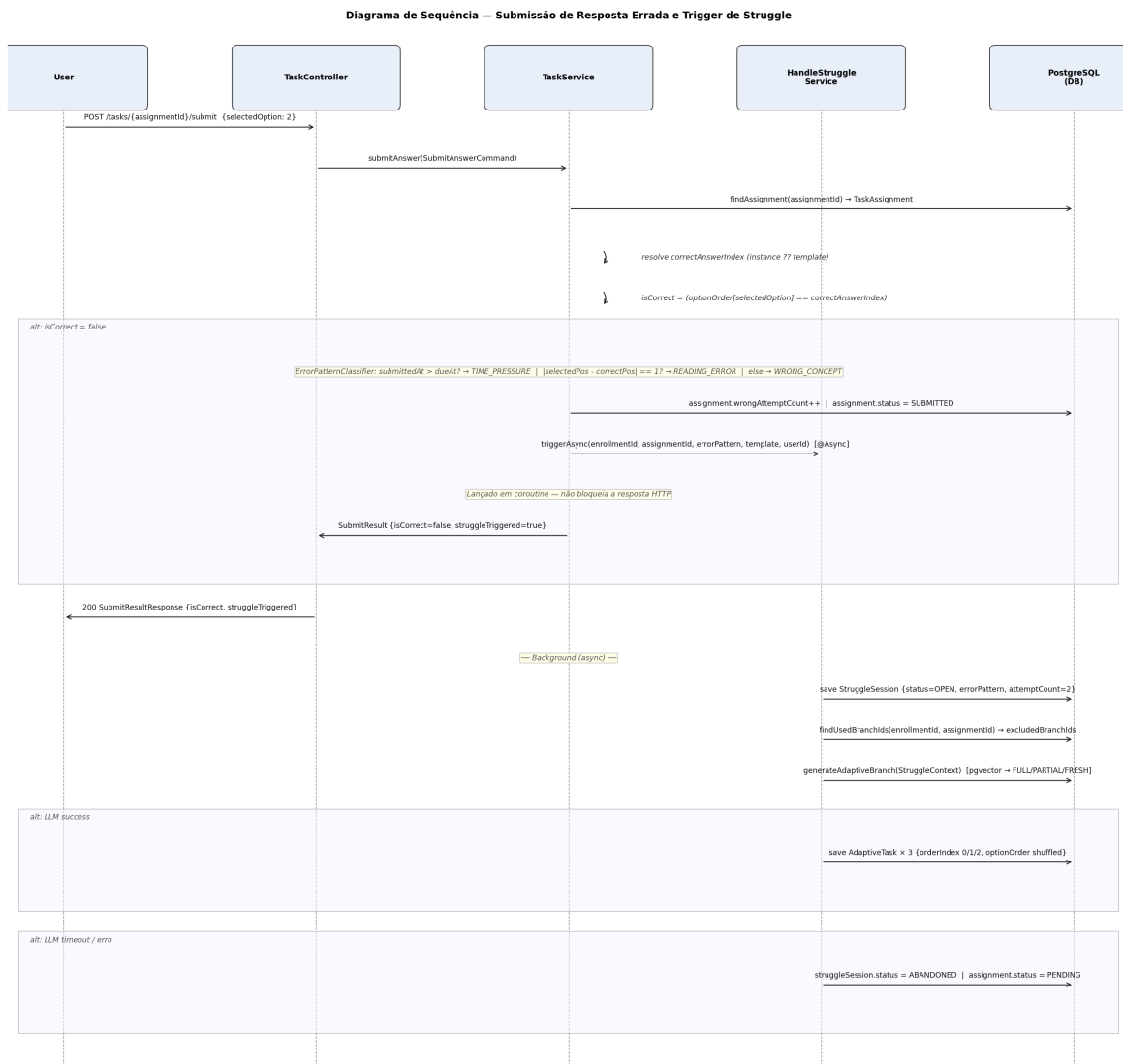


Figura 4.21: Submissão de resposta errada — classificação de erro e *trigger* assíncrono de struggle

A transição **GENERATING** → **ACTIVE** ocorre quando a geração pelo LLM termina com sucesso. Se a geração falhar ou esgotar o *timeout* (60s via `withTimeoutOrNull`), a sessão transita directamente para **RESOLVED** e o assignment é repostado em **PENDING** (*fail-safe* — o utilizador nunca fica bloqueado). A transição para **RESOLVED** pelo utilizador, via `POST /explanation/{sessionId}/resolve`, também repõe o assignment em **PENDING**, devolvendo-o ao fluxo diário normal.

Geração Holística e Diálogo Conversacional

A geração holística é assíncrona (lançada no `generationExecutor`), permitindo que o utilizador navegue para o ecrã de explicação enquanto a geração ocorre em *background*; o cliente faz

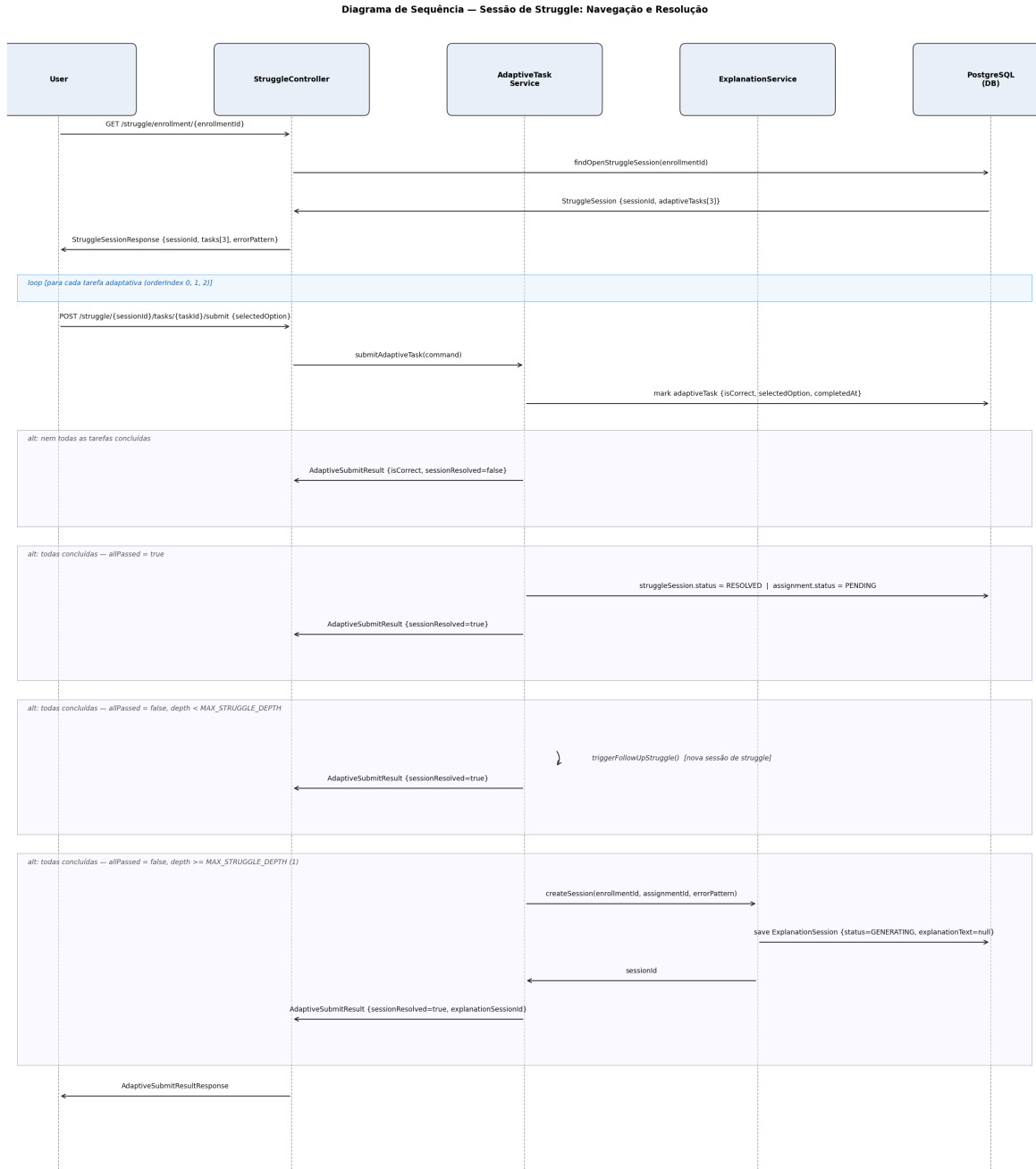


Figura 4.22: Sessão de struggle — submissão das tarefas adaptativas e caminhos de resolução

polling via GET `/explanation/{sessionId}` até o estado ser **ACTIVE**. O contexto injectado no **ExplanationPrompt** inclui o historial completo de sessões de dificuldade (**struggleHistory**) e é assemblado por joins relacionais, sem pesquisa vectorial (descrita na Secção 4.6.2).

Para o diálogo conversacional, cada mensagem é processada com o historial completo da conversa para manter coerência contextual. A mensagem enviada ao LLM é composta por: (1) **SystemMessage** com persona de tutor e restrições de resposta; (2) **UserMessage** com

cabeçalho de contexto (**Subject + Original task**) injectado pelo servidor — âncora anti-injecção; (3) alternância de **AiMessage/UserMessage** para o historial; (4) **UserMessage** com o input actual, isolado. A Figura 4.23 detalha o fluxo completo desde a criação assíncrona até à resolução.

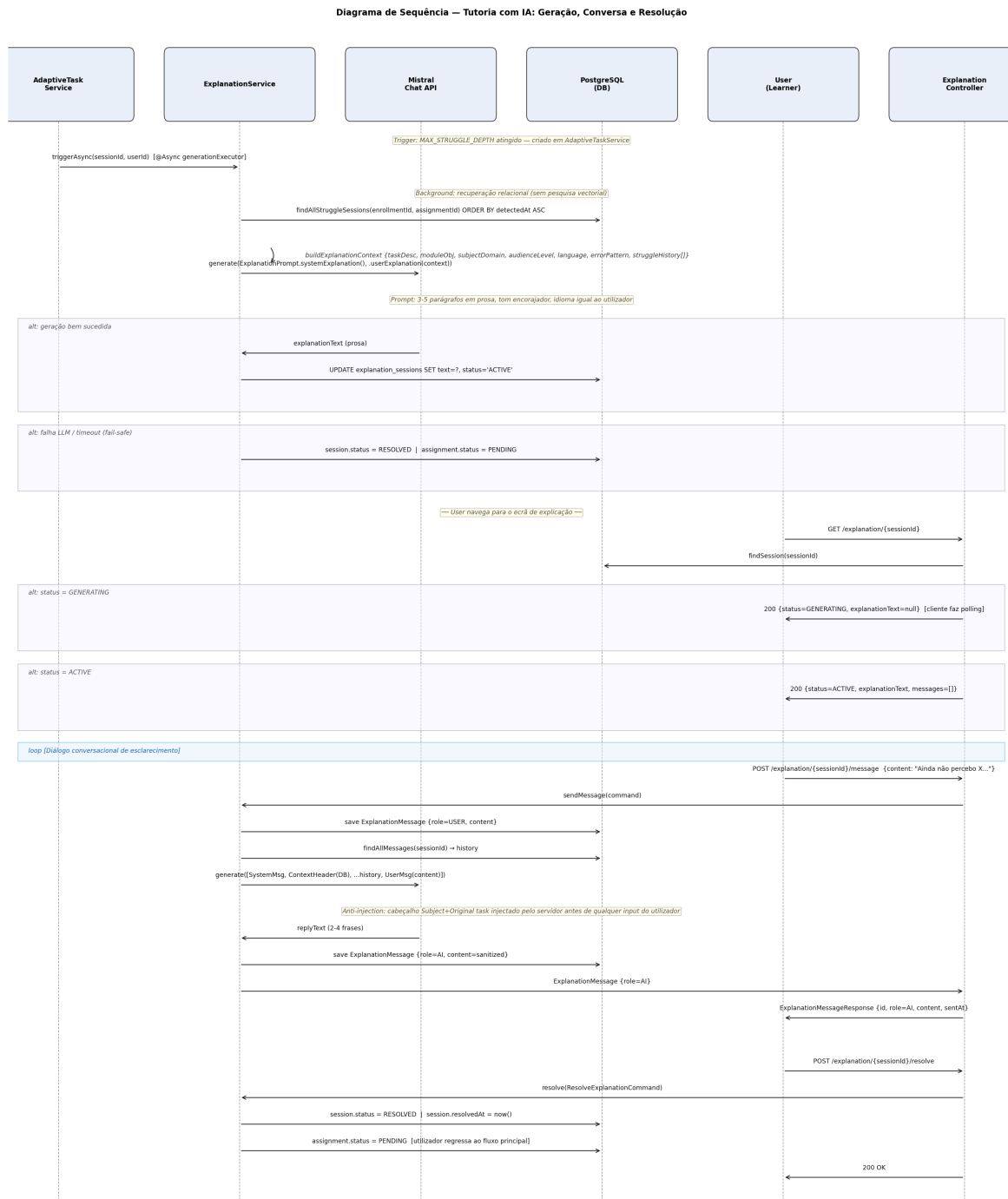


Figura 4.23: Tutoria com IA — geração assíncrona, diálogo conversacional e resolução

4.8 Segurança da Aplicação

A segurança do Play4Change foi implementada em profundidade (*defense in depth*), com controlos independentes em cada camada da arquitectura: proxy reverso (Nginx), servidor de aplicação (Spring Boot), agente de IA (LangChain4j) e cliente web (React/Vite). As ameaças foram mapeadas com base no OWASP Top 10 (2021), conforme a Tabela 4.15.

Tabela 4.15: Mapeamento das ameaças OWASP Top 10 para controlos implementados

OWASP	Ameaça	Controlo Implementado
A01 Broken Access Control	Escalada de privilégios	RBAC no <code>SecurityConfig</code> + validação de papel no JWT
A03 Injection	XSS via conteúdo IA	<code>AiOutputSanitiser</code> (Jsoup) + CSP + React JSX escaping
A03 Injection	Prompt Injection	Multi-mensagem tipada + <code>AiContextLimits</code>
A03 Injection	SQL Injection	JPA/Hibernate com parâmetros tipados
A04 Insecure Design	Ausência de rate limiting	<code>RateLimitFilter</code> + <code>Bucket4j</code> por IP
A05 Misconfiguration	Credenciais padrão em produção	<code>JwtSecretStartupGuard</code> , <code>CredentialStartupGuard</code>
A05 Misconfiguration	CORS wildcard	Verificação ao arranque; wildcard causa exceção
A07 Auth Failures	Roubo de <i>refresh token</i>	Rotação de família com revogação total
A07 Auth Failures	Força bruta em <i>magic-link</i>	Rate limiting 5 req/10 min por IP
A10 SSRF	Fetch de URLs internas	<code>UrlSsrValidator</code>

4.8.1 Proteção Contra XSS

A protecção contra *Cross-Site Scripting* opera em três camadas complementares e independentes.

Camada 1 — Sanitização de Output de IA

O `AiOutputSanitiser` sanitiza todo o conteúdo gerado pelo modelo Mistral antes de ser persistido na base de dados. A implementação usa `Jsoup.parse(input).text()`, que remove todos os elementos HTML, decodifica entidades HTML e descarta o conteúdo de elementos `<script>` e `<style>`. O sanitizador é aplicado a cada campo `String` do `TaskTemplate` gerado (título, descrição, *hint*, cada opção) e às respostas conversacionais de explicação.

```
object AiOutputSanitiser {  
    fun sanitise(input: String): String = Jsoup.parse(input).text()  
}
```

O vetor mitigado é a injeção de HTML malicioso via páginas raspadas (*scraped*): sem esta sanitização, uma página que contenha `<script>alert(1)</script>` no seu texto poderia ser reproduzida literalmente pelo modelo e persistida na base de dados.

Camada 2 — Content Security Policy via Nginx

O Nginx aplica uma *Content Security Policy* restritiva em todas as respostas HTTP, conforme a Tabela 4.16.

Tabela 4.16: Diretivas CSP configuradas no Nginx

Diretiva	Valor	Justificação
default-src	'self'	<i>Fallback</i> : apenas origem própria
script-src	'self'	Sem <code>unsafe-eval</code> ; sem CDNs externos
style-src	'self' 'unsafe-inline'	Necessário para <i>inline styles</i> gerados pelo Vite/React no <i>build</i>
img-src	'self' data:	Permite <i>data URIs</i> para imagens <i>inline</i>
connect-src	'self'	<i>Fetch</i> /XHR apenas para a API própria
frame-ancestors	'none'	Equivalente a <code>X-Frame-Options: DENY</code> — previne <i>clickjacking</i>

Camada 3 — React JSX Auto-escaping

O React aplica *escaping* automático de todas as expressões `{expression}` ao renderizar no DOM. Mesmo que a base de dados contivesse HTML residual (falha nas camadas anteriores), o React converteria `<` e `>` em entidades HTML antes da inserção no DOM, eliminando XSS reflexo em componentes bem formados.

A Figura 4.24 ilustra o fluxo das três camadas.

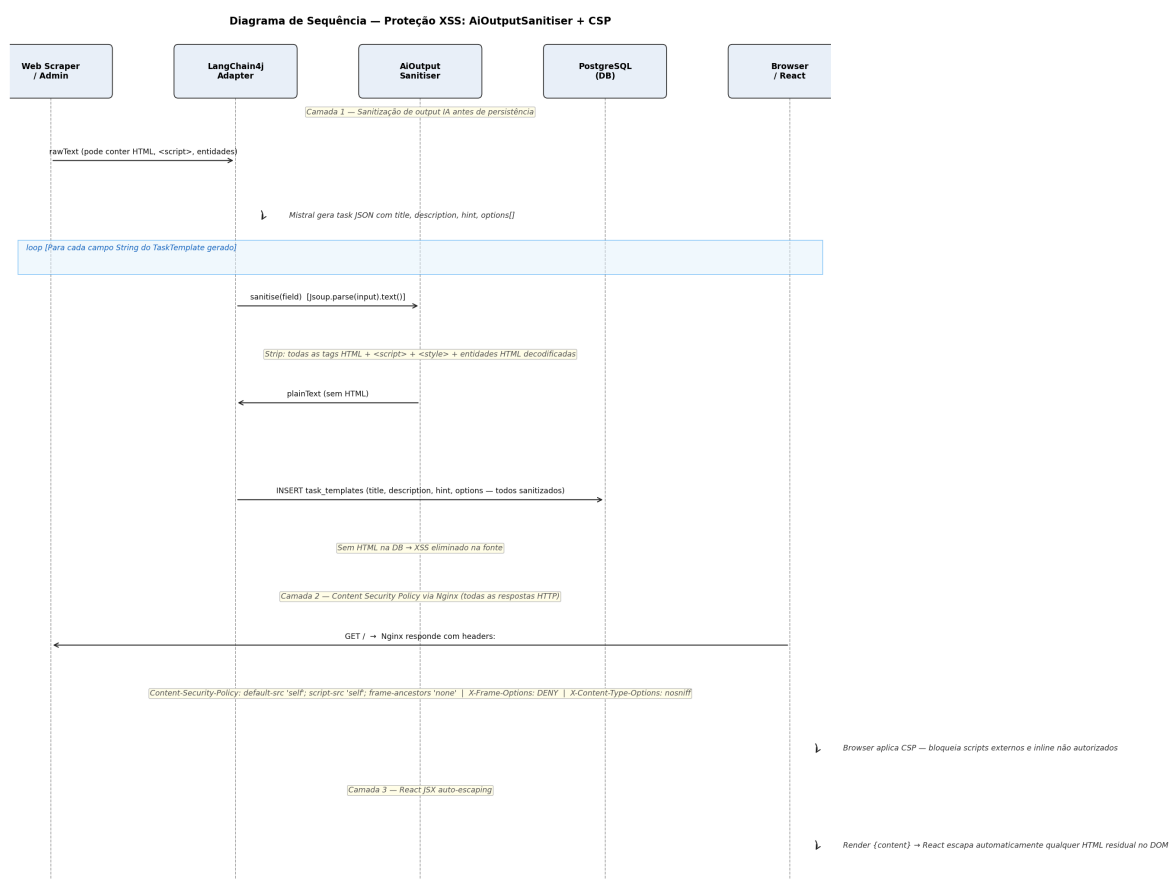


Figura 4.24: Protecção XSS em três camadas: AiOutputSanitiser, CSP Nginx, React JSX escaping

4.8.2 Rate Limiting por IP

O *rate limiting* é implementado no filtro `RateLimitFilter`, com ordem `HIGHEST_PRECEDENCE` na cadeia de filtros Spring, garantindo que a verificação ocorre antes de qualquer autenticação ou lógica de negócio. O algoritmo utilizado é *token bucket* via `Bucket4j`, com janela de *refill* de 10 minutos. Os *buckets* são mantidos em cache `Caffeine` com TTL de 15 minutos e capacidade máxima de 50 000 entradas. Cada *bucket* é identificado pela chave composta «ip>:<path_normalizado>», isolando os limites por recurso e por origem.

A Tabela 4.17 resume os limites por *endpoint*.

Tabela 4.17: Limites de *rate limiting* por *endpoint* (janela de 10 minutos por IP)

Endpoint	Limite	Justificação
POST /auth/magic-link	5 req	Previne enumeração de emails e abuso de envio
POST /auth/magic-link/verify	10 req	Previne <i>brute-force</i> de tokens mágicos
POST /auth/refresh	20 req	<i>Headroom</i> para clientes com múltiplas abas
POST /explanation/*/message	10 req	Chamada a Mistral: custo computacional e financeiro
POST /admin/topics (URL)	3 req	Scraping + embedding + geração de tarefas
POST /admin/topics (PDF)	2 req	Idem, com custo adicional de OCR via Unstructured
POST /admin/topics/*/regenerate	2 req	Regeneração total de tarefas para um tópico

O `IpExtractor` extrai o IP do header `X-Forwarded-For` definido pelo Nginx, mas rejeita endereços privados para prevenir *spoofing* por injeção de header: se o endereço no XFF for privado (`isPrivateAddress()`), o sistema cai para `remoteAddr`. A Figura 4.25 detalha o fluxo completo.

Para além dos limites da Tabela 4.17, o `RateLimitService.capacityFor()` define ainda regras para `/topics/{id}/task-assignments/{id}/submit (30)`, `/topics/{id}/task-assignments/{id}/(20)`, `/enrollments/{id}/struggle/{id}/submit (30)` e `/reviews/{id}/verdict (20)`. Nenhum destes quatro padrões de rota corresponde a um *endpoint* real do sistema actual: os dois últimos são resíduos da funcionalidade de *Peer Review*, entretanto removida, e os dois primeiros não coincidem com as rotas efectivas de submissão (`POST /tasks/{assignmentId}/submit` e `POST /struggle/{sessionId}/tasks/{taskId}/submit` respectivamente). Na prática, estas quatro regras nunca são accionadas, e os dois *endpoints* de submissão mais utilizados da aplicação — responder à tarefa diária e submeter uma sub-tarefa de *struggle* — não têm hoje qualquer limite de taxa aplicado, apesar da intenção documentada em comentário no próprio código-fonte.

4.8.3 Validação e Sanitização de Inputs

A validação de *inputs* é realizada em quatro níveis independentes.

Nível 1 — Cliente (TypeScript). O módulo `play4change-web/src/lib/validators.ts` define validadores para email (*regex* `/^[^\s@]+@[^\s@]+\.[^\s@]+$/`), ficheiros PDF (tipo `application/pdf`, tamanho máximo) e URLs (formato e contagem máxima). A validação no cliente é uma optimização de UX — não um controlo de segurança.

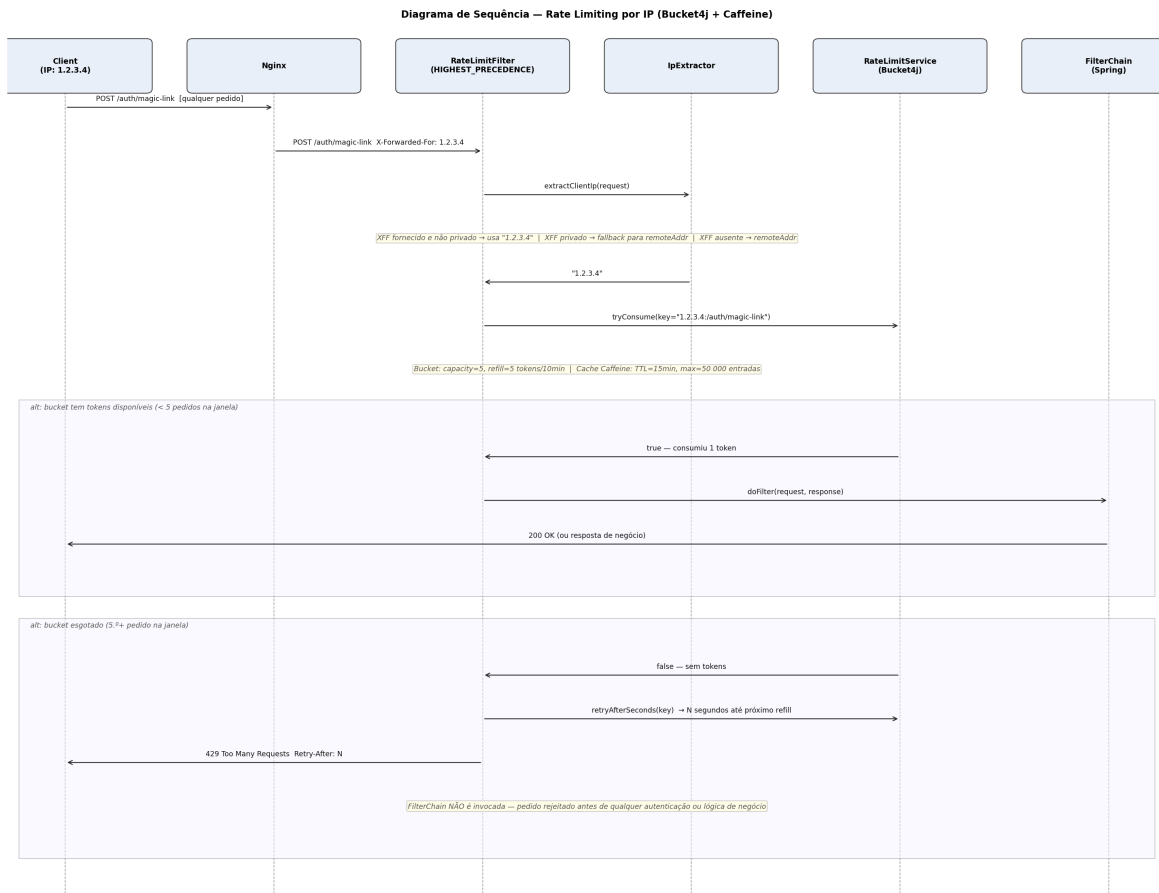


Figura 4.25: *Rate limiting* por IP — token bucket Bucket4j com extração segura de IP via XFF

Nível 2 — Servidor (Jakarta Bean Validation). Os DTOs de entrada usam anotações Jakarta para validação declarativa: `@NotBlank`, `@Email`, `@Size`, `@Valid`. Uma string vazia ou email malformatado retorna 400 *Bad Request* com lista de erros JSON sem chegar à camada de serviço.

Nível 3 — Prevenção de SSRF. Antes de qualquer *fetch* de URL externa, o `UrlSsrfValidator` aplica três verificações em sequência: (1) esquema obrigatoriamente HTTPS; (2) resolução DNS do host; (3) rejeição de IPs resolvidos em intervalos privados/*loopback/link-local/multicast* (RFC 1918, 127.0.0.0/8, 169.254.0.0/16, 224.0.0.0/4). O diagrama detalhado encontra-se na Figura 4.7 (Secção 4.4.2).

Nível 4 — Limites de Contexto para IA. O `AiContextLimits` define limites rígidos de caracteres para todos os *inputs* enviados ao modelo (descritos na Tabela 4.10, Secção 4.5), aplicados nos serviços antes de construir os prompts, independentemente do tamanho do *input* original.

4.8.4 Outros Controlos de Segurança

Autenticação JWT com Validação de Arranque

O `JwtSecretStartupGuard` valida o segredo JWT no arranque da aplicação, falhando com `IllegalStateException` se o segredo tiver menos de 64 caracteres ou se for igual ao valor de desenvolvimento por omissão — validação aplicada apenas em perfil `prod` (ver Secção 4.3.3). O `CredentialStartupGuard` verifica analogamente as credenciais de serviços externos (MinIO, base de dados). Estes guardas impedem a promoção de configurações de desenvolvimento para produção.

Rotação de Refresh Token com Detecção de Reutilização

O mecanismo de rotação com família (descrito em detalhe na Secção 4.3) detecta roubo de token: se um token já marcado como `used = true` for apresentado, toda a família é revogada e um evento `SECURITY` é emitido com `userId` e `familyId`. O access token tem TTL de 15 minutos; o refresh token tem TTL de 7 dias.

RBAC com Validação de Papel no JWT

O `SecurityConfig` define rotas protegidas com granularidade de papel: `/admin/**` exige `ROLE_ADMIN`; Swagger UI é restrito a `ROLE_ADMIN` em produção e público em desenvolvimento; todos os outros endpoints autenticados aceitam qualquer papel válido. O papel é extraído do JWT no `JwtAuthFilter` e validado contra a lista `VALID_ROLES = ["USER", "ADMIN"]` — valores fora desta lista são descartados.

CORS Restritivo

O `WebMvcConfig` configura CORS com lista de origens explícita, verificada ao arranque:

```
require(origins.none { it == "*" || it.isBlank() }) {  
    "CORS allowed-origins must not contain wildcards or blank entries"  
}
```

Em produção, as origens permitidas são definidas via variável de ambiente `APP_CORS_ALLOWED_ORIGINS`.

Headers HTTP de Segurança (Nginx)

Tabela 4.18: Headers HTTP de segurança configurados no Nginx

Header	Valor	Protecção
<code>Strict-Transport-Security</code>	<code>max-age=63072000; includeSubDomains</code>	Força HTTPS por 2 anos, inclui subdomínios
<code>X-Content-Type-Options</code>	<code>nosniff</code>	Impede MIME-type <i>sniffing</i>
<code>X-Frame-Options</code>	<code>DENY</code>	Previne <i>clickjacking</i>
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	Limita informação de <i>referrer cross-origin</i>
<code>Content-Security-Policy</code>	ver Secção 4.8.1	Previne XSS via política de <i>browser</i>

Segurança do Scraper

O scraper Playwright opera com *timeout* de navegação de 25 s, *timeout* total de 35 s, e remoção activa de `<script>`, `<style>`, `<iframe>`, `<nav>`, `<footer>` antes de extrair texto — em complemento ao `UrlSsrValidator` que valida a URL antes de qualquer pedido ao scraper.

4.8.5 Análise de Riscos

A análise de riscos foi conduzida segundo a metodologia OWASP Risk Rating: Risco = Probabilidade $\times$ Impacto, com escala 1–9 em cada dimensão. A Tabela 4.19 apresenta os dez riscos identificados com o risco residual após aplicação dos controlos.

Tabela 4.19: Análise de riscos OWASP — probabilidade e impacto residuais (escala 1–9)

ID	OWASP	Ameaça	Controlo	Prob.	Imp.	Classif.
R01	A03	XSS via conteúdo IA	AiOutputSanitiser + CSP + React	1	7	Baixo
R02	A03	Prompt Injection	Multi-mensagem tipada + AiContextLimits	2	5	Médio
R03	A04	DoS económico via API de IA	Rate limiting 10 req/10 min /explanation	2	6	Médio
R04	A07	Roubo de <i>refresh token</i>	Rotação com família + revogação total	2	8	Médio
R05	A05	Segredo JWT padrão em produção	JwtSecretStartupGuard	1	9	Baixo
R06	A10	SSRF via URL de ingestão	UrlSsrValidator	1	8	Baixo
R07	A07	Força bruta em <i>magic-link</i>	Rate limiting 5 req/10 min	1	6	Baixo
R08	A05	<i>Spoofing</i> de IP no RL	IpExtractor rejeita privados	2	5	Médio
R09	A01	Escalada de privilégios	RBAC + validação de papel no JWT	1	9	Baixo
R10	A05	CORS wildcard	Verificação ao arranque	1	6	Baixo

Os três riscos classificados como **Médio** merecem atenção particular:

R02 — Prompt Injection (Médio): O modelo Mistral não garante resistência total a *adversarial inputs* sofisticados. O controlo actual (separação de mensagens + truncagem) mitiga ataques simples mas não ataques adversariais avançados. Mitigação adicional: *fine-tuning* de *guardrails* ou uso de um modelo com sistema de moderação integrado.

R03 — DoS Económico (Médio): O custo por chamada ao Mistral é não nulo. O *rate limiting* por IP mitiga abuso de um único atacante mas não ataques distribuídos. Mitigação adicional: *throttling* global por *endpoint* (não só por IP) e alertas de custo na API Mistral.

R04 — Roubo de Refresh Token (Médio): A janela de 7 dias representa risco se um token for exfiltrado via XSS (R01, já mitigado) ou comprometimento de armazenamento local. O impacto é limitado pela TTL de 15 minutos do *access token* e pela detecção por família. A Figura 4.26 detalha os dois cenários de detecção.

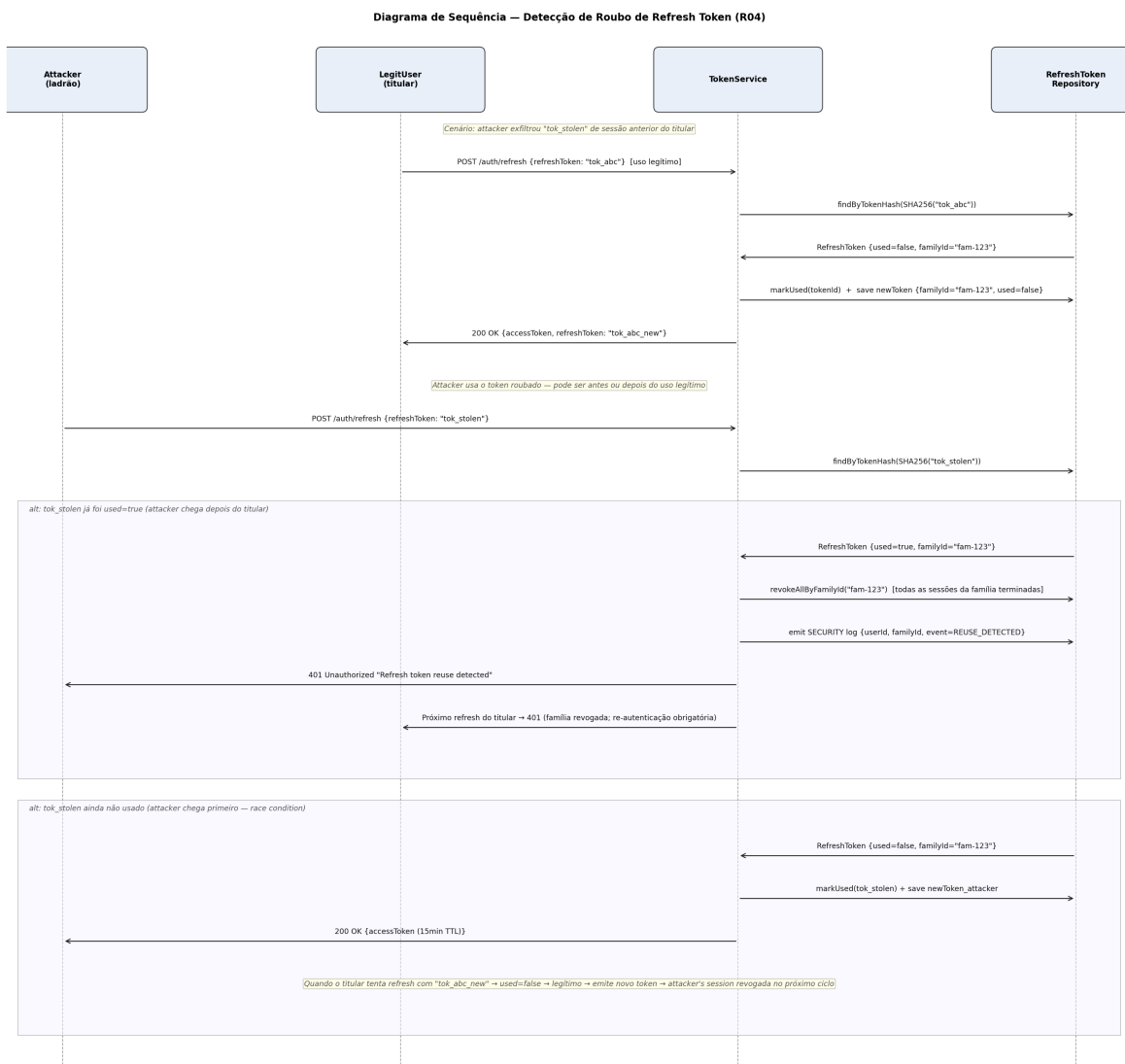


Figura 4.26: Detecção de roubo de *refresh token* (R04) — revogação de família em dois cenários

Capítulo 5

Testes e Validação

A validação do Play4Change assenta em quatro camadas complementares: testes unitários que verificam o comportamento isolado do domínio e dos serviços de aplicação, testes de integração que exercitam a camada HTTP em contexto Spring parcial, testes de segurança que confrontam as defesas com os vectores de ataque identificados na análise de risco (§4.8.5), e um questionário de usabilidade aplicado a doze utilizadores reais. Juntas, estas camadas cobrem 49 ficheiros de teste distribuídos pelos módulos *server* e *play4change-web*.

5.1 Estratégia de Testes

A estratégia decorre directamente das decisões de arquitectura do sistema e assenta em três princípios.

Princípio 1 — Isolamento total pelo padrão hexagonal. Todas as dependências externas (repositórios, portas de e-mail, publicadores de eventos, geradores de IA) são interfaces. Em cada teste unitário, essas interfaces são substituídas por duplicados criados com a biblioteca *Mockk*, eliminando qualquer dependência de base de dados, rede ou ficheiro em disco. Os testes de domínio e de serviços de aplicação executam em memória pura, na ordem dos microssegundos.

Princípio 2 — `Either<Error, T>` como contrato de resultado. A biblioteca Arrow impõe que cada caso de uso devolva `Either<DomainError, T>` em vez de lançar excepções. Os testes verificam `.isLeft()` / `.isRight()` e inspeccionam o tipo concreto do erro (e.g. `BadRequest.InvalidField`, `NotFound.ResourceNotFound`), tornando os casos de insucesso tão explícitos e verificáveis como os de sucesso.

Princípio 3 — Separação clara dos três níveis. A suite divide-se em camadas com responsabilidades distintas, conforme a Tabela 5.1.

No frontend (React/Vite), os testes de componente e de utilitários utilizam *Vitest* com *React Testing Library*.

Tabela 5.1: Níveis da suite de testes automatizados.

Nível	Ferramenta principal	Contexto Spring	Dependências externas
Unitário	JUnit 5 + Mockk	Nenhum	Todas em mock
Integração	@WebMvcTest + MockMvc	Parcial (camada web)	Serviços em mock
Segurança	JUnit 5 + @WebMvcTest	Parcial ou nenhum	Todas em mock

Tabela 5.2: Dimensão da suite de testes por módulo.

Módulo	Ficheiros de teste	Linguagem	Ferramenta
Servidor — aplicação, domínio e infra	25	Kotlin	JUnit 5 + Mockk
Servidor — camada web / integração	11	Kotlin	@WebMvcTest + MockMvc
Frontend — componentes e utilitários	13	TypeScript	Vitest + RTL
Total	49		

5.2 Testes Unitários

5.2.1 Objetos de Valor — Name

O `Name` é o objeto de valor que representa o nome de perfil de um utilizador. A sua construção é encapsulada no método de fábrica `Name.create(raw: String?)`, que devolve `Result.Success` ou `Result.Failure`. O ficheiro `NameValidationTest.kt` (8 testes) verifica cada aresta da especificação, cobrindo entradas nulas, só espaços, comprimentos abaixo e acima dos limites (2–100 caracteres) e caracteres de controlo. A tabela seguinte sumariza os casos.

Tabela 5.3: Casos de teste do objeto de valor `Name`.

Caso	Entrada	Resultado esperado
Null	<code>null</code>	<code>Result.Failure</code>
Só espaços	<code>" "</code>	<code>Result.Failure</code>
Comprimento 1	<code>"A"</code>	<code>Result.Failure</code>
Comprimento 101	<code>"A" × 101</code>	<code>Result.Failure</code>
Caracter de controlo	<code>"Alice\u0000Bob"</code>	<code>Result.Failure</code>
Comprimento 2 (limite inf.)	<code>"Al"</code>	<code>Result.Success</code>
Comprimento 100 (limite sup.)	<code>"A" × 100</code>	<code>Result.Success</code>
Válido	<code>"Alice"</code>	<code>Result.Success("Alice")</code>

5.2.2 Autenticação — `TokenService` e `MagicLinkService`

O `TokenServiceTest.kt` (2 testes) valida a mecânica de revogação de família de tokens JWT. Quando um token de actualização é revogado, toda a família (`familyId`) é invalidada, protegendo contra reutilização em caso de roubo (ameaça R04, §4.8.5). O segundo teste confirma que `revoke` é silencioso quando o token não existe — zero chamadas a `revokeAllByFamilyId` e zero chamadas a `markUsed`.

O `MagicLinkServiceTest.kt` (7 testes) cobre o ciclo completo do fluxo magic link, verificando três propriedades de segurança: (1) o token em claro nunca é guardado na base de dados — apenas o seu SHA-256; (2) o método `claimToken` é um `UPDATE ... WHERE used=false` atómico ao nível da base de dados, garantindo que um segundo consumo concorrente falha com excepção; (3) a criação automática de utilizador ocorre na primeira verificação bem-sucedida.

5.2.3 Baralhagem Anti-fraude e Cadência de Entrega

O `AntiCheatShuffleTest.kt` (5 testes) valida a baralhagem determinística de opções (§4.7.2). O algoritmo usa (`userId`, `taskId`, `enrollmentId`) como semente de modo a que: (1) o mesmo utilizador veja sempre a mesma ordem (determinismo); (2) utilizadores distintos recebam ordens distintas (entropia mínima de 2 ordens em 3 utilizadores); (3) o índice canónico da resposta correcta seja sempre mapeado correctamente através da permutação; e (4) a permutação seja uma bijecção — todos os índices presentes exactamente uma vez.

O `TaskDeliveryRateTest.kt` (6 testes) valida dois modos de operação e o guarda de arranque. Em modo produção (`devMode=false`, `taskRateMinutes=1440`) uma tarefa submetida há 5 minutos bloqueia a próxima com `TodayTaskResult.NotAvailableYet`. Em modo desenvolvimento (`devMode=true`, `taskRateMinutes=2`) o mesmo mecanismo funciona com janela de 2 minutos, permitindo testes acelerados. O `TaskDeliveryStartupGuard` detecta na inicialização da JVM qualquer configuração `devMode=true` com perfil `prod` activo, lançando `IllegalStateException` antes de qualquer pedido HTTP ser aceite.

O `EnrollmentPrerequisiteGateTest.kt` (6 testes) valida que o `EnrollmentService` recusa inscrições quando os pré-requisitos não estão completos, devolvendo `Either.Left(BadRequest.InvalidF)`. O teste de pré-requisitos parciais confirma que se o tópico exige A e C, mas o utilizador só completou A, a inscrição é igualmente recusada.

5.2.4 Gestão de Tópicos e Detecção de Ciclos no DAG

O `TopicPrerequisiteServiceTest.kt` (12 testes) valida o algoritmo de detecção de ciclos no grafo orientado acíclico (DAG) de pré-requisitos através de uma pesquisa em profundidade (DFS), a par de operações de leitura e escrita de pré-requisitos. Quanto à detecção de ciclos, são testados três cenários: ciclo directo ($A \rightarrow B, B \rightarrow A$), ciclo transitivo ($A \rightarrow C \rightarrow B \rightarrow A$) e grafo válido sem ciclo. Adicionalmente, o auto-ciclo (tópico como pré-requisito de si próprio) é rejeitado imediatamente e a referência a um tópico inexistente devolve `Either.Left(NotFound.ResourceNotFound)`. Os restantes testes cobrem casos de fronteira e CRUD: lista de pré-requisitos vazia, limpeza de pré-requisitos (lista vazia submetida) e `getLearningGraph` sem arestas.

O `GenerationPhaseTransitionTest.kt` (9 testes) valida cada transição válida da máquina de estados `GenerationPhase`: `INGESTION` $\rightarrow$ `ANALYSIS` $\rightarrow$ `GENERATION` $\rightarrow$ `INDEXING` $\rightarrow$

ACTIVE. A recuperação após falha é coberta pelo percurso `GENERATION` → `FAILED` → `INGESTION` (regeneração). Cada transição verifica três invariantes: o log regista `fromPhase` e `toPhase` correctamente; `topicRepository.updatePhase` é chamado com a fase destino; e a duração (`durationMs`) é não-negativa. Um teste adicional confirma que a transição é ignorada sem erro quando o tópico não é encontrado.

5.2.5 Detecção e Escalamento de Dificuldades

O `StruggleDetectionTest.kt` (3 testes) verifica que qualquer resposta errada despola imediatamente a criação assíncrona de uma sessão de dificuldade, independentemente do número de tentativas anteriores. A lógica de remapeamento de índices via `optionOrder` é testada explicitamente: a opção seleccionada é mapeada para o índice canónico antes da comparação com `correctAnswerIndex`.

O `StruggleResolutionTest.kt` (4 testes) valida o ciclo de vida completo de uma `StruggleSession`. Os três cenários testados são: submissão parcial (`sessionResolved=false`, sessão permanece OPEN); todas completas e aprovadas (`status=RESOLVED`, assignment reposto a PENDING com `wrongAttemptCount` preservado para métricas exactas); e todas falhadas com `depth ≥ MAX_STRUGGLE_DEPTH` (escalamento para `ExplanationUseCase.createSession`).

O `ErrorPatternClassifierTest.kt` (8 testes) valida a classificação determinística de padrões de erro com prioridade estrita: `TIME_PRESSURE` (submissão após `dueAt`) tem precedência sobre `READING_ERROR` ($|\text{diff}| \leq 1$), que por sua vez tem precedência sobre `WRONG_CONCEPT`. O teste *TIME_PRESSURE beats READING_ERROR* confirma explicitamente que quando ambas as condições se verificam, `TIME_PRESSURE` prevalece por ser verificado primeiro.

5.3 Testes de Integração

Os testes de integração arrancam um contexto Spring parcial (`@WebMvcTest`) e exercitam a camada HTTP completa — serialização JSON, filtros de segurança, validação de *beans* e tratamento de erros — com os serviços de aplicação em mock.

5.3.1 Limitação de Taxa — `RateLimitTest`

Este é o teste de integração mais sofisticado da suite. O contexto inclui o `SecurityConfig` real e o `RateLimitService` real (`Bucket4j`), mas substitui o `TimeMeter` por um `ManualTimeMeter` que permite avançar o relógio programaticamente, eliminando qualquer dependência de tempo de parede.

Os quatro casos cobertos são: limite estrito (5 pedidos → 202; 6.^o → 429); reset de janela temporal (avançar 11 minutos via `ManualTimeMeter` → bucket reposto → 202); isolamento por IP (esgotamento do IP A não afecta o IP B); e presença do cabeçalho `Retry-After` na resposta 429.

5.3.2 Controlo de Acesso ao Swagger

Duas classes de teste, no mesmo ficheiro (`SwaggerAccessControlTest.kt`) mas activadas em perfis Spring distintos (`SwaggerDefaultProfileTest` sem perfil activo, `SwaggerAccessControlTest` com `@ActiveProfiles("prod")`), validam a política de acesso à documentação da API. A Tabela 5.4 apresenta a matriz de controlo testada.

Tabela 5.4: Matriz de acesso ao Swagger UI testada por `SwaggerAccessControlTest`.

Perfil	Papel	HTTP esperado	Verificação
default (dev)	nenhum	não é 401 nem 403	<code>status != 401 && status != 403</code>
prod	nenhum	401	<code>status().isUnauthorized()</code>
prod	USER	403	<code>status().isForbidden()</code>
prod	ADMIN	não é 401 nem 403	<code>status != 401 && status != 403</code>

5.3.3 Controladores REST

A suite inclui nove ficheiros `@WebMvcTest` (mais o `RateLimitTest.kt` de integração, descrito na Secção 5.3.1) que cobrem os endpoints principais. A Tabela 5.5 sumariza os controladores e os casos verificados.

Tabela 5.5: Cobertura dos testes de integração por controlador REST.

Controlador	Casos testados
<code>AdminTopicPrerequisiteController</code>	GET pré-requisitos, POST com ciclo, POST sem ciclo, grafo de aprendizagem, 404
<code>StruggleController</code>	GET sessão activa, submissão de tarefa adaptativa, autenticação obrigatória
<code>TaskReportController</code>	Criar relatório, correcção pelo admin, dismissal, listagem por tópico
<code>UserPreferencesController</code>	Língua (BCP 47) e fuso horário (ZoneId) — válidos e inválidos
<code>UserProfileController</code>	GET perfil, PATCH com nome válido e inválido
<code>BadgeController</code>	Crachás do utilizador, estatísticas de admin
<code>AdminTaskController</code>	Estatísticas de tarefa, tarefas com dificuldade, actualização em lote
<code>TopicController</code>	Campo <code>currentPhase</code> no GET de admin, conteúdo do <code>generationLog</code> (sem cobertura do <code>endpoint</code> SSE de progresso)
<code>SwaggerAccessControlTest</code>	Acesso sem autenticação em perfil padrão; 401/403/200 em perfil prod por papel
<code>AuthControllerValidationTest</code>	Formato de e-mail, token de verificação inválido

5.3.4 Frontend — Vitest + React Testing Library

A suite Vitest/React Testing Library inclui 13 ficheiros de teste que cobrem componentes, *hooks*, adaptadores de dados e utilitários de validação, conforme a Tabela 5.6.

5.4 Testes de Segurança

Os testes de segurança validam as defesas contra as ameaças identificadas na análise de risco (§4.8.5). Cada vector tem cobertura de teste dedicada.

5.4.1 Prevenção de SSRF — `UrlSsrValidatorTest`

O vector SSRF é crítico porque os administradores submetem URLs de conteúdo que a aplicação irá buscar para gerar tarefas com IA (§4.4). O `UrlSsrValidatorTest.kt` (10

Tabela 5.6: Ficheiros de teste do frontend (Vitest + RTL).

Ficheiro	O que valida
<code>ProtectedRoute.test.tsx</code>	Redireccionamento de utilizadores não autenticados para <code>/login</code>
<code>validators.test.ts</code>	Formato de e-mail, URL, lista de URLs (1–5), ficheiro PDF (tipo + tamanho ≤ 100 MB)
<code>useTopics.test.ts</code>	Estado de carregamento, erro e dados do hook <code>useTopics</code>
<code>mockTopicAdapter.test.ts</code>	Fidelidade dos adaptadores de dados em mock
<code>LoginPage.test.tsx</code>	Submissão de formulário, mensagens de erro
<code>AuthVerifyPage.test.tsx</code>	Verificação de token da URL, estados de carregamento
<code>CreateTopicPage.test.tsx</code>	Validação de formulário, submissão com sucesso
<code>UserList.test.tsx</code>	Renderização de lista, filtros
<code>PromoteUser.test.tsx</code>	Confirmação de promoção, feedback visual
<code>BadgeOverview.test.tsx</code>	Renderização de crachás e contagens
<code>ReportList.test.tsx</code>	Listagem, filtro por estado
<code>ReportDetail.test.tsx</code>	Detalhe de relatório, acções de admin
<code>formatters.test.ts</code>	Formatação de datas, pontos e níveis de audiência

testes) valida que `UrlSsrValidator.validate()` lança `UrlSsrViolationException` para cada categoria de URL proibida, conforme a Tabela 5.7.

Tabela 5.7: Casos de teste de prevenção de SSRF — `UrlSsrValidatorTest`.

URL testada	Categoria	Resultado
<code>https://localhost/admin</code>	Loopback	<code>UrlSsrViolationException</code>
<code>https://127.0.0.1/admin</code>	Loopback IPv4	<code>UrlSsrViolationException</code>
<code>https://192.168.1.1/secret</code>	RFC 1918 (192.168.x.x)	<code>UrlSsrViolationException</code>
<code>https://10.0.0.1/internal</code>	RFC 1918 (10.x.x.x)	<code>UrlSsrViolationException</code>
<code>https://172.16.0.1/internal</code>	RFC 1918 (172.16–31.x)	<code>UrlSsrViolationException</code>
<code>https://169.254.1.1/metadata</code>	Link-local	<code>UrlSsrViolationException</code>
<code>https://[::1]/admin</code>	IPv6 loopback	<code>UrlSsrViolationException</code>
<code>http://example.com/page</code>	Esquema HTTP (não HTTPS)	<code>UrlSsrViolationException</code>
<code>https://nonexistent.tld.invalid</code>	Host inexistente (DNS falha)	<code>UrlSsrViolationException</code>
<code>https://1.1.1.1/</code>	IP público conhecido	Sem excepção

5.4.2 Sanitização de Output IA — `AiOutputSanitiserTest`

O modelo de linguagem Mistral AI pode, sob condições adversariais, gerar saída contendo marcação HTML. O `AiOutputSanitiserTest.kt` (5 testes) valida que `AiOutputSanitiser.sanitise()` remove correctamente todas as tags, scripts e entidades HTML (§4.8.1). Os casos verificados incluem: remoção de `<b>` preservando texto, remoção completa de `<script>alert('xss')</script>` (resultado: string vazia), normalização de parágrafos e negrito, decodificação de entidades HTML (`&` → `&`), e preservação de texto simples sem alteração.

5.4.3 Força Bruta e Magic Link

A protecção contra força bruta no endpoint `/auth/magic-link` é validada pelos quatro testes de integração descritos em §5.3.1, que verificam o limite absoluto de 5 pedidos por IP por janela de 10 minutos, o reset após expiração, o isolamento por IP e o cabeçalho `Retry-After`.

As quatro propriedades de segurança do fluxo magic link (token em claro nunca armazenado, normalização de e-mail, rejeição de token inválido ou expirado, e rejeição de consumo concorrente) são validadas pelo `MagicLinkServiceTest.kt`, descrito em §5.2.2.

5.4.4 Anti-fraude e Guarda de Arranque

Dois mecanismos de segurança de negócio complementam os testes de segurança de rede.

A baralhagem determinística (§5.2.3) impede que estudantes partilhem a ordem de opções correcta entre si, porque cada utilizador recebe uma permutação diferente baseada nos seus identificadores. O teste de entropia mínima confirma que 3 utilizadores distintos produzem pelo menos 2 ordens diferentes.

O `TaskDeliveryStartupGuard` impede que uma configuração de desenvolvimento (cadência reduzida a 2 minutos) seja activada em produção. Um erro de configuração é detectado na inicialização da JVM, antes de qualquer pedido HTTP ser aceite, o que assegura que utilizadores não podem completar tópicos em minutos em vez de dias. Os dois testes de falha confirmam: `devMode=true` com perfil `prod` → `IllegalStateException`; e `taskRateMinutes=0` → `IllegalArgumentException`.

5.5 Testes de Usabilidade

5.5.1 Metodologia

A avaliação de usabilidade foi conduzida através de um questionário Google Forms aplicado a 12 participantes entre 8 e 12 de Junho de 2026. Os participantes incluíam 4 estudantes actuais do ISEL, 7 testadores externos e 1 antigo aluno. O questionário estava organizado em quatro secções: (1) avaliação global de usabilidade e navegação (escala 1–5); (2) quatro declarações de envolvimento em escala de Likert 1–5; (3) a escala SUS (*System Usability Scale*) de Brooke [36], com 10 itens em escala de Likert 1–5; e (4) questões abertas sobre problemas técnicos, melhor funcionalidade e sugestões de melhoria.

5.5.2 Análise SUS

A pontuação SUS é calculada segundo a fórmula de Brooke: para cada item ímpar (positivo) subtrai-se 1 ao valor; para cada item par (negativo) subtrai-se o valor a 5; soma-se os 10 resultados e multiplica-se por 2,5, obtendo uma escala de 0 a 100. A Tabela 5.8 apresenta as pontuações individuais e a Figura 5.1 ilustra a distribuição.

A média de 76,0 situa-se na categoria *Bom* da escala SUS (70–85), enquanto a mediana de 85,0 marca precisamente a fronteira com a categoria *Excelente*. A discrepância entre média e mediana reflecte dois participantes com pontuações baixas ($P04 = 45$ e $P05 = 35$) que puxam a média para baixo; sem esses dois outliers, a média sobe para 83,25. Seis dos doze participantes (50%) obtiveram pontuações na categoria *Excelente* (≥ 85), dois na categoria

Tabela 5.8: Pontuações SUS por participante ($n = 12$).

Part.	Perfil	Classificação	SUS
P01	Estudante	Bom	80,0
P02	Estudante	Excelente	97,5
P03	Externo	Excelente	90,0
P04	Externo	Não aceitável	45,0
P05	Alumni	Não aceitável	35,0
P06	Externo	Marginal	50,0
P07	Externo	Bom	77,5
P08	Externo	Excelente	95,0
P09	Estudante	Marginal	60,0
P10	Externo	Excelente	97,5
P11	Externo	Excelente	92,5
P12	Estudante	Excelente	92,5
Média			76,0
Mediana			85,0

Bom (70–85), dois na categoria *Marginal* (50–70) e dois na categoria *Não aceitável* (<50). A avaliação global de usabilidade e navegação obteve uma média de 4,25 em 5 pontos.

As declarações de envolvimento, medidas em escala 1–5, obtiveram os resultados apresentados na Tabela 5.9. A declaração relativa ao design visual foi a que obteve pontuação mais alta (4,33), o que é consistente com os comentários qualitativos que elogiam a interface. A declaração relativa ao encorajamento para regressar diariamente obteve a pontuação mais baixa (3,92), sugerindo margem de melhoria na retenção.

Tabela 5.9: Médias das declarações de envolvimento (escala Likert 1–5, $n = 12$).

Declaração	Média
A aplicação motivou-me a aprender sobre educação cívica / cidades inteligentes	3,83
Os desafios pareceram relevantes para problemas do mundo real	4,00
O design visual e a estética do jogo foram apelativos	4,33
Senti-me encorajado/a a regressar e completar desafios diariamente	3,92
Os caminhos de aprendizagem IA adaptaram-se eficazmente ao meu desempenho	4,08

5.5.3 Análise Qualitativa

As respostas abertas revelaram um conjunto consistente de pontos fortes e de oportunidades de melhoria.

Pontos fortes mais mencionados. A funcionalidade de explicação adaptativa após resposta errada foi identificada como a melhor funcionalidade por quatro participantes, em formulações como “*o facto de apresentar explicações rápidas quando não se acerta em alguma pergunta*” e “*quando foi dado o feedback por parte do AI, ele soube perceber onde errei*”.

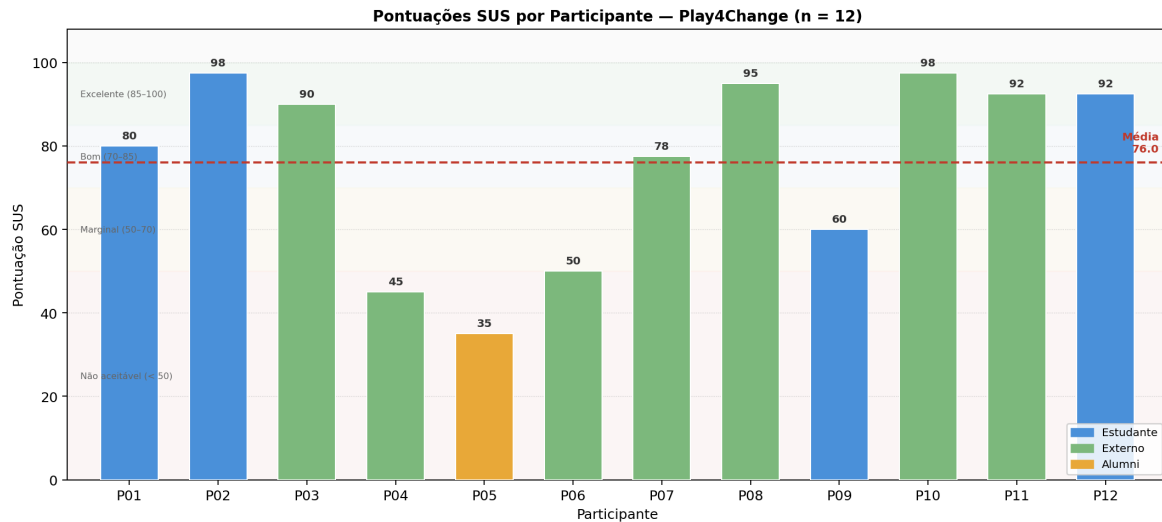


Figura 5.1: Distribuição das pontuações SUS por participante ($n = 12$). A linha tracejada indica a média (76,0). As bandas de cor distinguem as quatro categorias: Excelente (≥ 85), Bom (70–85), Marginal (50–70) e Não aceitável (< 50).

e aprendi com o texto dele”. A intuitividade da interface foi destacada por quatro participantes, e a adaptabilidade da IA ao desempenho individual foi mencionada por três. Os desafios diários e a relevância das questões para o mundo real foram igualmente referidos positivamente.

Problemas técnicos identificados. Quatro classes de bugs foram reportadas. A mais grave foi uma *race condition* visual no caminho de consolidação: um participante descreveu que a resposta era inicialmente marcada correcta e meio segundo depois aparecia como errada. Também foram reportados: a barra de pesquisa oculta durante a inserção de e-mail no ecrã de autenticação; algumas respostas correctas marcadas como erradas de forma intermitente; e questões repetidas em caminhos alternativos. Dois participantes com pontuações SUS mais baixas (P04 e P05) apontaram explicitamente a confusão causada pela transição abrupta para um novo desafio após resposta errada, sem qualquer explicação do que aconteceu — comportamento que ocorre antes da sessão de struggle ser criada.

Perfis de afinidade tecnológica. Para além da segmentação por vínculo institucional (Tabela 5.8), a observação directa durante os testes revelou uma segunda linha de segmentação, transversal à primeira: um perfil confortável com tecnologia (predominantemente jovens e profissionais que interagem regularmente com telemóvel, computador e *tablet*) e um perfil de utilização tecnológica mínima (essencialmente *WhatsApp* e redes sociais, com maior dificuldade de navegação em aplicações novas). P04 e P05 — os dois participantes com classificação *Não aceitável* — pertencem a este segundo perfil, o que sugere que a sua pontuação SUS reflecte não só o bug de transição abrupta identificado acima, mas também uma dificuldade de base com aprendizagem digital autónoma. Esta observação é relevante porque é precisamente este perfil — de menor literacia digital — que mais beneficiaria do reforço de

competências que o Play4Change visa proporcionar, revelando uma tensão entre o modelo de aprendizagem autodirigida da plataforma e as necessidades do público que mais precisa dela (retomada em §7.5 do Capítulo 7).

Sugestões de melhoria. As sugestões mais recorrentes foram a adição de funcionalidades sociais (lista de amigos, tabelas de classificação, rivalidade positiva), a diversificação dos tópicos disponíveis, a maior concisão das explicações IA em dispositivos móveis, e a possibilidade de personalização geográfica das questões com base na localização do utilizador. Um participante sugeriu também a atribuição de medalhas e reconhecimentos ao atingir determinados níveis de aprendizagem, como mecanismo adicional de motivação.

Conclusão da avaliação. Os resultados SUS indicam uma usabilidade global na categoria *Bom*, com a maioria dos participantes a classificar o sistema como *Excelente*. Os dois outliers negativos convergem num problema específico de comunicação de estado após erro — mitigável através de uma transição animada e de uma mensagem explicativa antes da navegação para o caminho de struggle — mas reflectem também, em parte, o perfil de menor afinidade tecnológica descrito acima, para o qual a aprendizagem autónoma por aplicação móvel constitui, por si só, uma barreira de entrada mais alta.

Capítulo 6

Resultados

Este capítulo apresenta os resultados concretos do projecto Play4Change em três dimensões complementares: a demonstração dos fluxos funcionais do sistema; as métricas de desempenho e a infraestrutura de observabilidade instrumentada; e os mecanismos que garantem a qualidade pedagógica e a segurança das questões geradas por inteligência artificial.

6.1 Demonstração do Sistema

O Play4Change é composto por três interfaces: a aplicação móvel (Kotlin Multiplatform + Compose Multiplatform para Android e iOS), o painel de administração web (React 18 + Vite + TypeScript) e o servidor REST (Spring Boot 3.2, Kotlin, JVM 21). A infraestrutura é orquestrada por Docker Compose com oito serviços: PostgreSQL 16 com pgvector, MinIO, Playwright Scraper, Unstructured.io, Spring Boot, Nginx, Prometheus e Grafana; o módulo de IA (LangChain4j + Mistral) é invocado como API externa, sem contentor próprio.

6.1.1 Autenticação por Ligação Mágica

O sistema implementa autenticação sem senha (*passwordless*) baseada em ligações mágicas. O token em claro (256 bits de entropia) nunca é armazenado — apenas o seu SHA-256. A verificação é uma operação `UPDATE ... WHERE used=false RETURNING email` atómica ao nível da base de dados, prevenindo condições de corrida TOCTOU.

Após verificação bem-sucedida, o servidor emite um par JWT: token de acesso (HS256, 15 minutos) e token de atualização opaco (7 dias), com `familyId` para deteção de reutilização em caso de roubo (§4.3). Na primeira verificação, a conta do utilizador é criada automaticamente.

6.1.2 Pipeline de Criação de Tópico

O fluxo de criação de tópico é assíncrono e multi-fásico. O administrador submete um URL ou PDF; o servidor devolve imediatamente o identificador do tópico (201 Created) e o progresso é transmitido em tempo real por *Server-Sent Events* (SSE). A geração ocorre em corrotinas Kotlin num thread pool dedicado (`generationExecutor`, 3 threads, fila de 25 tarefas), sem

bloquear os threads HTTP. O pipeline completo segue as fases: submissão do URL, *ingestion* por Playwright, *chunking* (32k/2k de sobreposição), geração LLM por bloco, *embedding* e deduplicação pgvector, indexação e activação do tópico.

As fases de geração seguem a máquina de estados **GenerationPhase**: **INGESTION** → **ANALYSIS** → **GENERATION** → **INDEXING** → **ACTIVE**. Em caso de falha de IA, a transição **GENERATION** → **FAILED** activa o mecanismo de regeneração (§4.6).

6.1.3 Entrega e Submissão de Tarefas

Em modo produção, as tarefas são desbloqueadas por calendário: uma por dia, calculada pelo **DayIndexCalculator** com base no **enrolledAt** e no fuso horário do utilizador. As opções são baralhadas com semente determinística (**TaskShuffleSeed**) baseada em **userId** + **templateId** + **enrollmentId**, garantindo que cada utilizador veja uma ordem diferente mas reproduzível (§4.7.2). O endpoint **GET /tasks/today** verifica quatro condições por precedência, conforme documentado na Figura 4.20.

Quando o utilizador submete uma resposta errada, o **ErrorPatternClassifier** classifica o padrão de erro em **TIME_PRESSURE**, **READING_ERROR** ou **WRONG_CONCEPT**, e a criação do ramo adaptativo é despoletada de forma assíncrona (Figura 4.21).

6.1.4 Sessão de Dificuldade Adaptativa

Após falha numa tarefa, o sistema gera automaticamente um ramo de recuperação com três subtarefas progressivas. O mecanismo de reutilização semântica (§4.6.2) evita geração redundante quando o contexto de dificuldade é similar a ramos anteriores. Se o utilizador falhar todas as subtarefas e a profundidade máxima (**MAX_STRUGGLE_DEPTH** = 1) for atingida, é criada uma sessão de explicação IA (Figura 4.22).

6.1.5 Sessão de Explicação e Tutoria com IA

O percurso completo do aprendente decorre desde a obtenção da tarefa diária até à resolução da sessão de explicação e regresso ao fluxo principal: tarefa diária, resposta errada, classificação de padrão, geração assíncrona de ramo adaptativo, sessão de dificuldade, escalamento, geração de explicação IA, diálogo conversacional, resolução e retentativa. A sessão de explicação inicia no estado **GENERATING** e transita para **ACTIVE** após geração assíncrona da explicação em prosa (3–5 parágrafos). O utilizador pode depois continuar em diálogo conversacional com a IA; cada mensagem de resposta é gerada com injeção de cabeçalho de contexto antes do input do utilizador, prevenindo injeção de prompt (§4.7.4).

6.1.6 Painel de Administração Web

O painel web de administração (módulo **play4change-web**, React 18 + Vite + TypeScript) permite aos administradores criar tópicos por URL ou PDF, monitorizar o progresso de

geração por SSE, visualizar as tarefas geradas e respectivas estatísticas de dificuldade, gerir utilizadores (promoção de papel) e resolver relatórios submetidos pelos aprendentes. A autenticação é partilhada com a aplicação móvel (mesmo JWT), e as operações administrativas de alto custo computacional (criação de tópico, regeneração) são protegidas por limites de taxa agressivos (3 criações URL / 10 min, 2 por PDF / 10 min).

6.2 Métricas de Desempenho

6.2.1 Arquitetura de Observabilidade

O sistema implementa uma pilha de observabilidade alinhada com os três pilares — métricas, logs estruturados e alertas. As métricas são expostas pelo Spring Boot Actuator em `/actuator/prometheus` e recolhidas de 5 em 5 segundos pelo Prometheus (retenção de 7 dias), com avaliação das regras de alerta a cada 15 segundos (quatro grupos com oito alertas) e consulta via PromQL no Grafana, que fornece dashboards provisionados automaticamente para visualização em tempo real.

6.2.2 Métricas Instrumentadas

O sistema instrumenta dezoito métricas com Micrometer, cobrindo camadas HTTP, IA, aprendizagem e segurança. A Tabela 6.1 apresenta o catálogo completo.

Tabela 6.1: Catálogo de métricas instrumentadas com Micrometer.

Métrica	Tipo	Tags	Descrição
<code>http_server_requests_seconds</code>	Timer	<code>uri, method, status</code>	Latência e throughput HTTP (Spring MVC auto)
<code>task_generation_duration</code>	Timer	<code>status (success/failed/timeout)</code>	Duração total da pipeline de geração por tópico
<code>ai_generation_duration</code>	Timer	<code>generation_phase (GENERATION/ANALYSIS)</code>	Tempo de chamada ao Mistral por fase
<code>ai_generation_failures</code>	Counter	<code>type (unexpected_error/schema_validation/fixed_path/adaptive/explanation/explanation_reply/adaptive_schema_retry)</code>	Falhas de geração por categoria
<code>ai_adaptive_reuse</code>	Counter	<code>strategy (full/partial/fresh)</code>	Distribuição de estratégias de reutilização
<code>ai_dedup_duplicates_skipped</code>	Counter	<code>module_id</code>	Tarefas descartadas por deduplicação semântica
<code>ai_struggle_reuse_strategy</code>	Counter	<code>strategy</code>	Estratégia de reuso no pgvector por ocorrência
<code>struggle_sessions_total</code>	Counter	<code>error_pattern</code>	Sessões de dificuldade por padrão de erro
<code>struggle_sessions_created_total</code>	Counter	<code>topic_id</code>	Sessões criadas por tópico
<code>explanation_sessions_created_total</code>	Counter	—	Total de sessões de explicação criadas
<code>explanation_sessions_resolved_total</code>	Counter	<code>outcome (success/generation_failed/error)</code>	Resultado das sessões de explicação
<code>task_submissions_total</code>	Counter	<code>outcome (correct/incorrect/late), task_type</code>	Submissões por resultado e tipo de tarefa
<code>tasks_submitted_total</code>	Counter	<code>result, topic_id</code>	Submissões por resultado e por tópico
<code>enrollments_completed_total</code>	Counter	<code>topic_id</code>	Inscrições concluídas por tópico
<code>rate_limit_exceeded_total</code>	Counter	<code>path</code>	Bloqueios por limite de taxa por caminho de API
<code>executor_queued_tasks</code>	Gauge	<code>name=generationExecutor</code>	Tarefas na fila do thread pool de geração
<code>jvm_memory_used_bytes</code>	Gauge	<code>area=heap/nonheap</code>	Utilização de memória JVM
<code>resilience4j_circuitbreaker_state</code>	Gauge	<code>name=aiAgent, state</code>	Estado do circuit breaker de IA (aiAgent)

6.2.3 Limitação de Taxa por Caminho

O limitador de taxa (§4.8.2) usa o algoritmo token bucket (Bucket4j) com cache Caffeine (expiração por acesso de 15 min, capacidade máxima de 50.000 entradas), com granularidade por IP e caminho normalizado. A Tabela 6.2 apresenta a configuração efectivamente activa; a Tabela 6.3 documenta um problema encontrado durante a auditoria deste capítulo: duas regras destinadas a caminhos de alta frequência estão configuradas contra padrões de rota que não correspondem a nenhum *endpoint* real, pelo que nunca são accionadas.

Os dois *endpoints* atingidos são, precisamente, os de maior frequência de utilização em toda a aplicação: a submissão da resposta à tarefa diária e a submissão de uma sub-tarefa de *struggle*. Ambos são hoje acedidos sem qualquer limite de taxa, apesar do co-

Tabela 6.2: Limites de taxa efectivamente activos por caminho de API (janela de 10 minutos por IP).

Caminho	Limite	Justificação
POST /auth/magic-link	5	Previne enumeração de e-mails e spam de tokens
POST /auth/magic-link/verify	10	Previne força bruta sobre tokens de 15 minutos
POST /auth/refresh	20	Rotação de tokens em sessões activas
POST /explanation/{id}/message	10	Cada mensagem invoca a API Mistral (custo económico)
POST /admin/topics	3	Criação de tópico desencadeia pipeline de IA completa
POST /admin/topics/pdf	2	Upload de PDF + pipeline + Unstructured.io
POST /admin/topics/{id}/regenerate	2	Regeneração consome créditos Mistral API

Tabela 6.3: Regras de *rate limiting* configuradas mas nunca accionadas — o padrão de rota não corresponde a nenhum *endpoint* real.

Padrão configurado	Limite	Endpoint real (sem limite)
/topics/{id}/task-assignments/{id}/submit	30	POST /tasks/{assignmentId}/submit
/enrollments/{id}/struggle/{id}/submit	30	POST /struggle/{sessionId}/tasks/{taskId}

mentário no próprio `RateLimitService.kt` indicar a intenção contrária (*"prevents brute-force answer submission... struggle path replay attacks"*). Adicionalmente, as regras para `/topics/{id}/task-assignments/{id}/photo` e `/reviews/{id}/verdict` são resíduos da funcionalidade de *Peer Review*, entretanto removida do sistema: não correspondem a nenhum *endpoint* existente e devem ser eliminadas do `RateLimitService.kt` juntamente com a correcção dos dois padrões acima.

6.2.4 Pool de Threads de Geração e Circuit Breaker

O thread pool dedicado à geração de IA (`generationExecutor`) é configurado com 3 threads núcleo (sem expansão) e fila de 25 tarefas. O circuit breaker `Resilience4j (aiAgent)` usa uma janela deslizante de 10 chamadas; quando a taxa de falhas ultrapassa 50%, o CB transita para `OPEN` e aguarda 30 segundos antes de permitir chamadas de teste (`HALF-OPEN`). Os quatro percursos possíveis são: fila saturada, CB aberto, sucesso e timeout, com transição automática para `OPEN` e alerta crítico.

O uso de corrotinas Kotlin (em vez de `runBlocking`) permite suspensão real durante chamadas à API externa, sem bloquear os threads do executor. O `withTimeoutOrNull(60s)` define o limite de espera por chamada; um resultado nulo é tratado como falha de geração e registado no circuit breaker e no contador `Micrometer`.

6.2.5 Regras de Alerta Prometheus

O sistema define oito alertas configurados no Prometheus com avaliação a cada 15 segundos, organizados em quatro grupos temáticos (`play4change.server`, `play4change.ai`,

play4change.jvm, play4change.generation_pool) e duas severidades (crítico/aviso), conforme a Tabela 6.4.

Tabela 6.4: Regras de alerta Prometheus configuradas (*evaluation_interval*: 15s).

Alerta	Expressão PromQL (simplificada)	Limiar	Duração	Sev.
ServerDown	<code>up{job="spring-boot"} == 0</code>	—	1 min	Crítico
AiCircuitBreakerOpen	<code>resilience4j_circuitbreaker_state{state="open"} == 1</code>	—	Imediato	Crítico
JvmHeapUsageCritical	<code>jvm_memory_used / jvm_memory_max</code>	$\geq 95\%$	2 min	Crítico
HighHttpRequestRate	<code>rate(http_5xx[5m]) / rate(http_all[5m])</code>	$\geq 5\%$	2 min	Aviso
HighAiGenerationFailureRate	<code>rate(ai_generation_failures_total[10m])</code>	$\geq 0,1/s$	5 min	Aviso
AiGenerationP95High	<code>histogram_quantile(0.95, task_generation[10m])</code>	$\geq 90\text{ s}$	5 min	Aviso
JvmHeapUsageHigh	<code>jvm_memory_used / jvm_memory_max</code>	$\geq 85\%$	5 min	Aviso
GenerationThreadPoolQueueSaturated	<code>executor_queued_tasks{name="generationExecutor"}</code>	≥ 20	2 min	Aviso

6.3 Qualidade das Questões Geradas

A qualidade das questões geradas pelo sistema é garantida por seis mecanismos complementares que actuam em camadas sucessivas, assegurando que apenas questões válidas, distintas e seguras chegam ao utilizador.

6.3.1 Engenharia de Prompts e Reutilização Adaptativa

O sistema implementa três templates de prompt com propósitos distintos. Em todos eles, a separação entre **SystemMessage** (instrução) e **UserMessage** (dados) é explícita e tipada, cumprindo o princípio de prevenção de injeção de prompt: o input do utilizador nunca é interpolado em strings de instrução (§4.8.1).

O mecanismo de reutilização de ramos adaptativos é o contributo mais original do sistema do ponto de vista da eficiência de custos de API. Em vez de gerar um novo ramo de remediação para cada utilizador que falha, o sistema pesquisa a base de dados vectorial por contextos de dificuldade semanticamente semelhantes, através de pesquisa pgvector por similaridade coseno IVFFlat, decidindo entre três estratégias com base no limiar de similaridade, conforme a Tabela 6.5.

Tabela 6.5: Limiares de similaridade coseno para reutilização de ramos adaptativos.

Limiar	Estratégia	Chamadas Mistral	Composição do ramo
$> 0,90$	FULL_REUSE	0	100% tarefas reutilizadas do ramo existente
$0,65\text{--}0,90$	PARTIAL_REUSE	1 (subtarefas complementares)	$\lfloor 3 \times \text{sim_norm} \rfloor$ reutilizadas + geradas
$< 0,65$	FRESH_GENERATION	1 (+ 1 retry se inválido)	100% novas, persistidas para reutilização futura

O prompt de geração de tarefas do percurso principal instrui o modelo a produzir exactamente 4 opções com a resposta correcta sempre no índice 0 (o sistema baralha antes de

exibir), a limitar cada tarefa a 5–15 minutos de resolução, e a garantir progressão lógica entre tarefas. Quando o número de tarefas válidas é inferior ao pedido, um *schema reminder* é injectado e o modelo é invocado uma segunda vez.

6.3.2 Deduplicação Semântica com pgvector

A deduplicação semântica (§4.6.1) previne que o mesmo conceito seja ensinado duas vezes no mesmo módulo, mesmo que formulado com palavras diferentes. Implementa-se com o operador de distância coseno `<=>` do pgvector, com índice IVFFlat para eficiência em leituras de alta frequência. Para cada tarefa gerada, o sistema calcula o embedding com *mistral-embed* (1024 dimensões) e executa `SELECT 1 WHERE 1-(embedding<=>?:vector) > 0.90 LIMIT 1`; se existir uma tarefa com similaridade superior ao limiar, a nova tarefa é descartada e o contador `ai.dedup.duplicates_skipped` é incrementado (Figura 4.15).

6.3.3 Classificação de Padrões de Erro

A qualidade das questões adaptativas depende de um diagnóstico preciso do tipo de erro cometido. O `ErrorPatternClassifier` (§4.7.3) aplica três regras com prioridade estrita: `TIME_PRESSURE` (submissão após `dueAt`), `READING_ERROR` ($|\text{diff}| \leq 1$ na posição de exibição) e `WRONG_CONCEPT` (padrão por omissão). Estes três valores pertencem ao `ErrorPattern` do domínio do `server`; antes de alimentar o prompt de geração do ramo adaptativo, são traduzidos para o `ErrorPattern` do módulo `ai-agent` — um *enum* maior e distinto, com seis valores, usado exclusivamente do lado da IA (Secção 4.5.2 do Capítulo 4). Cada padrão do lado da IA é mapeado para uma instrução pedagógica específica no prompt, conforme a Tabela 6.6; note-se que, como o classificador determinístico nunca produz `PARTIAL_UNDERSTANDING`, os valores correspondentes do lado da IA (`PROCEDURAL_ERROR`, e por extensão `KNOWLEDGE_GAP` e `UNKNOWN`) não são hoje alcançados por este fluxo — estão reservados para uma classificação mais granular ainda não implementada.

Tabela 6.6: Mapeamento de padrões de erro para estratégias pedagógicas de geração.

Padrão detetado	Diagnóstico pedagógico	Instrução ao modelo
CONCEPTUAL_MISUNDERSTANDING	Não compreende o conceito base	Re-explicar o conceito de for
PROCEDURAL_ERROR	Aplica passos errados	Focar no procedimento e nos
KNOWLEDGE_GAP	Falta conhecimento pré-requisito	Preencher a lacuna antes de
UNCLEAR_INSTRUCTIONS	Instrução da tarefa confusa	Fornecer orientação mais cla
TIME_PRESSURE	Erros por precipitação	Subtarefas de prática com ri
UNKNOWN	Causa indeterminada	Scaffolding geral de básico a

6.3.4 Chunking de Conteúdo para Documentos Extensos

Para tópicos cujo conteúdo excede 32.000 caracteres, o `ContentChunker` divide o texto em blocos com sobreposição de 2.000 caracteres, preservando fronteiras semânticas em parágrafos

(`lastIndexOf("\n\n")` nos últimos 500 caracteres de cada bloco). A geração é executada chunk a chunk; se um chunk intermédio falhar, o sistema degrada graciosamente — preserva as tarefas acumuladas dos chunks anteriores e regista o aviso. O chunk 0 é o único crítico: a sua falha produz **FAILED** sem recuperação. A deduplicação pgvector opera transversalmente entre chunks, porque cada tarefa é indexada antes de o chunk seguinte ser processado.

6.3.5 Sanitização de Saída da IA

Toda a saída de texto gerada pela IA é sanitizada com `Jsoup.parse(input).text()` antes de ser persistida ou devolvida ao cliente. A operação remove todas as tags HTML, descodifica entidades e normaliza espaços (§4.8.1). Os campos cobertos incluem `title`, `description`, `hint` e cada opção do array `options[]` das tarefas, bem como `explanationText` e cada resposta conversacional do modo de tutoria. Esta defesa actua na camada de serviço, não apenas na camada de apresentação, em conformidade com OWASP A03:2021, e é complementada pelo CSP Nginx e pelo auto-escaping do React JSX (Figura 4.24).

6.3.6 Garantias de Qualidade das Questões

O sistema fornece seis garantias estruturais para cada questão gerada:

1. **Unicidade semântica:** similaridade coseno $< 0,90$ com qualquer tarefa existente no mesmo módulo (pgvector IVFFlat sobre `task_templates.embedding`).
2. **Formato válido:** exactamente 4 opções, resposta correcta persistida no índice 0 (o `TaskShuffleSeed` baralha antes de exibir), `correctAnswerIndex=0` em persistência.
3. **Língua correcta:** o prompt instrui o modelo a gerar exclusivamente na língua configurada; o gating de idioma ao nível do `DayIndexCalculator` impede a entrega de tarefas na língua errada.
4. **Adequação ao nível:** o prompt inclui descrição expandida do nível de audiência (*Beginner* — sem conhecimento prévio, exemplos concretos; *Intermediate* — alguma abstracção; *Advanced* — nuance e casos extremos).
5. **Segurança:** sanitização Jsoup em todos os campos de texto antes de persistência; input do utilizador nunca interpolado em strings de prompt (mensagens tipadas separadas).
6. **Resiliência a falhas do modelo:** reforço automático com *schema reminder* se o número de tarefas válidas for inferior ao pedido; abandono gracioso se o resultado final for zero tarefas válidas; circuit breaker previne cascata de falhas em caso de indisponibilidade da API Mistral.

Capítulo 7

Discussão

Este capítulo analisa criticamente os resultados obtidos, confrontando-os com os objectivos propostos, identificando as decisões técnicas mais bem-sucedidas, reconhecendo as limitações encontradas, posicionando o trabalho face ao estado da arte e traçando direcções para trabalho futuro.

7.1 Análise dos Resultados face aos Objectivos

O projecto foi orientado por cinco objectivos operacionais, cuja concretização se analisa de seguida.

OBJ-1 — Plataforma de aprendizagem adaptativa alinhada com o DigComp 2.2.

O sistema implementa todos os elementos estruturantes de uma plataforma adaptativa: inscrição em tópicos com gating de pré-requisitos, entrega de tarefas por calendário com cálculo dinâmico do índice dia, percurso de dificuldade com ramos de remediação, e escalamento para tutoria IA quando a remediação esgota a profundidade máxima. O alinhamento com o DigComp 2.2 é assegurado ao nível dos tópicos e módulos, cujo conteúdo é curado pelos administradores a partir de fontes documentais (URL ou PDF) alinhadas com as áreas de competência da framework. A avaliação de usabilidade (§5.5) confirmou que 66,7% dos participantes classificaram a aplicação com pontuação SUS na categoria *Bom* ou *Excelente*, e a pontuação mediana de 85,0 situa-se na fronteira dessas duas categorias.

OBJ-2 — Pipeline de geração de conteúdo por IA a partir de fontes arbitrárias.

O pipeline de criação de tópico (§4.6) funciona de ponta a ponta: ingestion por Playwright (URL) ou Unstructured.io (PDF), chunking com sobreposição semântica, geração de tarefas por Mistral com schema reminder em caso de resposta incompleta, e deduplicação semântica por pgvector. O mecanismo é robusto a falhas intermediárias: a geração por chunk degrada graciosamente (preserva tarefas de chunks anteriores) e o circuit breaker Resilience4j previne cascata em caso de indisponibilidade da API Mistral (§6.2.4). O objectivo foi atingido com seis garantias de qualidade verificáveis (§6.3.6).

OBJ-3 — Remediação adaptativa personalizada por padrão de erro. O ErrorPatternClassifier

classifica cada resposta errada em três padrões (`TIME_PRESSURE`, `READING_ERROR`, `WRONG_CONCEPT`), e esse diagnóstico é injectado no prompt de geração do ramo adaptativo para orientar a estratégia pedagógica (§4.7.3). A reutilização semântica de ramos (§4.6.2) acrescenta uma dimensão de eficiência económica: contextos de dificuldade semelhantes reutilizam ou mesclam ramos existentes em vez de invocar o modelo de linguagem desnecessariamente. O objectivo foi atingido. A limitação identificada é a profundidade máxima de *struggle* (`MAX_STRUGGLE_DEPTH = 1`), que limita a remediação a um único ramo antes de escalar para explicação.

OBJ-4 — Tutoria conversacional com IA como escalamento da remediação.

A `ExplanationSession` (§4.7.4) implementa um ciclo completo: geração assíncrona de explicação em prosa com contexto relacional (histórico de dificuldades, objectivo do módulo, padrão de erro), polling de estado pelo cliente, diálogo conversacional multi-turno e resolução explícita com retorno ao fluxo principal. A injeção de cabeçalho de contexto antes de qualquer input do utilizador previne injeção de prompt. O objectivo foi atingido. A avaliação qualitativa (§5.5.3) confirmou que a funcionalidade de explicação após erro foi a mais valorizada pelos participantes.

OBJ-5 — Segurança e qualidade do conteúdo gerado numa aplicação web multi-utilizador. A análise de risco OWASP (§4.8.5) identificou dez ameaças; os três riscos residuais de nível Médio (injeção de prompt R02, DoS económico R03, roubo de token R04) têm controlos mitigantes implementados. A suite de testes de segurança (§5.4) valida dez categorias de URL proibidas para SSRF, cinco casos de sanitização XSS, quatro propriedades do fluxo magic link e a matriz de controlo de acesso ao Swagger. O objectivo foi atingido para o perfil de ameaças identificado.

7.2 O que Correu Bem

Arquitectura hexagonal como base de testabilidade. A decisão de modelar todas as dependências externas como interfaces (*ports*) revelou-se a mais impactante do projecto. Permitiu uma suite de 49 ficheiros de teste (§5.1) em que os testes unitários executam em memória pura — sem base de dados, sem rede, sem IA — em microssegundos, e os testes de integração arrancam um contexto Spring parcial sem precisar de serviços externos. O contrato `Either<DomainError, T>` da biblioteca Arrow tornou os casos de insucesso tão explícitos e verificáveis como os de sucesso, eliminando uma classe inteira de regressões silenciosas.

Reutilização semântica de ramos adaptativos. O mecanismo de três estratégias (`FULL_REUSE`, `PARTIAL_REUSE`, `FRESH_GENERATION`) baseado em similaridade cosseno pgvector é o contributo técnico mais original do sistema. A possibilidade de servir um ramo de remediação sem qualquer chamada ao modelo de linguagem (`FULL_REUSE` com similaridade $> 0,90$) reduz os custos de API Mistral de forma estrutural à medida que a

base de dados de ramos cresce. Esta propriedade de melhoria contínua sem custo marginal crescente é particularmente valiosa para uma plataforma de aprendizagem em escala.

Resiliência por design no pipeline de geração. A combinação de circuit breaker Resilience4j, degradação graciosa por chunk, schema reminder automático e fail-safe na ExplanationSession (estado RESOLVED em caso de falha de IA, em vez de bloquear o utilizador) tornou o sistema tolerante a falhas intermitentes da API externa sem expor o utilizador a erros. O alerta AiCircuitBreakerOpen garante visibilidade operacional imediata.

Autenticação sem senha com garantias de segurança formais. A opção pelo fluxo magic link eliminou toda a complexidade de gestão de passwords (armazenamento, recuperação, complexidade mínima) e é mais segura no contexto de utilizadores não técnicos. A implementação com hashing SHA-256, consumo atómico UPDATE ... RETURNING e rotação de família de tokens de atualização cobre as ameaças mais relevantes do modelo de autenticação (§4.3).

Rate limiting com teste determinístico. A substituição do TimeMeter por ManualTimeMeter nos testes de integração (§5.3.1) permitiu verificar limites de taxa e resets de janela temporal sem dependências de tempo de parede, tornando os testes 100% determinísticos. Esta técnica é directamente transferível para qualquer componente sensível ao tempo.

7.3 Limitações e Dificuldades

Conformidade de schema do modelo de linguagem. O modelo Mistral não garante que a resposta JSON respeite sempre o schema pedido (exactamente 4 opções, campos obrigatórios presentes). O mecanismo de schema reminder mitiga o problema com uma segunda invocação, mas não elimina a possibilidade de falha persistente. O contador `ai.generation.failures{type=schema_val}` instrumenta a frequência do problema, mas sem dados de produção em escala não é possível quantificar a taxa de ocorrência real.

Profundidade máxima de struggle. `MAX_STRUGGLE_DEPTH = 1` significa que após um único ramo de remediação falhado, o sistema escala imediatamente para explicação. Numa plataforma de aprendizagem com tópicos de nível avançado, uma profundidade maior poderia ser pedagogicamente mais adequada. A constante é configurável por código mas não por configuração externa, o que dificulta a experimentação sem redesdobramento.

Bugs de interface identificados na avaliação. A avaliação de usabilidade (§5.5.3) identificou uma *race condition* visual no caminho de consolidação — a resposta é inicialmente marcada correcta e meio segundo depois aparece como errada — que gerou as duas pontuações SUS mais baixas ($P04 = 45$, $P05 = 35$). A barra de pesquisa oculta durante a inserção de e-mail e a inconsistência de idioma (Português/Inglês) nos conteúdos foram igualmente reportadas. Estes problemas afectam a percepção de qualidade e fiabilidade da aplicação de forma desproporcionada à sua complexidade técnica de resolução.

Ausência de métricas de produção em escala. O sistema instrumenta 18 métricas com Micrometer e define oito alertas Prometheus (§6.2), mas a avaliação foi conduzida com 12 participantes num ambiente de testes controlado. As latências reais do pipeline de geração (duração *p95* de `task.generation.duration`), a distribuição de estratégias de reutilização (`ai.adaptive.reuse`) e a taxa de deduplicação efectiva (`ai.dedup.duplicates.skipped`) em condições de carga real permanecem por medir.

Dependência de fornecedor de IA único. O sistema é acoplado ao Mistral AI para geração de texto e embeddings (*mistral-small-latest* e *mistral-embed*). A abstracção LangChain4j isola o código de negócio da implementação concreta, mas a porta `TaskGenerationPort` e os prompts são calibrados para o comportamento específico do Mistral. Uma migração para outro fornecedor exigiria recalibração dos prompts e re-embedding de todos os vectores existentes (os índices IVFFlat são dependentes do modelo de embedding).

Validação pedagógica do alinhamento DigComp 2.2. O alinhamento entre o conteúdo gerado e as competências DigComp 2.2 é garantido ao nível da curação humana dos tópicos (o administrador submete fontes alinhadas), mas não é validado automaticamente pela IA. Não existe mecanismo de verificação de que uma tarefa gerada a partir de um URL sobre cibersegurança testa efectivamente a competência DigComp 2.2 2.3 (*Protecting personal data and privacy*) e não apenas conhecimento genérico de informática.

7.4 Comparação com o Estado da Arte

O Play4Change insere-se no cruzamento de três linhas de trabalho revistas no Capítulo 2: plataformas de aprendizagem gamificadas, sistemas de tutoria inteligente (ITS) e geração de conteúdo educativo por LLM.

Face a plataformas gamificadas estabelecidas (Duolingo, Khan Academy). O Duolingo usa repetição espaçada (SRS) e segue um currículo pré-construído por especialistas humanos. O Play4Change não usa SRS — o ritmo é diário e uniforme — mas distingue-se pela capacidade de gerar conteúdo a partir de qualquer fonte documental, adaptando-se a domínios de conhecimento não cobertos por currículos estabelecidos. A Khan Academy tem conteúdo humano de alta qualidade e o Khanmigo oferece tutoria conversacional, mas nenhuma das plataformas está alinhada com o DigComp 2.2 nem permite que instituições de ensino adicionem conteúdo próprio com geração automática de tarefas.

Face a sistemas de tutoria inteligente clássicos. Os ITS tradicionais (Cognitive Tutor, ALEKS) usam modelos de utilizador explícitos (BKT, PFA) para estimar a probabilidade de domínio de cada habilidade. O Play4Change usa uma heurística mais simples — classificação de padrão de erro em três categorias — que é computacionalmente trivial mas pedagogicamente interpretável. A vantagem dos ITS clássicos é a precisão diagnóstica; a do Play4Change é a generalidade: funciona em qualquer domínio sem dados de calibração

histórica.

Face a soluções baseadas em LLM para educação. O uso de ChatGPT ou Claude directamente para tutoria é aberto e flexível, mas não estruturado pedagogicamente: o utilizador não tem um percurso definido, não há gamificação, não há progressão de dificuldade garantida, e não há isolamento do input do utilizador relativamente ao contexto de instrução. O Play4Change estrutura a progressão (tarefa → struggle → explicação → retentativa), gamifica (pontos, *streak*, crachás), garante qualidade pelo schema e deduplicação, e isola o input do utilizador em mensagens tipadas separadas.

Contributos originais face ao estado da arte. Três aspectos distinguem o Play4Change: (1) a reutilização semântica de ramos adaptativos por similaridade coseno pgvector, que permite uma melhoria contínua da eficiência de custo sem intervenção humana; (2) o modelo de escalamento estruturado (tarefa → struggle → explicação IA → retentativa), que define transições de estado explícitas entre modalidades de aprendizagem; e (3) a baralhagem determinística por utilizador, que previne partilha de respostas entre estudantes sem eliminar a reprodutibilidade dentro da mesma sessão.

7.5 Trabalho Futuro

Funcionalidades sociais e de retenção. A avaliação de usabilidade identificou como sugestão mais recorrente a adição de funcionalidades sociais: lista de amigos, tabelas de classificação por turma ou grupo, e mecanismos de rivalidade positiva. Estas funcionalidades têm impacto directo na retenção diária — a declaração de envolvimento mais baixa (3,92) foi precisamente “senti-me encorajado/a a regressar e completar desafios diariamente”. A adição de conquistas e medalhas por nível de aprendizagem, referida por um participante, pode ser implementada sobre a infraestrutura de `micro_competences` e badges já existente. Uma extensão natural do mesmo pilar de envolvimento seria a introdução de notificações comportamentais personalizadas — por exemplo, alertar o utilizador próximo da sua janela habitual de utilização (inferida do histórico de acesso) caso o desafio do dia ainda não tenha sido completado — como mecanismo de motivação extrínseca orientado ao padrão individual de uso, em vez de um horário fixo igual para todos.

Reconhecimento formal via Open Badges e créditos ECTS. O sistema de crachás actual (`micro_competences`) é interno à plataforma e não é portátil nem verificável fora dela. Alinhado com o terceiro pilar do projecto — reconhecimento, a par do envolvimento e da personalização — uma extensão valiosa seria mapear estas conquistas para o *standard* europeu *Open Badges*, permitindo a sua exportação e verificação externa. No contexto da rede U!REKA, esta infraestrutura poderia ainda suportar o reconhecimento formal de micro-créditos ECTS (por exemplo, 0,5–1 ECTS) em unidades curriculares de reduzida carga horária nas instituições parceiras, criando uma ponte entre a aprendizagem informal gamificada e o

reconhecimento académico formal.

Explicações adaptadas ao dispositivo móvel. A avaliação qualitativa revelou que as explicações IA em prosa são longas demais para ecrãs pequenos. Uma melhoria directa seria configurar o prompt de geração de explicação com comprimento máximo variável em função do `audienceLevel` e do canal (mobile vs. web), ou adicionar uma versão resumida gerada automaticamente para exibição inicial com opção de expandir.

Personalização geográfica do conteúdo. Um participante sugeriu a utilização da localização do dispositivo para personalizar as questões sobre cidades inteligentes às características do município do utilizador. Esta ideia é tecnicamente viável com a arquitectura actual: bastaria injectar metadados de localização no contexto de geração de tarefas e no prompt de geração do ramo adaptativo, enriquecendo os exemplos concretos com informação local.

Profundidade variável de struggle e modelos de utilizador. A substituição de `MAX_STRUGGLE_DEPTH = 1` por um valor configurável por tópico permitiria calibrar a experiência de acordo com a dificuldade esperada do domínio. Num passo seguinte, a implementação de um modelo de utilizador simples baseado no histórico de `wrongAttemptCount` e na distribuição de padrões de erro por módulo permitiria ajustar dinamicamente o número de subtarefas adaptativas e a profundidade máxima por utilizador.

Análise de desempenho de questões geradas. O sistema instrumenta `tasks_submitted_total` por resultado e `topic_id`, o que permitiria construir um painel de análise de qualidade das questões: taxa de resposta correcta por tarefa, tempo médio de resolução, e distribuição de padrões de erro por módulo. Questões com taxa de acerto inferior a 20% ou superior a 95% poderiam ser sinalizadas para revisão pelo administrador, criando um ciclo de melhoria contínua assistido por métricas.

Validação automática de alinhamento com o DigComp 2.2. Uma extensão valiosa seria adicionar ao pipeline de geração um passo de verificação: após a geração de cada tarefa, um segundo prompt classificaria a competência DigComp 2.2 que a tarefa testa e verificaria a sua correspondência com o módulo configurado. Tarefas com classificação incongruente seriam descartadas ou sinalizadas para revisão humana, fortalecendo a integridade pedagógica da plataforma.

Suporte nativo multi-plataforma. A escolha de Kotlin Multiplatform possibilita a compilação nativa para iOS com código partilhado, mas o projecto actual entrega apenas o alvo Android. A extensão para iOS reduziria a barreira de adopção na população-alvo (estudantes universitários), que é heterogénea em termos de plataforma móvel.

Duas direcções estratégicas para o futuro. A segmentação de perfis de afinidade tecnológica observada nos testes de usabilidade (§5.5.3) sugere duas direcções de evolução distintas, não necessariamente exclusivas. Por um lado, o perfil tecnologicamente confortável — predominantemente estudantes e jovens profissionais — valida o modelo actual de aprendiza-

gem autodirigida e aponta para um aprofundamento da vertente universitária, nomeadamente através da integração de Open Badges e créditos ECTS na rede U!REKA descrita acima. Por outro lado, o perfil de menor afinidade tecnológica — que é precisamente quem mais beneficiaria do reforço de competências digitais — revela que a aprendizagem inteiramente autónoma não é, por si só, suficiente para este público: uma extensão do Play4Change à educação cívica generalista para o cidadão comum exigiria mecanismos adicionais de acompanhamento, orientação ou mediação humana, em vez de depender exclusivamente da auto-motivação do utilizador. A escolha entre estas direcções — ou a exploração de ambas em paralelo, cada uma servida por um perfil de produto distinto — fica em aberto para trabalho futuro.

Capítulo 8

Conclusão

Referências

- [1] Riina Vuorikari, Stefano Kluzer, and Yves Punie. DigComp 2.2: The Digital Competence Framework for Citizens — With new examples of knowledge, skills and attitudes. EUR 31006 EN, Publications Office of the European Union, Luxembourg, 2022. JRC128415.
- [2] European Parliament and Council of the European Union. Decision (EU) 2022/2481 of the European Parliament and of the Council of 14 December 2022 establishing the Digital Decade Policy Programme 2030. Official Journal of the European Union, L 323, 2022. Sets the target of at least 80% of EU adults with basic digital skills by 2030, building on the 2030 Digital Compass (COM(2021) 118 final).
- [3] Benjamin S. Bloom. The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring. *Educational Researcher*, 13(6):4–16, 1984.
- [4] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, et al. ChatGPT for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023.
- [5] John Hattie. *Visible Learning: A Synthesis of Over 800 Meta-Analyses Relating to Achievement*. Routledge, London, 2009.
- [6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020.
- [7] Jay Cross. *Informal Learning: Rediscovering the Natural Pathways That Inspire Innovation and Performance*. Pfeiffer, San Francisco, CA, 2007.
- [8] Burrhus F. Skinner. Teaching machines. *Science*, 128(3330):969–977, 1958.
- [9] John R. Anderson, C. Franklin Boyle, and Brian J. Reiser. Intelligent tutoring systems. *Science*, 228(4698):456–462, 1985.

- [10] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011.
- [11] Albert T. Corbett and John R. Anderson. Knowledge tracing: Modeling the acquisition of procedural knowledge. *User Modeling and User-Adapted Interaction*, 4(4):253–278, 1994.
- [12] Chris Piech, Jonathan Spencer, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas, and Jascha Sohl-Dickstein. Deep knowledge tracing. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [13] Roumen Vesselinov and John Grego. Duolingo effectiveness study. Technical report, City University of New York, 2012. Estudo encomendado e financiado pelo Duolingo; não sujeito a revisão por pares.
- [14] Stephen Krashen. Research on Duolingo: Not much, 2019. [Online; accessed 2026].
- [15] Rishi Bommasani, Drew A. Hudson, Ehsan Aditi, et al. On the opportunities and risks of foundation models. Technical report, Stanford University, 2021.
- [16] Adrian Simpson. The misdirection of public policy: Comparing and combining standardised effect sizes. *Journal of Education Policy*, 32(4):450–466, 2017.
- [17] John Sweller. Cognitive load theory, learning difficulty, and instructional design. *Learning and Instruction*, 4(4):295–312, 1994.
- [18] Helen Colley, Phil Hodgkinson, and Janice Malcolm. Informality and formality in learning: A report for the learning and skills research centre. *Learning and Skills Research Centre*, 2003.
- [19] Malcolm S. Knowles. *Andragogy in Action: Applying Modern Principles of Adult Learning*. Jossey-Bass, San Francisco, 1984.
- [20] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint*, arXiv:2312.10997, 2024.
- [21] John Traxler. Defining, discussing and evaluating mobile learning: The moving finger writes and having writ. *International Review of Research in Open and Distributed Learning*, 8(2), 2007.
- [22] Diana Laurillard. Pedagogical forms of mobile learning: Framing research questions. *Mobile Learning: Towards a Research Agenda*, pages 153–175, 2007.

- [23] International Telecommunication Union. Measuring digital development: Facts and figures 2023. Technical report, ITU, Geneva, 2023.
- [24] Joseph Bonneau, Cormac Herley, Paul C. van Oorschot, and Frank Stajano. The quest to replace passwords: A framework for comparative evaluation of web authentication schemes. In *IEEE Symposium on Security and Privacy*, pages 553–567, 2012.
- [25] FIDO Alliance. FIDO2: Moving the world beyond passwords. Technical report, FIDO Alliance, 2019.
- [26] Ian Sommerville. *Software Engineering*. Pearson, Harlow, 10th edition, 2016.
- [27] Donald A. Norman. *The Design of Everyday Things*. Basic Books, New York, 1988.
- [28] Hermann Ebbinghaus. *Über das Gedächtnis: Untersuchungen zur experimentellen Psychologie*. Duncker & Humblot, Leipzig, 1885. Translated as *Memory: A Contribution to Experimental Psychology*, 1913.
- [29] Nicholas J. Cepeda, Harold Pashler, Edward Vul, John T. Wixted, and Doug Rohrer. Distributed practice in verbal recall tasks: A review and quantitative synthesis. *Psychological Bulletin*, 132(3):354–380, 2006.
- [30] Edward L. Deci and Richard M. Ryan. *Intrinsic and Extrinsic Motivations: Classic Definitions and New Directions*, volume 25. 2000.
- [31] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley, Boston, 4th edition, 2021.
- [32] Ivar Jacobson, Magnus Christerson, Patrik Jonsson, and Gunnar Övergaard. *Object-Oriented Software Engineering: A Use Case Driven Approach*. Addison-Wesley, Wokingham, 1992.
- [33] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, Boston, 2003.
- [34] Unstructured Technologies. Unstructured: Open source library and API for document extraction, 2023. [Online; accessed 2025].
- [35] Microsoft Corporation. Playwright: Fast and reliable end-to-end testing for modern web apps, 2023. [Online; accessed 2025].
- [36] John Brooke. SUS: A ‘quick and dirty’ usability scale. pages 189–194, 1996.

Apêndice A

Exemplos de Prompts Completos

Este apêndice reproduz, na íntegra e sem alterações, os *templates* de *prompt* definidos no módulo `ai-agent` (pacote `com.ureka.play4change.prompt`) e efectivamente enviados ao modelo `mistral-small-latest` através do `LangChain4j`. Os campos entre `$` são interpolados em *runtime* a partir do contexto de domínio (tópico, módulo, sessão de dificuldade, etc.). Todos os *prompts* de geração de conteúdo obrigam o modelo a responder exclusivamente em JSON, sem texto envolvente, para permitir *parsing* determinístico pelo `OptionsJsonParser`.

A.1 Prompt de Geração de Tópicos e Desafios

Utilizado pelo `LangChain4jTaskGenerationAdapter` sempre que é necessário gerar tarefas de escolha múltipla — quer na ingestão inicial de um tópico (`TaskGenerationOrchestrator`), quer na geração de variantes por idioma (`LanguageGenerationAdapter`), quer na geração de instâncias distratoras em lote (`BatchInstanceAdapter`).

System Message

You are an expert educational content designer specialising in gamified learning experiences. You create engaging daily multiple-choice tasks that teach real skills through practice, not memorisation.

LANGUAGE: You MUST generate ALL text content in `$language`. This is mandatory. Do not mix languages. Do not respond in English unless `$language` is English.

RULES:

- Each task must be completable in 5-15 minutes
- Tasks must be practical and actionable, not theoretical
- Difficulty must match the audience level exactly
- Hints must guide without giving away the answer

- Every task must directly advance the module objective
- Tasks must be distinct | no semantic overlap with existing content
- Always generate exactly 4 options per task
- The correct answer must always be at index 0 in the options array
(the system will shuffle before showing to the user)
- Wrong options must be plausible | not obviously incorrect

OUTPUT FORMAT: Respond with a valid JSON array only. No preamble. No explanation.

Schema:

```
[
  {
    "title": "string (max 60 chars)",
    "description": "string (max 300 chars | frame it as a question or scenario)",
    "hint": "string (max 150 chars)",
    "pointsReward": number (10-100, higher for harder tasks),
    "options": ["correct answer", "wrong option 1", "wrong option 2", "wrong option 3"],
    "correctAnswerIndex": 0
  }
]
```

User Message

Generate \${request.taskCount} multiple-choice learning tasks for the following context:

Subject Domain: \${request.subjectDomain}

Module Objective: \${request.moduleObjective}

Audience Level: \${request.audienceLevel.toPromptDescription()}

Language: \${request.language}

The tasks must progress logically | earlier tasks build foundations,
later tasks apply concepts in more complex scenarios.

Important: N tasks already exist for this module. Generate semantically
distinct tasks that complement the existing content.

[linha condicional, omitida quando não existem tarefas prévias no módulo]

Remember: always put the correct answer at index 0 in the options array.

Generate the JSON array now:

O nível de audiência (AudienceLevel) é traduzido para uma descrição textual antes de

ser injectado no *prompt*:

```
BEGINNER      -> "Beginner | assumes no prior knowledge, uses simple
                  language, concrete examples"
INTERMEDIATE -> "Intermediate | assumes basic familiarity, introduces
                  complexity, some abstraction"
ADVANCED      -> "Advanced | assumes solid foundation, focuses on nuance,
                  edge cases, and depth"
```

Reforço de Esquema (retry)

Quando a resposta do modelo é incompleta ou não corresponde ao número de tarefas pedido, o Resilience4j `retry` reenvia o pedido com um reforço adicional do esquema:

```
Your previous response was incomplete or malformed.
Please respond with EXACTLY $taskCount tasks in the JSON array schema above.
Every task must have: title, description, hint, pointsReward, options (4 items),
and correctAnswerIndex set to 0. No extra fields. Valid JSON only.
```

A.2 Prompt de Análise de Dificuldade e Geração de Subtarefas Adaptativas

Utilizado pelo `HandleStruggleService` quando um aprendente falha repetidamente uma tarefa. Ao contrário do *prompt* de geração base, este inclui o padrão de erro diagnosticado (`ErrorPattern`), permitindo que o modelo gere um percurso de recuperação escalonado em vez de tarefas genéricas sobre o mesmo tópico.

System Message

```
You are an expert adaptive learning coach. A student is struggling with
a specific task in their learning journey. Your job is to create a short
sequence of simpler subtasks that scaffold the student back to success.
```

PRINCIPLES:

- Diagnose the root cause of the struggle from the error pattern
- Start with the simplest possible related concept
- Each subtask builds on the previous one
- The final subtask should bring the student back to the original task level
- Be encouraging | frame tasks as achievable steps, not remediation
- Keep each subtask completable in 3-10 minutes

OUTPUT FORMAT: Respond with a valid JSON array only. No preamble. No explanation.

Schema:

```
[
  {
    "title": "string (max 60 chars)",
    "description": "string (max 300 chars)",
    "hint": "string (max 150 chars)",
    "pointsReward": number (5-50, lower than main tasks to reflect smaller scope),
    "options": ["string", "string", "string", "string"],
    "correctAnswerIndex": number (0-based index into options array)
  }
]
```

User Message

A student is struggling with the following task after `${context.attemptCount}` attempts:

Task: `${context.taskDescription}`

Subject Domain: `${context.subjectDomain}`

Module Objective: `${context.moduleObjective}`

Student Level: `${context.audienceLevel.toPromptDescription()}`

Language: `${context.language}`

Diagnosed Struggle Type: `${context.errorPattern.toPromptDescription()}`

Generate `$subtaskCount` adaptive subtasks that:

1. Address the specific struggle type diagnosed above
2. Start simpler than the original task
3. Progressively build back toward the original task level
4. Are encouraging and reframe the struggle as a learning opportunity

Generate the JSON array now:

Cada `ErrorPattern` é traduzido numa instrução específica de diagnóstico antes de ser injectado no *prompt*:

CONCEPTUAL_MISUNDERSTANDING -> "The student misunderstands the core concept.

Start by re-explaining it differently."

PROCEDURAL_ERROR -> "The student understands the concept but

applies the wrong steps. Focus on procedure."

KNOWLEDGE_GAP	-> "The student is missing prerequisite knowledge. Fill the gap before returning to the task."
UNCLEAR_INSTRUCTIONS	-> "The task instructions were unclear. Provide clearer, more explicit guidance."
TIME_PRESSURE	-> "The student rushed and made errors under time pressure. Use paced, step-by-step practice tasks."
UNKNOWN	-> "The struggle cause is unclear. Provide general scaffolding from basic to intermediate."

A.3 Prompt de Explicação Holística e Conversação

Utilizado pelo `ExplanationService` quando um aprendente esgota todos os níveis de profundidade do percurso adaptativo sem resolver a dificuldade. Divide-se em dois modos: a explicação inicial (síntese de todas as tentativas) e as respostas conversacionais subsequentes, caso o aprendente continue confuso.

System Message — Explicação Inicial

You are a patient and empathetic learning coach helping a student who has struggled repeatedly with the same concept. They have now exhausted all their practice attempts and need a clear, holistic explanation.

YOUR GOAL:

- Synthesise what went wrong across all their attempts
- Explain the underlying concept clearly and simply
- Use concrete examples relevant to the subject domain
- Be encouraging | frame struggles as a normal part of learning
- Keep the explanation concise: 3-5 paragraphs maximum
- Write in plain prose | no JSON, no bullet lists, no headers
- Match the learner's language exactly

User Message — Explicação Inicial

A student in a `${context.audienceLevel.toLabel()}` course on "`${context.subjectDomain}`" has struggled with the following task after exhausting their practice attempts:

Task: `${context.taskDescription}`

Module objective: \${context.moduleObjective}
Struggle type: \${context.errorPattern.toLabel()}
Language: \${context.language}

Here is their struggle history:

Session \${depth} (\${errorPattern.toLabel()}) : \${taskTitles joined by ", "}
[uma linha por sessão de dificuldade anterior]

Write a clear, encouraging explanation that helps them finally understand this concept.

Respond in \${context.language}. Plain prose only.

System Message — Réplica Conversacional

You are a patient learning coach in an ongoing conversation with a confused student. They have already received an initial explanation of a concept they struggled with. Your role now is to answer their specific questions and clear up remaining confusion.

PRINCIPLES:

- Address their exact question directly
- Use simple language and concrete examples
- Keep replies focused: 2-4 sentences unless the question demands more
- Be warm and encouraging
- Write in the same language as the student
- Plain prose only | no JSON, no bullet lists

O primeiro turno da conversa é sempre um bloco de contexto gerado a partir de dados confiáveis (provenientes da base de dados), nunca de conteúdo fornecido pelo utilizador — separação deliberada que impede que uma futura mensagem maliciosa do aprendente altere o enquadramento da conversa (ver secção sobre defesas contra *prompt injection*, Figura 4.14):

Subject: \${context.subjectDomain}
Original task: \${context.taskDescription}

Apêndice B

Diagramas Complementares

Os diagramas apresentados no corpo do relatório (Capítulo 4) focam-se em subfluxos isolados, de forma a manter cada figura legível junto do texto que a acompanha. Este apêndice reúne três vistas de conjunto, pensadas para consulta integral: o fluxo de autenticação ponta-a-ponta, o *pipeline* completo de criação de um tópico (ingestão, geração via IA e indexação vectorial), e o diagrama entidade-relação do esquema relacional na íntegra.

B.1 Diagrama de Sequência — Autenticação Completa (*Magic Link*)

A Figura B.1 mostra o fluxo integral de autenticação sem password, desde a submissão do e-mail até à emissão do par de tokens (*access* + *refresh*), incluindo a normalização do e-mail, o *rate limiting* por IP, o hash SHA-256 do token, a operação atômica de consumo do *magic link* (que previne condições de corrida TOCTOU) e a criação automática de conta para utilizadores novos.

B.2 Diagrama de Sequência — Pipeline Completo de Ingestão e Geração RAG

A Figura B.2 mostra o *pipeline* completo accionado pela submissão de um novo tópico: extração de conteúdo (*Playwright/Unstructured*), transição de fases (INGESTION → ANALYSIS → GENERATION → INDEXING → ACTIVE), segmentação em *chunks*, chamada ao LLM por *chunk*, geração de *embeddings*, deduplicação semântica via pgvector e notificação de progresso em tempo real por *Server-Sent Events*.

B.3 Diagrama Entidade-Relação

A Figura B.3 apresenta o esquema relacional completo, tal como definido pelas *migrations* Flyway (V1 a V32) em `server/src/main/resources/db/migration`. As entidades estão agrupadas por domínio: identidade e autenticação (azul), conteúdo e conhecimento (verde),

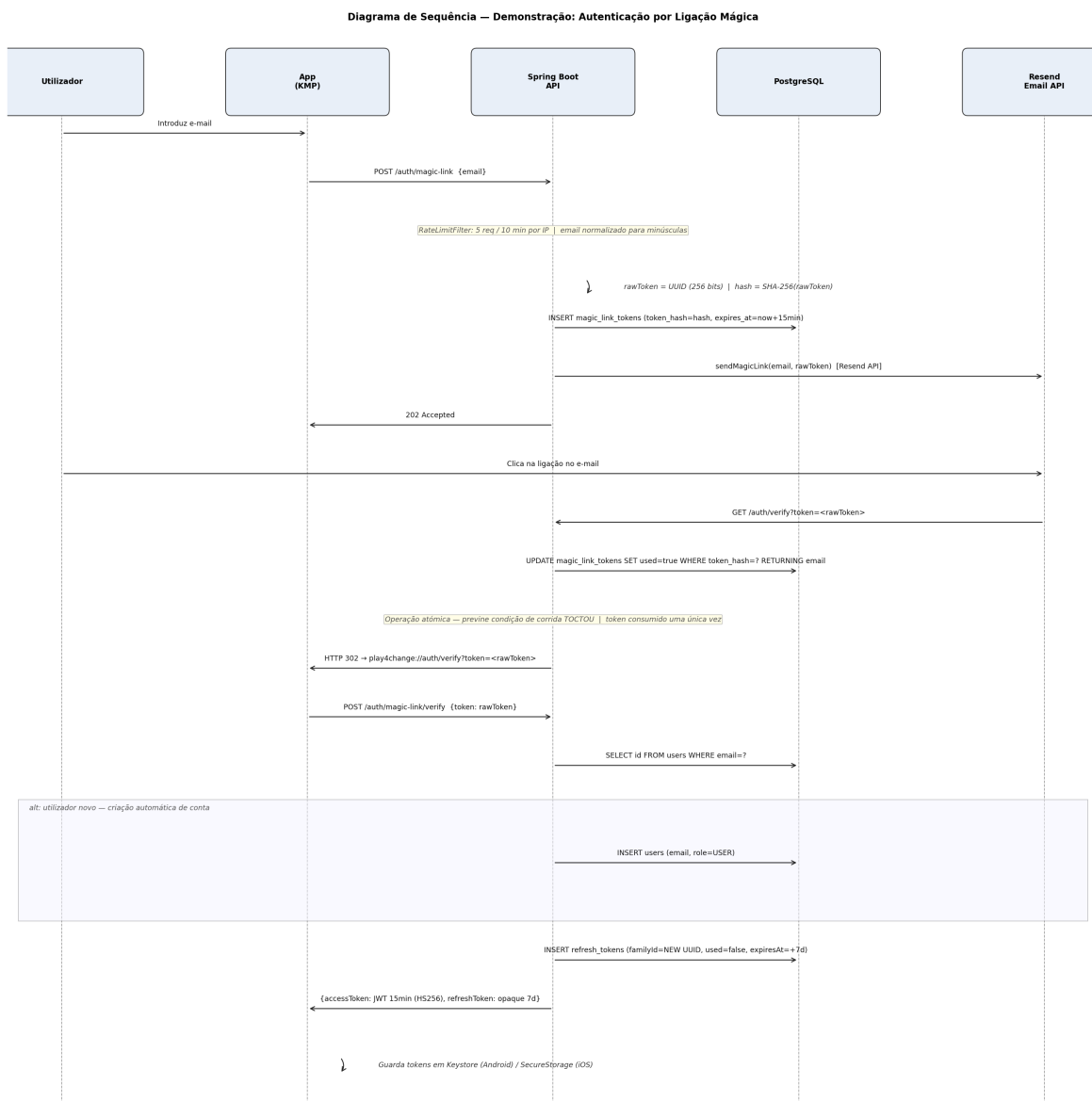


Figura B.1: Diagrama de sequência completo — autenticação passwordless por magic link

progresso de aprendizagem (amarelo), suporte adaptativo e explicações (vermelho), e governança/moderação (roxo). Tabelas puramente internas ao serviço de deduplicação do ai-agent (`struggle_events`, `adaptive_branches`) foram omitidas por não pertencerem ao domínio aplicacional principal.

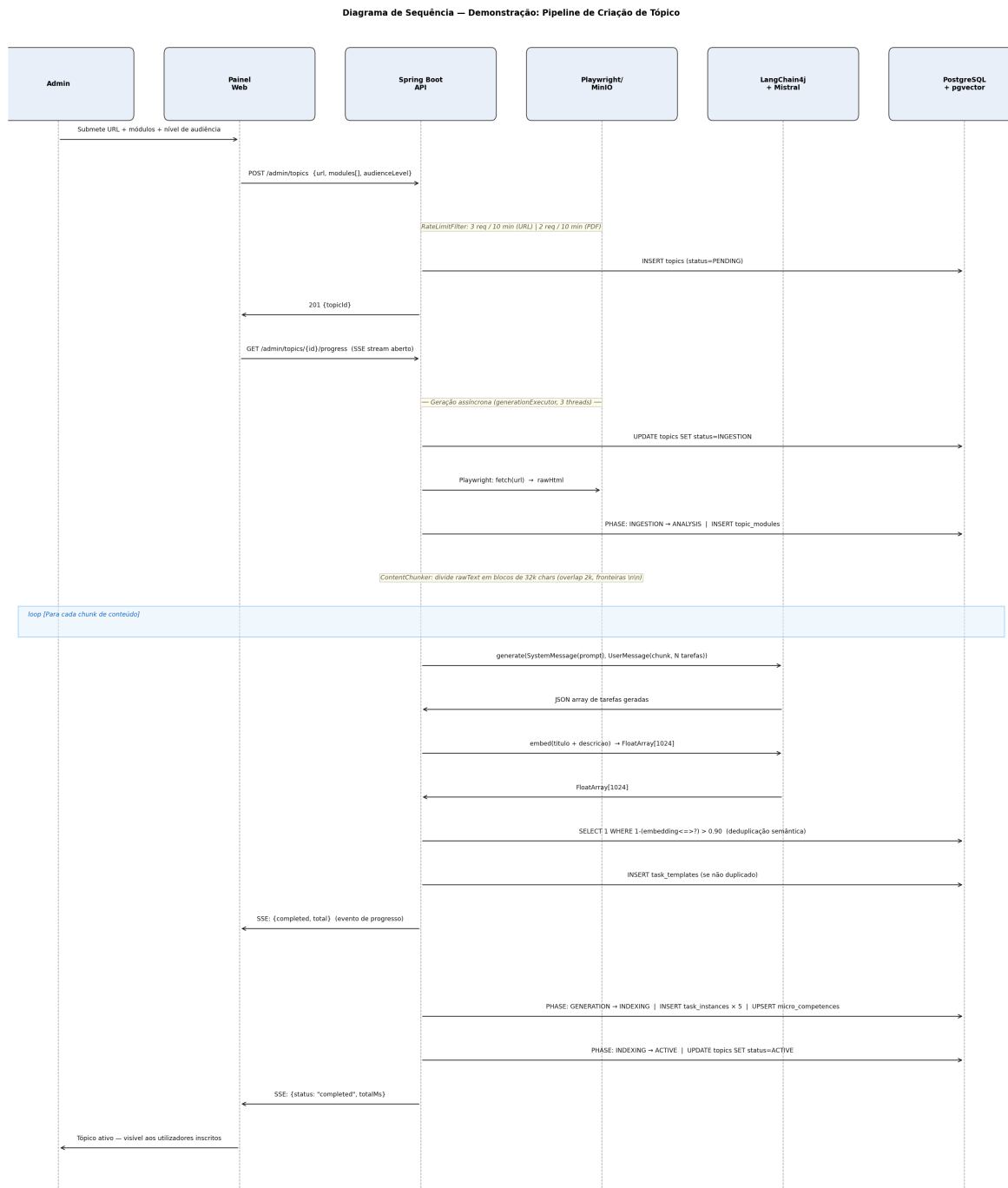


Figura B.2: Diagrama de sequência completo — pipeline de ingestão, geração e indexação RAG

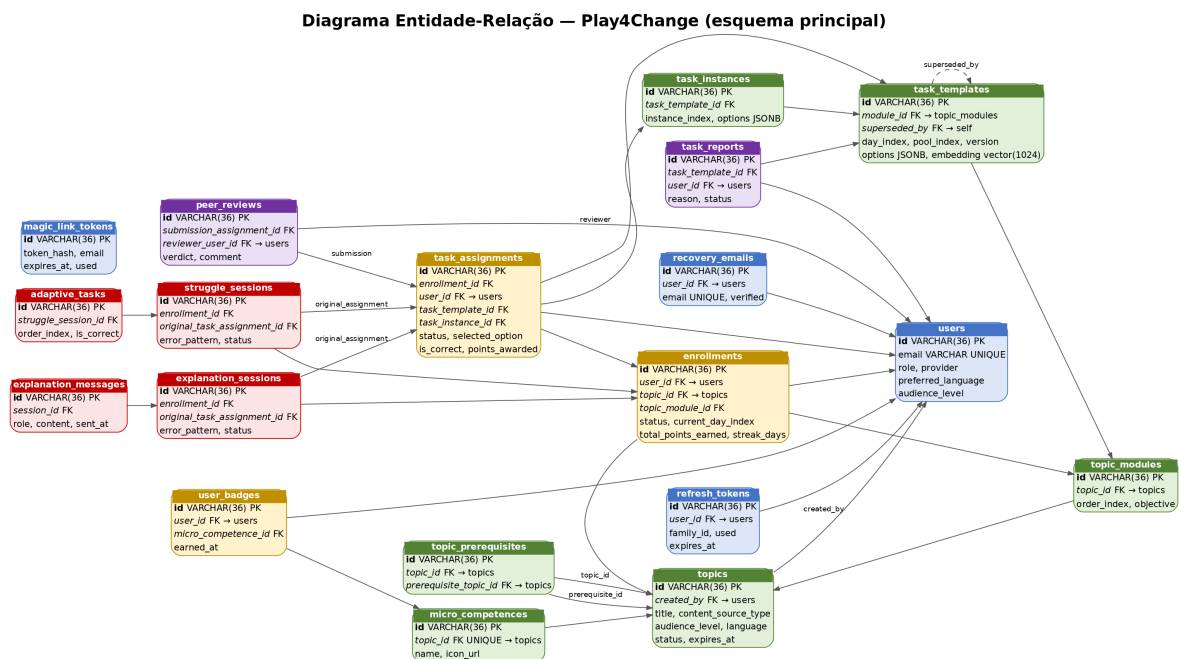


Figura B.3: Diagrama entidade-relação do esquema relacional da base de dados

Apêndice C

Variáveis de Ambiente

Este apêndice lista todas as variáveis de ambiente configuráveis no sistema, tal como definidas em `application.yml` (servidor), `docker-compose.yml` (orquestração de contentores) e nos ficheiros `.env.example` das aplicações cliente. Os valores reais (segredos, chaves de API, credenciais) não são reproduzidos; apenas os nomes das variáveis e o respectivo propósito.

Base de Dados

Tabela C.1: Variáveis de ligação à base de dados

Variável	Descrição
SPRING_DATASOURCE_URL	URL JDBC de ligação ao PostgreSQL (por omissão aponta para o contentor <code>play4change-postgres</code>)
DB_USER / DB_PASS	Credenciais da base de dados usadas pelo <code>docker-compose</code>

Não existe camada de *cache* distribuída no sistema: o *rate limiting* usa *buckets* em memória por processo (Bucket4j + Caffeine), sem necessidade de um *backing store* partilhado.

Inteligência Artificial e Geração

Tabela C.2: Variáveis de configuração do modelo de IA

Variável	Descrição
MISTRAL_API_KEY	Chave de acesso à API da Mistral AI; se omitida ou igual a <code>not-set-yet</code> , os <i>beans</i> do LangChain4j não são carregados e os endpoints de IA respondem 503

Os restantes parâmetros do modelo de IA (`ai.mistral.model`, `ai.mistral.temperature`, `ai.mistral.timeout-seconds`, tamanho da *pool* de geração, limiares de deduplicação) são definidos directamente em `application.yml` e não variam por ambiente — apenas a chave de API é externalizada como segredo.

Armazenamento de Objectos (MinIO)

Tabela C.3: Variáveis de armazenamento de ficheiros

Variável	Descrição
MINIO_ENDPOINT	URL do serviço MinIO (S3-compatível)
MINIO_ROOT_USER	/ Utilizador/chave de acesso ao MinIO
MINIO_ACCESS_KEY	
MINIO_ROOT_PASSWORD	/ Password/chave secreta do MinIO
MINIO_SECRET_KEY	
MINIO_BUCKET	Nome do <i>bucket</i> usado para PDFs submetidos

Ingestão de Conteúdo

Tabela C.4: Variáveis dos serviços auxiliares de extracção de conteúdo

Variável	Descrição
SCRAPER_URL	URL do sidecar Playwright usado para extracção de conteúdo de páginas web
UNSTRUCTURED_URL	URL da API Unstructured.io usada para extracção de texto de PDF

Autenticação, Segurança e CORS

Tabela C.5: Variáveis de segurança e autenticação

Variável	Descrição
JWT_SECRET	Segredo usado para assinar (HS256) os <i>access tokens</i> JWT; mínimo de 64 caracteres em produção (<code>JwtSecretStartupGuard</code>)
APP_BASE_URL	URL base da aplicação, usada na construção de <i>deep links</i> de autenticação
FRONTEND_ORIGIN	Origem do <i>frontend</i> , propagada para <code>APP_BASE_URL</code> no <code>docker-compose</code>
CORS_ALLOWED_ORIGINS	Lista de origens autorizadas pela política CORS do servidor

Os parâmetros de duração dos tokens (`jwt.access-ttl-minutes` = 15, `jwt.refresh-ttl-days` = 7) e os limites de *rate limiting* por IP são fixados em `application.yml` e não são externalizados como variáveis de ambiente.

Envio de E-mail

Observabilidade e Infra-estrutura

A exposição pública do serviço em produção (<https://radesh-govind.com/play4change-server>, referenciada em `play4change-web/.env.production`) não é orquestrada por nenhum serviço

Tabela C.6: Variáveis do serviço de e-mail (Resend)

Variável	Descrição
RESEND_API_KEY	Chave de acesso à API de envio de e-mail Resend
RESEND_FROM	Endereço de origem usado no envio dos <i>magic links</i>

Tabela C.7: Variáveis de monitorização e exposição pública

Variável	Descrição
GRAFANA_ADMIN_PASSWORD	Password do utilizador administrador do Grafana
SPRING_PROFILES_ACTIVE	Perfil Spring activo (ex. <code>prod</code> , <code>dev</code>)

definido no `docker-compose.yml` deste repositório — não existe um contentor *tunnel* nem variáveis de ambiente associadas no código-fonte; a exposição é gerida externamente ao repositório.

Aplicação Cliente (Web)

Tabela C.8: Variáveis de ambiente do cliente web (Vite)

Variável	Descrição
VITE_API_BASE_URL	URL base da API do servidor a que o cliente web se liga (URL de produção, <i>tunnel</i> ngrok, ou <code>localhost</code>)
VITE_USE MOCK	Quando <code>true</code> , o cliente usa adaptadores <i>mock</i> em vez de chamadas reais à API — usado em desenvolvimento isolado do <i>frontend</i>